

목차

YOLOv2 FPGA 가속기—학습 해설서	7
0. 이 해설서를 읽기 전에	7
Part 0. 기초 개념 (Foundations)	8
Part 1. 프로젝트와 환경	8
Part 2. 소프트웨어 골든 레퍼런스	9
Part 3. 하드웨어 큰 그림	9
Part 4. RTL 모듈 상세 (코드 해설 중심)	9
Part 5. 동작과 타이밍	10
Part 6. 검증 (Testbench)	10
Part 7. Vivado 프로젝트	10
Part 8. 부록	10
진행 상태	10
1 장. CNN 과 합성곱 기초	11
1.1 신경망“추론”이란 무엇인가	11
1.2 특징맵과 채널—데이터의 모양	11
1.3 합성곱 (Convolution) —CNN 의 심장	12
1.4 stride 와 padding	13
1.5 1×1 합성곱 vs 3×3 합성곱	13
1.6 활성화 함수—ReLU	14
1.7 풀링 (Pooling)	14
1.8 업샘플 (Upsample)	15
1.9 이 가속기가 다루는 연산 총정리	15
1.10 합성곱은 곧 곱셈-누적 (MAC) 의 산이다	16
1.11 이 장의 요약	16
2 장. 신경망 양자화 기초	17
2.1 왜 부동소수점 대신 정수인가?	17
2.2 핵심 아이디어—scale 로 정수에 매핑하기	18
2.3 이 프로젝트의 양자화 방식	18
2.4 INT8 곱셈-누적—누적은 왜 INT32 인가	19
2.5 bias 더하기	19
2.6 Descaling —다시 작게 만들기	19

2.7 반올림—음수에서 생기는 1 LSB 차이	20
2.8 클리핑—0~255 로 자르기	21
2.9 전체 흐름 한눈에 보기	21
2.10 양자화 오차와 mAP	22
2.11 이 장의 요약	22
3 장. FPGA 하드웨어 기초	23
3.1 FPGA 란 무엇인가	23
3.2 기본 부품 (1) —LUT 와 플립플롭 (FF)	24
3.3 기본 부품 (2) —DSP48 (곱셈기)	24
3.4 기본 부품 (3) —BRAM (블록 메모리)	24
3.5 면적·속도·전력의 트레이드오프	25
3.6 RTL 에서 비트스트림까지	26
3.7 타이밍 클로저 (Timing Closure)	26
3.8 이 프로젝트의 FPGA —Artix-7 XC7A100T	26
3.9 이 장의 요약	27
4 장. RTL 설계와 타이밍 기초	28
4.1 RTL 과 클럭—조합 논리 vs 순차 논리	28
4.2 FSM —상태 머신	29
4.3 파이프라인—왜 계산을 쪼개는가	30
4.4 BRAM read latency 가 만드는 문제	30
4.5 valid 신호—데이터가 진짜인지 표시	31
4.6 signed 연산—부호를 잊으면 큰일	31
4.7 uninitialized X —시뮬레이션의 함정	32
4.8 왜 이 설계의 타이밍이 특히 까다로운가	32
4.9 이 장의 요약	32
5 장. AXI 버스 프로토콜 기초	34
5.1 버스란 무엇인가	34
5.2 Master 와 Slave	34
5.3 Handshake —valid 와 ready 의 악수	34
5.4 AXI4-Lite —간단한 제어용	35
5.5 AXI4 (Full) —대량 데이터용 burst	35
5.6 DMA —CPU 없이 메모리를 옮기기	36
5.7 이 프로젝트의 AXI 사용 정리	36
5.8 이 장의 요약	36
6 장. YOLO 객체 탐지 기초	38
6.1 객체 탐지란—분류와의 차이	38
6.2 Grid 기반 탐지—이미지를 격자로	38
6.3 Anchor Box —박스 모양의 후보	39
6.4 한 예측에 담기는 정보	39

6.5 출력 채널 195 의 정체	39
6.6 2-Scale 탐지와 Skip Connection	40
6.7 NMS —중복 박스 정리	41
6.8 하드웨어 vs 소프트웨어 분담	42
6.9 이 장의 요약	42
7 장. 프로젝트 개요	43
7.1 이 프로젝트를 한 문장으로	43
7.2 목표와 점수 공식	43
7.3 타겟 하드웨어	44
7.4 시스템 큰 그림	44
7.5 하드웨어와 소프트웨어의 역할 분담	45
7.6 4 단계 개발 로드맵	46
7.7 핵심 설계 결정 미리 보기	46
7.8 이 장의 요약	47
8 장. 개발 환경과 빌드	48
8.1 왜 두 개의 환경인가	48
8.2 SMB 공유와 경로 차이	49
8.3 프로젝트 디렉토리 구조	49
8.4 skeleton C 레퍼런스 빌드 (Linux)	49
8.5 골든 hex 파일 생성	50
8.6 Vivado 합성 (Windows)	50
8.7 Vivado 시뮬레이션 (검증)	50
8.8 FPGA 매크로—가장 자주 하는 실수	51
8.9 검증은 Vivado 2025 로	51
8.10 흔한 함정 체크리스트	51
8.11 이 장의 요약	52
9 장. skeleton C 골든 레퍼런스	53
9.1 skeleton 의 두 가지 역할	53
9.2 핵심 함수—forward_convolutional_layer_q()	53
9.3 양자화 계수—do_quantization()	55
9.4 scale → RTL shift —save_quantized_model()	55
9.5 골든 데이터 워크플로우	56
9.6 이 장의 요약	57
10 장. 22-Layer 네트워크 구조	58
10.1 cfg 파일 읽는 법	58
10.2 cfg 를 레이어로 펼치기	58
10.3 네트워크 토폴로지 (데이터 흐름)	59
10.4 레이어별 상세 표	60
10.5 연산은 세 종류로 나뉜다	61

10.6 출력 채널 195 의 의미 (복습)	62
10.7 연산량—어디에 시간이 쓰이나	62
10.8 이 장의 요약	62
11 장. 하드웨어 아키텍처 개요	64
11.1 19 개의 RTL 파일	64
11.2 모듈은 어떻게 포개져 있나 (계층)	65
11.3 데이터는 어떻게 흐르나	65
11.4 144-MAC —가속의 심장 (미리 보기)	66
11.5 어느 레이어를 어느 모듈이 처리하나	67
11.6 온칩 메모리—무엇을 어디에 두나	67
11.7 이 장의 요약	68
12 장. 데이터 표현과 메모리 맵	69
12.1 수 표현—비트폭이 단계마다 커진다	69
12.2 특징맵의 세 가지 옷	70
12.3 REPACK —옷 갈아입히기	71
12.4 가중치는 어떻게 저장되나	71
12.5 bias 는 sign-extend 로	71
12.6 dscaling 과 clamp (코드 미리보기)	72
12.7 DRAM 메모리 맵—무엇이 어디에	72
12.8 온칩 메모리 맵	73
12.9 이 장의 요약	73
13 장. RTL —yolo_engine (최상위 FSM)	74
13.1 핵심 사상—하나의 일꾼, 22 번 재사용	74
13.2 포트—무엇이 드나드나	74
13.3 레이어 파라미터 테이블	75
13.4 “지금 몇 번 레이어?”—phase 레지스터	75
13.5 53-state FSM —전체 지도	76
13.6 합성곱 레이어 실행 흐름 (가장 중요)	76
13.7 제어 레지스터—CPU 와의 약속	78
13.8 DMA 목적지 분배	79
13.9 출력 버퍼 5 방향 분배	79
13.10 Route 와 레이어 스킵	80
13.11 이 장의 요약	80
14 장. RTL —Convolution 엔진 (코드 해설)	81
14.1 5 단 계층—곱셈기부터 제어까지	81
14.2 mul.v —INT8 곱셈기 한 개	82
14.3 add_tree_36in.v —36 개를 하나로	83
14.4 mac_stack.v —144 개 MAC 한꺼번에	83
14.5 mac_kern.v —누적과 후처리를 붙이다	84

14.6 post_process.v —bias·ReLU·시프트·클립	85
14.7 conv_top.v —합성곱 한 호출 제어	86
14.8 전체 latency 정리	87
14.9 3×3 과 1×1 이 같은 엔진을 쓰는 비결	87
14.10 이 장의 요약	88
15 장. RTL —메모리 버퍼 (코드 해설)	89
15.1 ifm_line_buf.v —입력 윈도우 공장	89
15.2 gbuff_param.v —가중치·bias 창고	91
15.3 dpram_wrapper.v / spram_wrapper.v —BRAM 껍데기	92
15.4 네 버퍼 비교	92
15.5 이 장의 요약	93
16 장. RTL —특수 연산 유닛 (코드 해설)	94
16.1 공통 패턴—load → process → store	94
16.2 max_pool_unit.v —stride-2 풀링	94
16.3 max_pool_s1_unit.v —stride-1 풀링 (L11 전용)	95
16.4 upsample_unit.v —2× 업샘플 (L18 전용)	96
16.5 REPACK 엔진—포맷 갈아입히기	97
16.6 특수 유닛 비교	97
16.7 이 장의 요약	98
17 장. RTL —AXI / DMA 인터페이스 (코드 해설)	99
17.1 통신 경계 전경	99
17.2 axi_dma_rd.v —DRAM 읽기 DMA	99
17.3 axi_dma_wr.v —DRAM 쓰기 DMA	100
17.4 yolo_engine_axi.v —CPU 제어 받기	101
17.5 user_define_h.v / define.v —매크로	101
17.6 Phase 3 —실제 SoC 통합 (전망)	102
17.7 이 장의 요약	103
18 장. 동작 과정과 타이밍	104
18.1 전체 추론 시퀀스	104
18.2 합성곱 한 레이어의 cycle 흐름	105
18.3 MAC 파이프라인 타이밍—latency vs throughput	105
18.4 BRAM latency 정렬—중합	106
18.5 특수 레이어 타이밍	106
18.6 DMA 타이밍	106
18.7 성능 추정	107
18.8 Energy —점수의 분모	107
18.9 이 장의 요약	108
19 장. 검증 전략과 인프라	109
19.1 검증 철학—골든과 비교	109

19.2 3 계층 검증—작은 것부터	109
19.3 2-Phase 검증—같은 레이어를 두 방식으로	110
19.4 통일 로그 포맷	111
19.5 Tolerance 와 delta 분포—디버깅의 단서	111
19.6 가짜 DRAM 모델	112
19.7 Software REPACK —골든 입력 준비	112
19.8 이 장의 요약	112
20 장. Layer 별 / 블록 Testbench	114
20.1 Layer verify TB 목록	114
20.2 검증 결과—L11 이 마법을 부린다	115
20.3 주요 TB 깊이 보기	115
20.4 블록 TB —모듈 하나씩	116
20.5 빠른 검증—iverilog	117
20.6 이 장의 요약	117
21 장. Vivado 프로젝트	118
21.1 프로젝트 개요	118
21.2 프로젝트 생성—create_project_25.tcl	118
21.3 BRAM IP —gen_bram_ips.tcl	119
21.4 제약—NEXYS_A7_100T.xdc	119
21.5 합성 / 시뮬 흐름	120
21.6 Vivado 2021 vs 2025 —검증 정확성의 함정	120
21.7 이 장의 요약	121
22 장. 부록—향후 작업 / 용어집	123
22.1 L19~L21 —검증 진행 중	123
22.2 Phase 3 —MicroBlaze + UART + DDR2	124
22.3 Phase 4 —비트스트림 / 보드 / 측정	125
22.4 용어집	126
22.5 약어	127
22.6 참고 자료와 우선순위	127
22.7 문서 유지보수 가이드	128
22.8 맷음말—전체 한 장으로	128

YOLOv2 FPGA 가속기—학습 해설서

팀 베타트론·중앙대학교 AIX2026 Deep Learning Hardware 설계 경진대회
22-layer YOLOv2 추론 가속기 SoC (Nexys A7-100T / Artix-7 XC7A100T)

0. 이 해설서를 읽기 전에

0.1 이 문서는 무엇인가요?

이 문서는 프로젝트를 처음 접하는 학부생이 **RTL**의 구조와 동작을 스스로 이해할 수 있도록 만든 학습 해설서입니다. 단순히 “무엇이 있는지”를 나열하는 것이 아니라, “왜 이렇게 만들었는지”, “이 코드가 정확히 무슨 일을 하는지”를 처음부터 끝까지 따라갈 수 있게 설명합니다.

이 프로젝트에는 두 종류의 문서가 있습니다.

문서	위치	대상	성격
기술 레퍼런스	../technical_reference/Claude(AI)·숙련자		간결·정확, 찾아보는 용
학습 해설서 (본 문서)	study_guide/	사람·학부생	배경지식 + 코드 해설, 읽고 배우는 용

두 문서는 같은 시스템을 다루지만, 본 해설서는 **배경 지식을 충실히 깔고, 설명을 두 배로 자세히 하며, 중요한 코드를 직접 짚어가며** 설명합니다. 깊은 정확성이 필요하면 기술 레퍼런스를, 처음 배운다면 본 해설서를 권합니다.

0.2 독자에게 가정하는 것

- **알고 있다고 가정:** 디지털 논리 회로 (게이트·플립플롭·레지스터), Verilog 기본 문법 (reg/wire, always, 비트 표기 8'd5, if/case), 2 진수·16 진수, 기초 프로그래밍 (C/Python).
- **처음이라고 가정하고 설명:** 합성곱 신경망 (CNN), 신경망 양자화, FPGA 자원 (DSP48·BRAM), RTL 파이프라인·타이밍, AXI 버스 프로토콜, YOLO 객체 탐지. → 이 6 가지는 Part 0 기초 개념에서 차근차근 설명합니다.

0.3 읽는 순서

처음 배우는 사람: Part 0 → Part 1 → ... → Part 8 (순서대로)
HW 지식이 있는 사람: Part 0 건너뛰고 Part 1 부터, 필요할 때 Part 0 참조
특정 모듈만 볼 사람: 해당 장으로 직행, 모르는 개념은 Part 0 링크 따라가기

각 기술 장은 시작 부분에 “이 장을 읽기 위한 준비”로 필요한 Part 0 개념을 링크합니다.

0.4 이 문서의 표기 약속

본 해설서는 이해를 돕기 위해 몇 가지 장치를 씁니다.

- **[비유]** 비유 박스: 어려운 하드웨어 개념을 일상적인 비유로 풀어줍니다.
- **[코드]** 코드 해설: 실제 RTL 코드를 인용하고 한 줄 한 줄 무슨 일을 하는지 설명합니다.
- **[왜?] 왜 이렇게 했을까?:** 설계 결정의 이유와 대안을 설명합니다.
- **[주의]** 흔한 오해 / 함정: 처음 배울 때 헷갈리기 쉬운 지점을 짚습니다.
- 코드 식별자는 `i_start`, `conv_top` 처럼 표기하고, 파일은 `yolo_engine.v` 처럼 링크합니다.

Part 0. 기초 개념 (Foundations)

RTL 을 읽기 전에 알아야 할 6 가지 배경 지식입니다. 이미 아는 주제는 건너뛰어도 됩니다.

장	파일	무엇을 배우나
1	CNN 과 합성곱 기초	합성곱·풀링·활성화·채널·stride·padding, 1×1 vs 3×3, 이 가속기가 다루는 연산
2	신경망 양자화 기초	왜 정수로? 고정소수점, scale, INT8 곱셈-누적, dscaling, 반올림
3	FPGA 하드웨어 기초	LUT·FF·DSP48·BRAM 이 뭔지, 합성/구현/비트스트림, 클럭과 면적·전력
4	RTL 설계와 타이밍 기초	FSM, 파이프라인, BRAM read latency, valid 신호, signed 연산, 왜 타이밍이 까다로운가
5	AXI 버스 프로토콜 기초	master/slave, AXI4-Lite vs AXI4, burst, handshake, DMA
6	YOLO 객체 탐지 기초	객체 탐지란, grid·anchor·bounding box, 검출 헤드, NMS, 출력 195 채널의 의미

Part 1. 프로젝트와 환경

장	파일	내용
7	프로젝트 개요	대회·목표·점수 공식, 시스템 큰 그림, HW/SW 역할 분담
8	개발 환경과 빌드	Linux/Windows 이중 환경, skeleton 빌드, Vivado 합성/시물

Part 2. 소프트웨어 골든 레퍼런스

장	파일	내용
9	skeleton C 레퍼런스	darknet 기반 정수 추론, hex 생성, 코드 해설
10	22-Layer 네트워크 구조	cfg 해석, 레이어별 형상·연산, skip connection

Part 3. 하드웨어 큰 그림

장	파일	내용
11	하드웨어 아키텍처 개요	모듈 계층, 데이터 흐름, 144-MAC 한눈에
12	데이터 표현과 메모리 맵	세 가지 데이터 포맷, descaling, DRAM/BRAM 맵

Part 4. RTL 모듈 상세 (코드 해설 중심)

장	파일	내용
13	yolo_engine —최상위 FSM	53-state FSM, 파라미터 mux, 포트 demux, 코드 해설
14	Convolution 엔진	mul→add_tree→mac_stack→mac_kern→conv의 한 줄씩 해설
15	메모리 버퍼	line buffer·param buffer·dpram/spram
16	특수 연산 유닛	pooling·upsample·REPACK
17	AXI / DMA 인터페이스	DMA read/write, AXI-Lite slave

Part 5. 동작과 타이밍

장	파일	내용
18	동작 과정과 타이밍	추론 시퀀스, MAC 파이프라인, 성능 추정

Part 6. 검증 (Testbench)

장	파일	내용
19	검증 전략과 인프라	2-Phase 검증, 로그 포맷, tolerance
20	Layer 별 / 블록 Testbench	L0~L18 + 블록 TB, 결과 해석

Part 7. Vivado 프로젝트

장	파일	내용
21	Vivado 프로젝트	프로젝트·BMG IP·합성/시뮬·2021/2025 이슈

Part 8. 부록

장	파일	내용
22	부록—향후 작업 / 용어집	L19~L21, Phase 3·4, host.py, 용어집

진행 상태

본 해설서는 **L0~L18 검증 완료 + L19~L20 RTL 완성** 기준으로 작성됩니다 (L19=Route REPACK, L20=detection shift 9, standalone PASS). L0→L20 full chain 검증과 FPGA 보드 동작이 완료되면 22 장 부록의 해당 내용을 본문으로 승격합니다.

더 깊은 정확성이나 빠른 참조가 필요하다면 기술 레퍼런스를, 충돌 시에는 **실제 소스 코드**를 최우선으로 삼으십시오.

Last updated: 2026-05-24 ·범위: L0~L18 검증 완료 + L19~L20 RTL 완성·대상: 학부생 입문자·작성: 팀 베타트론

1 장. CNN 과 합성곱 기초

Part 0 기초 개념—이 장은 사전 지식이 필요 없습니다.

이 가속기가 하는 일을 한마디로 하면 “합성곱 신경망 (CNN) 의 추론을 하드웨어로 빠르게 계산하는 것”입니다. 그러니 가장 먼저 CNN 이 무엇이고, 어떤 연산들로 이루어지는지 알아야 합니다. 이 장을 읽고 나면 10 장 네트워크 구조의 22 개 레이어가 무슨 일을 하는지, 그리고 14 장의 144-MAC 이 왜 필요한지 감이 잡힐 것입니다.

1.1 신경망“추론”이란 무엇인가

신경망은 두 단계로 나뉩니다.

- **학습 (training):** 정답이 있는 데이터로 신경망의 “가중치 (weight)”를 조금씩 조정하는 과정. 시간이 오래 걸리고 보통 GPU 로 합니다.
- **추론 (inference):** 이미 학습된 가중치를 가지고, 새 입력 (예: 사진 한 장) 을 넣어 답 (예: “고양이가 여기 있다”) 을 계산하는 과정.

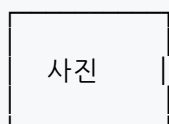
이 프로젝트는 추론만 합니다. 학습은 PC 에서 미리 끝냈고 (aix2024.weights 파일 에 가중치가 저장됨), FPGA 는 그 가중치로 입력 이미지를 처리하는 추론 가속기입니다.

[비유] 비유: 학습은 요리사가 레시피를 완성하는 과정 (오래 걸림), 추론은 완성된 레시피로 요리를 만드는 과정 (빠르게 반복) 입니다. 이 가속기는 “요리 기계”입니다.

1.2 특징맵과 채널—데이터의 모양

CNN 이 다루는 데이터는 특징맵 (feature map) 이라는 3 차원 덩어리입니다. 모양은 높이 (H) × 너비 (W) × 채널 (C) 로 표현합니다.

입력 이미지 (256 × 256 × 3)
H=256



W=256

채널 C=3 (빨강 R, 초록 G, 파랑 B)



- 컬러 사진은 빨강·초록·파랑 3 장의 흑백 이미지가 겹쳐진 것이라고 생각하면 됩니다. 그래서 입력 채널이 **3** 입니다.
- 신경망을 통과할수록 채널 수는 늘고 (더 추상적인 특징), 높아·너비는 줄어듭니다 (공간 압축). 예를 들어 이 네트워크는 $256 \times 256 \times 3 \rightarrow \dots \rightarrow 8 \times 8 \times 512$ 로 변합니다.

[주의] **흔한 오해**: “채널”은 컬러 (RGB) 에서 시작했지만, 중간 레이어의 채널은 색이 아니라 “어떤 특징의 강도”입니다. 예를 들어 한 채널은 “세로 모서리”, 다른 채널은 “둥근 모양”에 반응할 수 있습니다.

이 $H \times W \times C$ 덩어리가 메모리에 어떻게 저장되는지는 12 장 데이터 표현에서 중요하게 다룹니다 (NCHW 포맷 등).

1.3 합성곱 (Convolution) —CNN 의 심장

합성곱은 작은 필터 (커널) 를 이미지 위에서 미끄러뜨리며, 겹치는 부분을 곱해서 더하는 연산입니다.

1.3.1 가장 단순한 예: 1 채널, 3×3 커널

입력의 한 위치에 3×3 커널을 올려놓고, 9 개 숫자를 짝지어 곱한 뒤 모두 더합니다.

입력 일부 (3×3)	커널 (3×3)	계산																		
<table border="1"> <tr><td>1</td><td>2</td><td>3</td></tr> <tr><td>4</td><td>5</td><td>6</td></tr> <tr><td>7</td><td>8</td><td>9</td></tr> </table>	1	2	3	4	5	6	7	8	9	<table border="1"> <tr><td>a</td><td>b</td><td>c</td></tr> <tr><td>d</td><td>e</td><td>f</td></tr> <tr><td>g</td><td>h</td><td>i</td></tr> </table>	a	b	c	d	e	f	g	h	i	출력 한 점 = $= 1 \cdot a + 2 \cdot b + 3 \cdot c$ $+ 4 \cdot d + 5 \cdot e + 6 \cdot f$ $+ 7 \cdot g + 8 \cdot h + 9 \cdot i$ (곱셈 9 번 + 덧셈 8 번)
1	2	3																		
4	5	6																		
7	8	9																		
a	b	c																		
d	e	f																		
g	h	i																		

이렇게 나온 하나의 숫자가 출력 특징맵의 한 픽셀이 됩니다. 그리고 커널을 한 칸 옆으로 옮겨 다음 출력 픽셀을 계산합니다. 이것을 “슬라이딩 (sliding)”이라고 합니다.

커널이 미끄러지는 모습 (→ 방향으로 한 칸씩):

<table> <tr><td>■</td><td>■</td><td>■</td><td>.</td><td>.</td><td>.</td></tr> <tr><td>■</td><td>■</td><td>■</td><td>.</td><td>.</td><td>.</td></tr> <tr><td>■</td><td>■</td><td>■</td><td>.</td><td>.</td><td>.</td></tr> </table> 출력 (0,0)	■	■	■	.	.	.	■	■	■	.	.	.	■	■	■	.	.	.	→	<table> <tr><td>.</td><td>■</td><td>■</td><td>■</td><td>.</td><td>.</td></tr> <tr><td>.</td><td>■</td><td>■</td><td>■</td><td>.</td><td>.</td></tr> <tr><td>.</td><td>■</td><td>■</td><td>■</td><td>.</td><td>.</td></tr> </table> 출력 (0,1)	.	■	■	■	.	.	.	■	■	■	.	.	.	■	■	■	.	.	→	<table> <tr><td>.</td><td>.</td><td>■</td><td>■</td><td>■</td><td>.</td></tr> <tr><td>.</td><td>.</td><td>■</td><td>■</td><td>■</td><td>.</td></tr> <tr><td>.</td><td>.</td><td>■</td><td>■</td><td>■</td><td>.</td></tr> </table> 출력 (0,2)	.	.	■	■	■	.	.	.	■	■	■	.	.	.	■	■	■	.	→ ...
■	■	■	.	.	.																																																						
■	■	■	.	.	.																																																						
■	■	■	.	.	.																																																						
.	■	■	■	.	.																																																						
.	■	■	■	.	.																																																						
.	■	■	■	.	.																																																						
.	.	■	■	■	.																																																						
.	.	■	■	■	.																																																						
.	.	■	■	■	.																																																						

1.3.2 여러 입력 채널이 있을 때

입력 채널이 여러 개 (예: 4 채널) 면, 각 채널마다 별도의 3×3 커널이 있고, 모든 채널의 결과를 다시 더합니다.

출력 한 점 = (채널 0 의 3×3 곱셈합) + (채널 1 의 3×3 곱셈합) + (채널 2 의 3×3 곱셈합) + (채널 3 의 3×3 곱셈합)

즉 입력 채널이 C_i 개면, 출력 한 점을 만드는 데 $3 \times 3 \times C_i$ 번의 곱셈이 필요합니다.

[핵심] 여기서 **144-MAC** 의 단서가 나옵니다. 이 가속기는 한 번에 “4 개 입력 채널 $\times 3 \times 3 = 36$ 개 곱셈”을 처리합니다 ($36 = 4 \times 9$). 입력 채널이 4 보다 많으면 4 개씩 나눠서 여러 번 반복 (누적) 합니다. 자세한 건 14 장에서.

1.3.3 여러 출력 채널 (필터)

출력 채널을 C_o 개 만들고 싶으면, 서로 다른 커널 세트 (필터) 를 C_o 개 준비합니다. 필터 하나가 출력 채널 하나를 만듭니다.

필터 0 → 출력 채널 0
필터 1 → 출력 채널 1
⋮
필터 C_o-1 → 출력 채널 C_o-1

정리하면, 한 합성곱 레이어의 총 곱셈 횟수는:

출력높이 × 출력너비 × 출력채널 (C_o) × (커널크기² × 입력채널 (C_i))

이 숫자가 어마어마하게 큼니다. 예를 들어 이 네트워크의 L2 는 약 **7,550 만** 번의 곱셈을 합니다 (10 장). 그래서 곱셈을 병렬로 많이 하는 하드웨어 (**MAC 어레이**) 가 필요한 것입니다.

1.4 stride 와 padding

합성곱에는 두 개의 중요한 설정이 있습니다.

Stride (보폭)

커널을 한 번에 몇 칸씩 옮길지입니다. stride=1 이면 한 칸씩 (출력 크기 유지), stride=2 면 두 칸씩 (출력 크기 절반).

stride=1: [■] [■] [■] → 출력이 입력과 비슷한 크기
stride=2: [■] [■] [■] → 출력이 절반 크기

Padding (가장자리 채우기)

커널이 이미지 가장자리를 벗어날 때, 바깥을 0 으로 채워서 출력 크기를 맞춥니다. pad=1 이면 사방에 0 을 한 줄씩 둘러서, 3×3 stride-1 합성곱의 출력 크기가 입력과 같아집니다.

원본 4×4 padding 후 6×6 (0 테두리)

■ ■ ■ ■	→	0 0 0 0 0 0
■ ■ ■ ■		0 ■ ■ ■ ■ 0
■ ■ ■ ■		0 ■ ■ ■ ■ 0
■ ■ ■ ■		0 ■ ■ ■ ■ 0
■ ■ ■ ■		0 ■ ■ ■ ■ 0
		0 0 0 0 0 0

이 네트워크의 3×3 합성곱은 모두 stride=1, pad=1 이라 출력 크기 = 입력 크기입니다. 공간을 줄이는 일은 합성곱이 아니라 다음에 설명할 풀링이 담당합니다. 하드웨어에서 padding 은 15 장 line buffer 의 “boundary padding”으로 구현됩니다.

1.5 1×1 합성곱 vs 3×3 합성곱

이 네트워크에는 두 종류의 합성곱이 있습니다.

종류	커널	한 출력점의 곱셈 수	역할
3×3	3×3	9 × C _i	공간적 패턴 (모서리·질감) 추출
1×1	1×1	1 × C _i	채널 수 조정 (섞기), 공간은 그대로

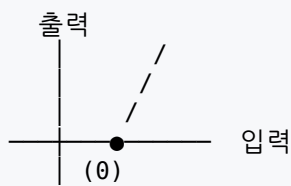
1×1 합성곱은 커널이 점 하나라서, 공간적으로는 아무것도 안 보고 채널 방향으로만 가중합을 합니다. 주로 채널 수를 늘리거나 줄일 때 씁니다 (예: L12 는 512 채널 → 256 채널).

[핵심] 하드웨어 관점의 핵심: 1×1 과 3×3 은 “한 출력점에 입력을 몇 개 모으느냐”만 다릅니다. 그래서 이 가속기는 같은 곱셈기 어레이를 두 경우에 재사용하고, 입력을 모아주는 방식만 다르게 합니다 (14 장 8.9). 새 하드웨어를 만들지 않는 것이 면적·전력 절약의 핵심입니다.

1.6 활성화 함수—ReLU

합성곱 결과에 비선형 함수를 적용해야 신경망이 복잡한 패턴을 배울 수 있습니다. 이 네트워크가 쓰는 활성화는 거의 다 **ReLU(Rectified Linear Unit)** 입니다.

$\text{ReLU}(x) = \max(0, x)$ 음수면 0, 양수면 그대로

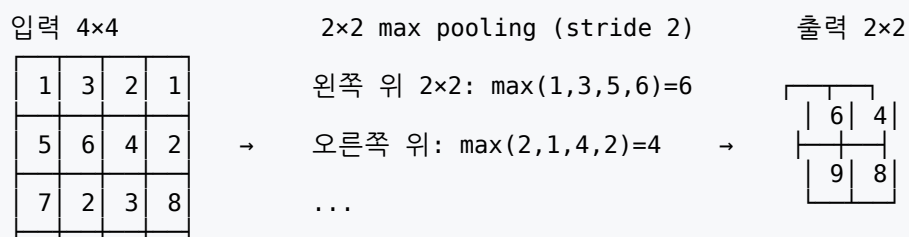


매우 단순합니다: “음수는 0 으로 만들고, 양수는 그대로 둔다.” 하드웨어로도 비교 한 번이면 됩니다 (14 장 post_process).

[주의] 예외: 검출 헤딩 L14·L20 은 ReLU 를 쓰지 않습니다 (linear). 이들은 좌표·확률을 출력해야 해서 음수도 의미가 있기 때문입니다. 이 차이가 하드웨어에서 i_relu_en 신호로 처리됩니다 (2 장, 14 장).

1.7 풀링 (Pooling)

풀링은 특징맵을 작게 줄이는 연산입니다. 이 네트워크는 **맥스 풀링 (max pooling)** 을 씁니다: 2×2 영역에서 가장 큰 값 하나만 남깁니다.



9	1	5	6
---	---	---	---

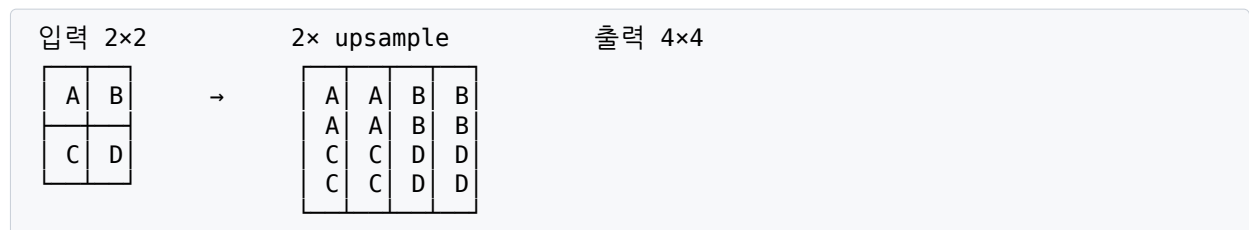
- **stride=2 풀링 (L1,3,5,7,9):** 2 칸씩 건너뛰어 크기가 **절반**이 됩니다. 이게 공간을 줄이는 주역입니다.
- **stride=1 풀링 (L11):** 1 칸씩 옮겨 크기가 **그대로**입니다. 같은 자리에서 주변 최댓값만 강조합니다.

[핵심] **L11 이 특별한 이유:** stride=1 풀링은 출력이 입력 크기와 같고, 윈도우가 겹칩니다. 일반 stride=2 풀링과 동작이 완전히 달라서, 하드웨어에서 **전용 모듈 max_pool_s1_unit** 을 따로 만들었습니다 (16 장). 이것 일반 풀링으로 처리하면 크기가 깨집니다.

[비유] 풀링의 또 다른 효과: max 는 “주변에서 가장 강한 신호”만 남기므로, 작은 ± 1 오차가 있던 값이 더 큰 이웃에 가려져 사라집니다. 이 성질이 20 장에서 “L11 을 지나면 누적 오차가 0 으로 흡수되는” 현상의 원인입니다.

1.8 업샘플 (Upsample)

업샘플은 풀링의 반대로, 특징맵을 **키웁니다**. 이 네트워크의 L18 은 가장 단순한 **최근접 이웃 (nearest-neighbor)** 방식으로 한 픽셀을 2×2 로 복제합니다.



곱셈이 전혀 없고 값을 복사할 뿐입니다. 그래서 하드웨어 **upsample_unit** 도 단순한 복사 FSM 입니다 (16 장). 업샘플은 작은 특징맵을 키워서 큰 특징맵과 합치기 위해 (skip connection) 씁니다 — 6 장 YOLO 기초에서 다시 다룹니다.

1.9 이 가속기가 다루는 연산 총정리

지금까지 배운 연산이 이 가속기가 하드웨어로 처리하는 전부입니다.

연산	하는 일	이 네트워크의 레이어	담당 하드웨어
합성곱 3×3	공간 패턴 추출	L0,2,4,6,8,10,13	conv_top+mac_kern
합성곱 1×1	채널 조정	L12,14,17,20	같은 모듈 (입력만 다르게)
ReLU	음수 제거	대부분 conv	post_process

연산	하는 일	이 네트워크의 레이어	담당 하드웨어
Max pool /2	공간 절반	L1,3,5,7,9	max_pool_unit
Max pool /1	강조 (크기 유지)	L11	max_pool_s1_unit
Upsample ×2	공간 2 배	L18	upsample_unit
Route(연결)	특징맵 잇기	L16,19	(연산 없음, 메모리 제어)

검출 헤드 이후의 좌표 디코딩·NMS 는 하드웨어가 아니라 소프트웨어가 합니다 (6 장, 9 장).

1.10 합성곱은 곧 곱셈-누적 (MAC) 의 산이다

이 장에서 가장 기억할 한 가지: 합성곱은 결국“곱하고 더하기”를 어마어마하게 많이 반복하는 일입니다. 이“곱하고 더하기”를 한 단어로 **MAC(Multiply-Accumulate)** 라고 부릅니다.

MAC 한 번 = (입력 한 개) × (가중치 한 개) 를 누적합에 더하기
합성곱 한 레이어 = 수천만 번의 MAC

CPU 는 MAC 을 한 번에 몇 개밖에 못 합니다. 그래서 이 프로젝트는 **MAC 을 144 개 동시에 수행하는 전용 회로**를 만듭니다. 이것이 11 장·14 장의 144-MAC 어레이이고, 가속의 핵심입니다.

CPU: MAC 1~8 개씩 순차 → 느림
가속기: MAC 144 개 병렬 → 빠름 (한 클럭에 4 픽셀 동시 생성)

이 MAC 계산을 어떻게 정수로 (부동소수점 없이) 정확하게 하느냐가 다음 장의 주제—양자화입니다.

1.11 이 장의 요약

- 신경망은 학습 (PC) → 추론 (이 FPGA) 으로 나뉘고, 이 프로젝트는 **추론 가속기**입니다.
- 데이터는 $H \times W \times C$ 특징맵이며, 신경망을 통과할수록 채널 ↑·공간 ↓.
- 합성곱은 커널을 슬라이딩하며 곱셈-누적; 한 출력점에 $k^2 \times C_i$ 번 곱셈, 레이어 전체는 수천만 MAC.
- stride 는 보폭, padding 은 가장자리 0 채우기. 3×3 은 공간 패턴, 1×1 은 채널 조정.
- ReLU(음수 → 0), Max pooling(/2 공간축소, /1 강조), Upsample($\times 2$ 복제) 이 합성곱을 보조.
- 핵심: **합성곱 = MAC 의 산** → 144 개 MAC 을 병렬로 돌리는 것이 가속의 본질.

다음 장에서는 이 MAC 을 부동소수점 대신 **정수 (INT8)** 로 하는 양자화를 배웁니다.

2 장. 신경망 양자화 기초

Part 0 기초 개념—1 장의 MAC 개념을 알면 좋습니다.

1 장에서“합성곱 = MAC 의 산”임을 배웠습니다. 그런데 PC 에서 학습된 신경망의 가중치는 0.0314, -1.872 같은 실수 (부동소수점) 입니다. 이 장은 그 실수들을 정수 (INT8) 로 바꿔서 계산하는 양자화 (quantization) 를 설명합니다. 양자화는 이 가속기의 정확도 (mAP) 와 에너지를 동시에 좌우하는 핵심 기술입니다.

2.1 왜 부동소수점 대신 정수인가?

신경망 학습은 보통 32 비트 부동소수점 (FP32) 으로 합니다. 하지만 추론을 FPGA 에서 할 때 FP32 를 쓰면 문제가 있습니다.

항목	FP32 (부동소수점)	INT8 (정수)
곱셈기 크기	큼 (FPGA DSP 여러 개)	작음 (DSP 1 개로 충분)
전력	높음	낮음
메모리	4 byte/값	1 byte/값 (4 배 절약)
속도	느림	빠름
정확도	100%	약간 손실 (보통 무시 가능)

[핵심] 점수 공식과의 연결: 이 대회 점수는 $10^4 / \text{Energy} \times \dots$ 로 에너지가 분모입니다 (7 장). 정수 연산은 부동소수점보다 전력을 훨씬 적게 쓰므로, INT8 양자화는 점수를 올리는 직접적 수단입니다. 메모리도 1/4 로 줄어 DRAM 접근 (가장 큰 에너지원) 이 줄어듭니다.

[비유] 비유: 부동소수점은“소수점 위치가 자유로운 정밀 저울”이고, 정수 양자화는“1g 단위로만 재는 단순 저울”입니다. 단순 저울은 약간 부정확하지만 훨씬 싸고 빠릅니다. 신경망은 다행히 약간의 부정확함을 견딜 수 있습니다.

2.2 핵심 아이디어—scale 로 정수에 매핑하기

양자화의 기본 아이디어는 간단합니다: 실수에 큰 수 (**scale**) 를 곱한 뒤 반올림하여 정수로 만듭니다.

```
정수값 = round( 실수값 × scale )
```

예를 들어 가중치 0.0314 에 scale 64 를 곱하면:

```
round(0.0314 × 64) = round(2.0096) = 2 (INT8 정수 2)
```

나중에 결과를 해석할 때는 다시 scale 로 나누면 원래 크기로 돌아옵니다. 정수끼리 곱하고 더한 뒤, 마지막에 한 번만 scale 을 풀어주면 되는 것이 핵심입니다.

[주의] **정수의 범위 제한**: INT8(부호 있는 8 비트) 은 **-128 ~ +127** 만 표현합니다. scale 을 곱한 결과가 이 범위를 넘으면 잘라냅니다 (클리핑, clipping). 그래서 scale 을 너무 크게 잡으면 큰 값들이 다 127 로 뭉개지고, 너무 작게 잡으면 작은 값들이 0 이 되어 정보를 잃습니다. **적절한 scale** 고르기가 양자화의 기술입니다.

2.3 이 프로젝트의 양자화 방식

이 프로젝트는 **레이어마다 고정된 scale(per-layer quantization)** 을 씁니다. 입력과 가중치에 각각 다른 scale 을 곱합니다 (skeleton 의 do_quantization(), 9 장에서 코드로).

```
입력 정수 = clip( 입력 실수 × input_quant_multiplier, 0..255 )
가중치 정수 = clip( 가중치 실수 × weight_quant_multiplier, -128..127 )
```

레이어	input_quant_multiplier	weight_quant_multiplier
L0 (첫 레이어)	128	16
L2~L20 (나머지)	8	64

[왜?] **왜 L0 만 다를까?**: 입력 이미지 픽셀은 0~1 로 정규화되어 값이 작습니다. 작은 값을 정수로 잘 표현하려면 큰 scale(128) 이 좋습니다. 반면 중간 레이어의 활성화값은 ReLU 를 거쳐 범위가 안정되어 작은 scale(8) 로 충분합니다. 가중치는 그 반대 경향이라 L0=16, 나머지 =64 입니다. 이 숫자들은 “정확도가 가장 적게 떨어지도록” 실험으로 고른 값입니다.

[주의] **입력은 UINT8(0~255), 가중치는 INT8(-128~127)**: 입력은 ReLU 이후라 음수가 없어 부호 없는 8 비트 (0~255) 로 저장하고, 가중치는 음수가 있어 부호 있는 8 비트입니다. 그런데 곱셈할 때는 둘 다 부호 있는 수로 다룹니다 (입력의 최상위 비트가 1 이어도 그건 큰 양수). 이 미묘함이 14 장 mul.v 에서 Phase 2 버그의 원인이 되었습니다.

2.4 INT8 곱셈-누적-누적은 왜 INT32 인가

이제 양자화된 정수로 MAC 을 합니다.

한 번의 곱셈: $\text{INT8} \times \text{INT8} \rightarrow$ 결과는?
최댓값: $(-128) \times (-128) = 16384 \rightarrow 16$ 비트면 충분 (INT16)

곱셈 하나는 INT16 에 들어갑니다. 그런데 합성곱은 이런 곱셈을 수백 ~ 수천 개 더합니다 (1 장 1.3). 16 비트짜리를 수천 개 더하면 16 비트를 넘쳐버립니다 (overflow).

누적합 최댓값 $\approx 16384 \times (3 \times 3 \times 256 \text{ 채널}) \approx 3,700$ 만 $\rightarrow 32$ 비트 필요 (INT32)

[핵심] 그래서 비트폭이 단계적으로 커집니다: 곱셈 결과는 INT16, 그것들을 모으는 가산 트리는 22 비트, 최종 누적기는 32 비트. 이 비트폭 변화가 14 장의 mul(16 비트) $\rightarrow$ add_tree_36in(22 비트) $\rightarrow$ accumulator(32 비트) 에 그대로 나타납니다. 비트가 부족하면 큰 값이 깨지고, 너무 많으면 회로가 낭비됩니다.

[비유] 비유: 작은 숫자 수천 개를 더할 때 작은 계산기 (16 자리) 로는 합계가 넘쳐서, 큰 종이 (32 자리) 에 적어가며 더하는 것과 같습니다.

2.5 bias 더하기

합성곱 누적이 끝나면 **편향 (bias)** 을 더합니다. bias 는 출력 채널마다 하나씩 있는 보정값입니다.

bias 도 양자화해야 하는데, 입력과 가중치 **scale** 을 둘 다 곱한 만큼 키워야 합니다. 왜냐하면 MAC 결과가 이미 $\text{input_scale} \times \text{weight_scale}$ 만큼 커져 있기 때문에, bias 도 같은 크기로 맞춰야 더할 수 있습니다.

bias 정수 = clip(bias 실수 $\times$ (input_quant_multiplier $\times$ weight_quant_multiplier), INT16)

[주의] **bias** 는 16 비트인데 누적은 32 비트: bias(INT16) 를 32 비트 누적값에 더하려면 **부호 확장 (sign-extend)** 을 해야 합니다. 단순히 위를 0 으로 채우면 (zero-extend) 음수 bias 가 거대한 양수가 되어 결과가 완전히 망가집니다. 이것이 12 장-CLAUDE.md 규칙 7 에서 강조하는 sign-extend 입니다.

16 비트 음수 bias 0xFFFF (= -1)
 $\rightarrow$ sign-extend: 0xFFFFFFFF (= -1) [O] 올바른
 $\rightarrow$ zero-extend: 0x0000FFFF (= 65535) [X] 완전히 다른 값

2.6 Descaling —다시 작게 만들기

MAC + bias 결과는 $\text{input_scale} \times \text{weight_scale}$ 만큼 부풀어 있는 큰 정수입니다. 이걸 다음 레이어가 쓸 수 있는 작은 정수로 되돌려야 합니다. 이 과정을 **descaling(역스케일링)** 이라 합니다.

원래는 나눗셈이지만, scale 이 2 의 거듭제곱이 되도록 설계해서 **비트 시프트 ($\gg$)** 로 대체합니다. 나눗셈은 하드웨어에서 비싸지만 시프트는 거의 공짜이기 때문입니다.

```
descaling = (MAC+bias 결과) >> shift_amount
```

```
shift_amount = log2( input_scale × weight_scale / 다음레이어 _input_scale )
```

예를 들어 L2 는 input_scale=8, weight_scale=64, 다음 레이어 input_scale=8:

배율 = $8 \times 64 / 8 = 64 = 2^6 \rightarrow \text{shift_amount} = 6$
 즉, 결과를 오른쪽으로 6 비트 시프트 (= 64 로 나누기)

레이어	계산	shift
L0	$128 \times 16 / 8 = 256 = 2^8$	8
L2~L13, L17	$8 \times 64 / 8 = 64 = 2^6$	6
L14 (검출 헤드 1)	$8 \times 64 / \mathbf{8} = 64 = 2^6$	6
L20 (검출 헤드 2)	$8 \times 64 / \mathbf{1} = 512 = 2^9$	9

[왜?] 검출 헤드인데 **L14** 와 **L20** 의 **shift** 가 왜 다를까?: next_input_multiplier(다음 합성곱의 입력 scale) 가 다르기 때문입니다. L14 다음에는 합성곱 **L17** 이 있어 next=8 이지만, L20 다음에는 합성곱이 없어 (L21 은 yolo) next=1 이 됩니다. 그래서 L14 는 $8 \times 64 / 8 = 64$ (shift **6**), L20 은 $8 \times 64 / 1 = 512$ (shift **9**). 둘 다 검출 헤드 (ReLU off) 이지만 shift 는 다릅니다.

[핵심] 이 shift 값들이 13 장 yolo_engine 의 레이어 파라미터 테이블 (L14_SHIFT=6, L20_SHIFT=9 등) 에 그대로 들어갑니다. skeleton 이 만든 CONVnn_param_scales.hex 파일에서 측정한 값입니다 (9 장).

2.7 반올림—음수에서 생기는 1 LSB 차이

여기 함정이 하나 있습니다. **C** 언어의 나눗셈과 **Verilog** 의 시프트가 음수에서 다르게 동작합니다.

-50 / 64 의 결과:
 C 언어 (/): 0 (toward-zero, 0 방향으로 버림)
 Verilog (>>>): -1 (floor, -∞ 방향으로 내림)

골든 (C) 이 0 인데 하드웨어 (>>>) 가 -1 이면 1 만큼 차이가 납니다. 이“1 LSB(최하위 비트) 차이”가 검증에서 mismatch 로 잡힙니다.

해결책은 음수일 때만 시프트 전에 ($2^{\text{shift}} - 1$) 을 더해주는 것입니다. 이러면 Verilog 시프트도 C 의 toward-zero 와 같아집니다.

```
round_bias = (값이 음수) ? (2^shift - 1) : 0;  
결과 = (값 + round_bias) >>> shift;
```

이 보정은 ReLU 가 있어 음수가 없는 일반 레이어에는 영향이 없고, **ReLU 없는 검출 헤드 (L14, L20) 에서 결정적**입니다. 14 장 post_process 에서 실제 코드로 봅니다. 이 보정을 빠뜨려서 L14 가 거의 다 1 씩 틀렸던 것이 20 장의 디버깅 사례입니다.

2.8 클리핑—0~255 로 자르기

descaling 후 최종 출력은 다음 레이어의 입력이 되므로, 다시 8 비트 범위로 잘라야 합니다.

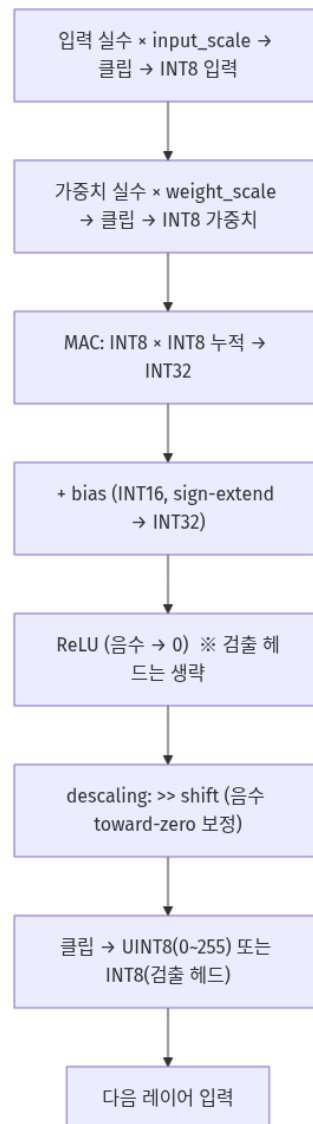
- 일반 레이어 (**ReLU 있음**): 0~255(UINT8). 음수는 0, 255 초과는 255 로.
- 검출 헤드 (**ReLU 없음**): -128~+127(INT8). 좌표·확률이라 음수 유지.

일반: [큰 음수] → 0, [0~255] → 그대로, [큰 양수] → 255
검출헤드: [< -128] → -128, [-128~127] → 그대로, [> 127] → 127

이 두 모드가 14 장 post_process 의 i_relu_en 신호로 걸립니다.

2.9 전체 흐름 한눈에 보기

한 합성곱 레이어의 양자화 추론을 처음부터 끝까지 모으면:



이 흐름이 14 장의 mac_kern + post_process 하드웨어와 단계마다 1:1 로 대응합니다. 그래서 양자화를 이해하면 그 하드웨어가 왜 그렇게 생겼는지 자연스럽게 이해됩니다.

2.10 양자화 오차와 mAP

양자화는 약간의 오차를 만듭니다. 검증에서 “ ± 1 LSB mismatch”가 보이는 이유입니다. 하지만:

- 대부분의 오차는 **1 LSB(최하위 비트 1 차이)** 수준이라 무시할 만합니다.
- max pooling 이 이런 작은 오차를 흡수합니다 (1 장 1.7, 20 장).
- 최종 검출 정확도 (mAP) 는 목표 0.2 를 넘기는 데 문제없는 수준입니다.

[핵심] 검증의 핵심 판정: | 하드웨어 - 골든 | ≤ 1 이면 **PASS**(양자화 noise 로 간주). 이 기준이 19 장 검증 전략의 TOLERANCE = 1 입니다.

2.11 이 장의 요약

- 양자화 = 실수에 scale 을 곱해 정수 (INT8) 로 만드는 것. 전력·메모리를 크게 절약 (점수의 Energy 향에 직결).
- 이 프로젝트는 레이어별 고정 scale: 입력 (L0=128, 나머지 =8), 가중치 (L0=16, 나머지 =64).
- 곱셈은 INT16, 누적은 INT32(넘침 방지). bias 는 두 scale 의 곱으로 키우고 **sign-extend** 로 더함.
- descaling 은 $\gg$ shift(2 의 거듭제곱 나눗셈). 음수는 toward-zero 보정 필요 (C 와 일치).
- 클립: 일반 레이어 UINT8(0~255), 검출 헤드 INT8(-128~127).
- 이 전 과정이 14 장 mac_kern+post_process 와 1:1 대응. 오차는 보통 ± 1 LSB(PASS 기준).

다음 장에서는 이 정수 연산을 실제로 수행할 그릇—**FPGA 하드웨어**의 기본 부품들을 배웁니다.

3 장. FPGA 하드웨어 기초

Part 0 기초 개념—사전 지식: 디지털 논리 (게이트·플립플롭) 기본.

1~2 장에서“무엇을 계산하는가”(합성곱, 양자화) 를 배웠다면, 이 장은“어디서 계산하는가”—**FPGA** 라는 하드웨어의 기본 부품을 소개합니다. FPGA 의 자원을 알아야 11 장의“144-MAC 을 DSP48 로”, “버퍼를 BRAM 으로”같은 설계 결정이 이해됩니다.

3.1 FPGA 란 무엇인가

FPGA(Field-Programmable Gate Array) 는“회로를 나중에 바꿀 수 있는 칩”입니다. CPU·ASIC 과 비교하면 위치가 분명해집니다.

	CPU	FPGA	ASIC
무엇	정해진 명령을 순서대로 실행	원하는 회로를 직접 구성	한 가지 회로로 고정 제작
유연성	소프트웨어로 자유	회로를 재구성 가능	불가 (제작 후 고정)
속도/효율	보통	높음 (전용 회로)	최고
개발 비용	낮음	중간	매우 높음

[비유] 비유: CPU 가“만능 일꾼 한 명이 작업 목록을 차례로 처리”하는 것이라면, FPGA 는“내가 원하는 모양의 조립 라인을 레고로 직접 짜서, 여러 작업을 동시에 흘려보내는”것입니다. 신경망처럼 같은 연산 (MAC) 을 엄청나게 반복하는 일은, 전용 조립 라인 (FPGA) 이 만능 일꾼 (CPU) 보다 훨씬 빠르고 효율적입니다.

FPGA 안에는 수많은 작은 부품이 격자처럼 깔려 있고, 우리는 그것들을 연결하는 방법 (설정) 을 정의합니다. 그 정의를 Verilog 같은 언어로 쓴 것이 RTL 이고, 그것을 부품 연결 정보로 바꾸는 과정이“합성/구현”입니다 (3.6).

3.2 기본 부품 (1) —LUT 와 플립플롭 (FF)

FPGA 의 가장 기본 부품 두 가지입니다.

LUT (Look-Up Table) —조합 논리

LUT 는 “입력 조합에 대한 출력을 적어둔 작은 표”입니다. 어떤 논리식이든 (AND, OR, 복잡한 if 문) 이 표로 흉내 낼 수 있습니다.

4-입력 LUT 예: 출력 = (a AND b) OR (c XOR d)
입력 abcd → 출력을 표에 미리 저장 → 입력이 오면 표에서 찾아 출력

플립플롭 (FF) —기억 (레지스터)

FF 는 “클럭이 똑딱일 때마다 값을 한 개 저장하는 칸”입니다. Verilog 의 reg 가 보통 FF 가 됩니다. 순차 논리 (시간에 따라 값이 바뀜) 의 기본입니다.

```
always @(posedge clk) // 클럭 상승 에지마다
    count <= count + 1; // count 를 FF 에 저장 (1 씩 증가)
```

[비유] 비유: LUT 는 “계산기”(입력 넣으면 즉시 답), FF 는 “메모지”(클럭 똑딱마다 한 값을 적어둠) 입니다. 회로는 계산기로 계산하고 메모지에 기록하기를 반복합니다.

3.3 기본 부품 (2) —DSP48 (곱셈기)

LUT 로도 곱셈을 만들 수 있지만 매우 비효율적입니다. 그래서 FPGA 에는 곱셈 전용 블록 **DSP48** 이 따로 박혀 있습니다.

- DSP48 하나는 큰 곱셈 (예: 18 비트 × 25 비트) 과 덧셈을 빠르게 합니다.
- 이 프로젝트의 INT8 × INT8 곱셈은 DSP48 하나로 충분합니다.
- Artix-7 XC7A100T 에는 DSP48 이 **240** 개 있습니다.

[핵심] 이 프로젝트는 **DSP48** 을 **144** 개 사용합니다 (11 장의 144-MAC). 240 개 중 144 개니까 여유가 있습니다. 곱셈을 LUT 로 만들면 240 개로는 어림없으니, DSP48 을 쓰는 것이 필수입니다. 14 장 mul.v 는 합성 시 xbip_dsp48_macro_0 이라는 DSP48 IP 를 직접 인스턴스화합니다.

[주의] “곱셈은 비싸다”는 감각: 하드웨어에서 곱셈기는 덧셈기보다 훨씬 크고 전력을 많이 씁니다. 그래서 “곱셈기 개수 (=144)”가 이 설계의 핵심 제약이자 성능 지표가 됩니다. 반대로 나눗셈은 더 비싸서, 2 장에서 본 것처럼 나눗셈을 시프트로 대체합니다.

3.4 기본 부품 (3) —BRAM (블록 메모리)

FPGA 안의 작지만 빠른 메모리입니다. FF 로도 메모리를 만들 수 있지만, 수천 ~ 수만 칸이 필요하다면 FF 는 너무 낭비라 BRAM 을 씁니다.

- BRAM 한 블록은 보통 36Kbit. Artix-7 XC7A100T 에는 약 135 개 (≈4.86Mbit).

- 데이터 폭과 깊이를 설정해서 다양한 모양으로 씁니다 (예: 128 비트 × 2048 칸, 32 비트 × 65536 칸).
- Xilinx에서는 **BMG(Block Memory Generator)** IP로 BRAM을 만듭니다 (21 장).

BRAM의 가장 중요한 성질—read latency

[핵심] **BRAM**은 “주소를 주면 그 다음 클럭에 데이터가 나옵니다” (read latency = 1 cycle). 즉시 나오지 않습니다.

클럭 0: 주소 A 입력
클럭 1: 데이터 [A] 출력 ← 1 클럭 늦게!

이 1 클럭 지연이 4 장에서 자세히 다룰 “타이밍 정렬”의 근원입니다. 여러 BRAM의 지연을 어긋나지 않게 맞추는 것이 이 설계에서 가장 까다로운 부분입니다.

[비유] 비유: BRAM은 “도서관 사서에게 책 번호를 말하면, 잠시 후 (1 클럭) 책을 가져다주는” 것과 같습니다. 말하자마자 책이 손에 있는 게 아닙니다.

이 프로젝트는 BRAM을 (1) 가중치 버퍼 (2) 입력 라인 버퍼 (3) 출력 버퍼로 씁니다 (11·15 장). 외부의 큰 데이터는 DRAM(보드의 DDR2)에 두고, 자주 쓰는 것만 BRAM에 올려 빠르게 접근합니다.

BRAM vs DRAM

	BRAM (온칩)	DRAM (보드의 DDR2)
위치	FPGA 칩 안	칩 밖
크기	작음 (~ 수백 KB)	큼 (수 MB~)
속도	빠름 (1 클럭)	느림 (여러 클럭 + 버스 거침)
전력	낮음	높음

[핵심] **DRAM** 접근은 가장 큰 에너지원입니다. 그래서 이 설계는 “한 번 읽은 데이터를 BRAM에 두고 최대한 재사용”하여 DRAM 접근을 줄입니다 (streaming weight, in-place pool 등—18 장). 이것이 Energy 점수를 올리는 핵심입니다.

3.5 면적·속도·전력의 트레이드오프

FPGA 설계는 세 가지를 저울질합니다.

- **면적 (area)**: LUT/FF/DSP/BRAM을 얼마나 쓰는가. 칩 용량 안에 들어가야 함.
- **속도 (clock frequency)**: 클럭을 얼마나 빠르게 돌릴 수 있는가 (MHz). 빠를수록 추론이 빠름 (fps ↑).
- **전력 (power)**: 얼마나 전기를 쓰는가. 점수의 Energy 항.

이들은 서로 얽혀 있습니다. 예를 들어 곱셈기를 144 개에서 288 개로 늘리면 더 빠르겠지만 (속도 ↑) 면적·전력이 늘고 DSP가 부족할 수 있습니다. 이 프로젝트가 **144 개로 고정**한 것은 이 균형점입니다 (11 장).

3.6 RTL 에서 비트스트림까지

우리가 쓴 Verilog 가 실제 FPGA 에서 동작하기까지의 과정입니다 (21 장에서 실제 명령으로).



1. **합성 (Synthesis)**: Verilog 를 LUT·FF·DSP·BRAM 같은 실제 부품의 연결로 변환.
2. **구현 (Implementation)**: 그 부품들을 칩의 실제 위치에 배치하고 전선으로 연결 (place & route).
3. **비트스트림 생성**: FPGA 를 설정할 .bit 파일 생성.
4. **적재**: .bit 를 FPGA 에 다운로드하면 우리가 설계한 회로가 칩 안에 만들어집니다.

이 프로젝트는 합성/구현/시뮬레이션을 **Vivado**(Xilinx 도구) 로 합니다 (8·21 장).

[핵심] **시뮬레이션 vs 합성**: 실제 칩에 올리기 전에, RTL 이 올바른지 PC 에서 **시뮬레이션**으로 검증합니다 (19 장). 시뮬레이션에서는 DSP48·BRAM 대신 그것을 흉내 내는 “behavioral 모델”을 씁니다. 이 전환이 4 장·21 장의 `define FPGA 매크로입니다.

3.7 타이밍 클로저 (Timing Closure)

FPGA 가 특정 클럭 (예: 100MHz = 10ns 주기) 으로 동작하려면, **모든 조합 논리가 한 클럭 주기 안에 계산을 끝내야** 합니다. LUT·전선을 거치는 데 시간이 걸리므로, 너무 복잡한 논리를 한 클럭에 몰면 제때 못 끝냅니다.

클럭 주기 (10ns) 안에: FF → LUT 들 → FF 계산이 완료되어야 함
완료 못 하면 → 타이밍 위반 (timing violation) → 동작 보장 불가

해결책은 **파이프라인** (긴 계산을 여러 클럭으로 쪼갬—4 장) 입니다. 모든 경로가 시간 안에 들어오면 “타이밍 클로저를 달성했다”고 합니다. 이 프로젝트의 최종 클럭 주파수는 구현 후 확정되며, 18 장의 fps 추정에 영향을 줍니다.

3.8 이 프로젝트의 FPGA —Artix-7 XC7A100T

자원	양	이 프로젝트 사용
로직 셀 (LUT/FF)	~101,440	FSM·제어·정렬 논리
DSP48	240 개	144 개 (144-MAC)
Block RAM	~135 개 (4.86Mbit)	가중치·라인·출력 버퍼
외부 메모리	DDR2 (보드)	가중치·IFM·OFM 저장

Digilent **Nexys A7-100T** 보드에 이 칩이 올라가 있고, UART(USB) 로 PC 와 통신합니다. DSP 는 여유롭고, **BRAM** 용량과 **DDR2** 대역폭이 주요 제약입니다.

3.9 이 장의 요약

- FPGA 는“회로를 직접 구성하는 칩”. 같은 연산을 대량 반복하는 신경망에 CPU 보다 효율적.
- 기본 부품: **LUT**(조합 논리·계산), **FF**(기억·레지스터), **DSP48**(곱셈 전용, 240 개 중 144 개 사용), **BRAM**(빠른 온칩 메모리).
- **BRAM** 은 **read latency 1** 클럭—주소를 주면 다음 클럭에 데이터. 이 지연 정렬이 설계의 핵심 난점 (4 장).
- DRAM 접근은 가장 큰 에너지원 → **BRAM** 재사용으로 줄이는 것이 Energy 점수의 핵심.
- RTL → 합성 → 구현 → 비트스트림. 시뮬레이션은 behavioral 모델로 (매크로 전환).
- 타겟: Artix-7 XC7A100T (DSP 240, BRAM ~135), Nexys A7-100T 보드.

다음 장에서는 이 부품들로 정확히 동작하는 회로를 만들기 위한 **RTL 설계와 타이밍**을 배웁니다.

4 장. RTL 설계와 타이밍 기초

Part 0 기초 개념—사전 지식: Verilog 기본 문법 (reg/wire, always), 3 장의 BRAM latency.

이 프로젝트의 RTL 을 읽다 보면 “왜 신호를 1 클럭 늦추지?”, “왜 주소를 미리 보내지?” 같은 의문이 계속 생깁니다. 이 장은 그 답인 **타이밍 정렬**의 원리를 설명합니다. 이걸 이해하면 14 장 conv_top 의 까다로운 코드가 “아, 그래서 그렇구나”로 바뀝니다. 실제로 이 프로젝트 Phase 2 버그의 상당수가 타이밍 정렬 문제였습니다.

4.1 RTL 과 클럭—조합 논리 vs 순차 논리

RTL(Register-Transfer Level) 설계는 두 가지 논리로 이루어집니다.

조합 논리 (combinational) —즉시 계산

입력이 바뀌면 (전선 지연만 빼고) 즉시 출력이 바뀝니다. Verilog 의 assign 이나 always @(*).

```
assign sum = a + b;           // a 나 b 가 바뀌면 즉시 sum 이 바뀜
```

순차 논리 (sequential) —클럭에 맞춰 기억

클럭 상승 에지 (posedge clk) 마다 값을 레지스터 (FF) 에 저장합니다. 값이 “한 클럭 뒤”에 반영됩니다.

```
always @(posedge clk)
    sum_reg <= a + b;         // 다음 클럭에 sum_reg 가 (a+b) 가 됨
```

[핵심] **핵심 구분**: 조합 논리는 “지금 이 순간”의 값, 순차 논리는 “한 클럭 전”값을 기억. 이 “한 클럭”의 차이를 정확히 추적하는 것이 타이밍 정렬입니다.

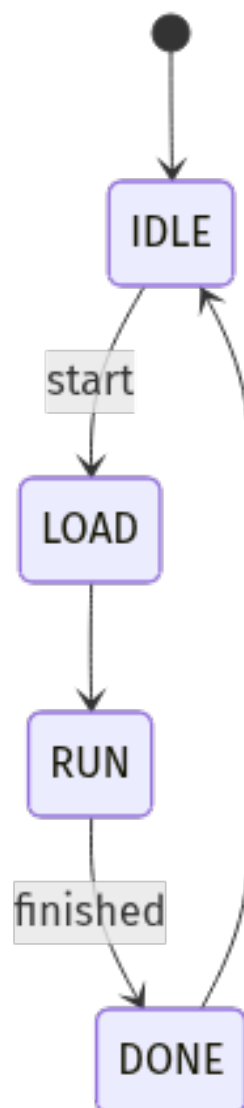
[비유] **비유**: 조합 논리는 “말하면 바로 메아리”, 순차 논리는 “녹음기에 녹음했다가 다음 박자에 재생”입니다.

4.2 FSM —상태 머신

복잡한 동작 (예: “DMA 로 데이터 받고 → 연산하고 → 결과 저장”) 을 순서대로 제어하려면 **유한 상태 머신 (FSM)** 을 씁니다. “지금 어느 단계인가”를 상태 레지스터에 저장하고, 조건에 따라 다음 상태로 넘어갑니다.

```
reg [2:0] state;
localparam IDLE=0, LOAD=1, RUN=2, DONE=3;

always @(posedge clk)
  case (state)
    IDLE: if (start)      state <= LOAD;    // 시작하면 LOAD 로
    LOAD:                state <= RUN;      // 적재 후 RUN 으로
    RUN: if (finished)    state <= DONE;    // 끝나면 DONE 으로
    DONE:                state <= IDLE;    // 다시 대기
  endcase
```



[핵심] 이 프로젝트의 yolo_engine 은 **53 개 상태**를 가진 거대한 FSM 으로 22 개 레이어를 순서대로 처리합니다. conv_top 도 6 개 상태의 작은 FSM 입니다. FSM 은 하드웨어의 “프로그램 흐름”이라고 보면 됩니다.

[비유] 비유: FSM 은 보드게임의 말과 같습니다. “지금 어느 칸 (상태) 에 있고, 주사위 결과 (조건) 에 따라 다음 칸으로 이동”합니다.

4.3 파이프라인—왜 계산을 쪼개는가

복잡한 계산 (예: 큰 곱셈) 을 한 클럭에 다 하려면 클럭을 느리게 해야 합니다 (3 장 타이밍 클로저). 대신 계산을 여러 단계로 쪼개고, 각 단계 사이에 FF 를 넣는 것이 **파이프라인**입니다.

파이프라인 없음: [———— 긴 계산 ———] → 클럭이 느려야 함
파이프라인 4 단: [계산 1]→FF→[계산 2]→FF→[계산 3]→FF→[계산 4] → 클럭 빠름

Latency 와 Throughput —헛갈리지 말 것

파이프라인을 이해할 때 두 개념을 구분해야 합니다.

- **Latency(지연)**: 입력이 들어가서 결과가 나오기까지 걸리는 클럭 수. (예: 4 단 파이프라인 → latency 4)
- **Throughput(처리량)**: 단위 시간당 처리하는 양. 파이프라인이 가득 차면 **매 클럭 하나씩** 결과가 나옵니다.

공장 비유: 자동차 한 대 만드는 데 4 단계 (latency 4) 지만,
라인이 가득 차면 매 단계마다 한 대씩 완성 (throughput 1/clock).

클럭:	0	1	2	3	4	5	6	
입력:	A	B	C	D	E	...		
출력:			-	-	A	B	C	... ← latency 4 후 매 클럭 출력

[핵심] 이 프로젝트의 곱셈기 mul 은 **latency 4**(4 단 파이프라인), 가산 트리도 **latency 4** 입니다. 합쳐서 MAC 어레이는 latency 8. 하지만 throughput 은 매 클럭 —파이프라인이 차면 매 클럭 새 결과가 나옵니다. 이 구분이 18 장 성능에서“latency 는 채움/비움에만 보이고 throughput 이 진짜 속도”라는 말의 의미입니다.

4.4 BRAM read latency 가 만드는 문제

3 장 3.4 에서“BRAM 은 주소를 주면 다음 클럭에 데이터가 나온다”(latency 1) 고 했습니다. 이게 왜 문제일까요?

가중치를 BRAM 에서 읽어 곱셈기에 넣는다고 합시다. 순진하게 짜면:

클럭 0: 가중치 주소 발사
클럭 0: 곱셈기에 가중치 입력 ← 아직 데이터가 안 나왔다! (X 값)

주소를 발사한 클럭에는 데이터가 아직 없습니다 (다음 클럭에 나옴). 그런데 곱셈기는 같은 클럭에 그 데이터를 기대하니, **타이밍이 1 클럭 어긋납니다**. 결과가 틀립니다.

해결책은 두 가지입니다.

방법 (1) 입력 쪽을 1 클럭 늦추기

데이터가 도착하는 클럭에 맞춰 “유효”신호를 1 클럭 지연시킵니다.

```
// 주소는 클럭 0 에 발사, 데이터는 클럭 1 에 도착
// → 곱셈 시작 신호도 1 클럭 늦춰 클럭 1 에 맞춤
reg vld_delayed;
always @(posedge clk) vld_delayed <= vld; // 1 클럭 지연
```

방법 (2) 주소를 미리 발사하기 (look-ahead)

다음에 필요할 데이터의 주소를 한 클럭 먼저 보냅니다. 그러면 정작 필요한 클럭에 데이터가 준비됩니다.

클럭 0: (다음 클럭에 쓸) 주소 미리 발사
클럭 1: 데이터 도착 + 사용 ← 딱 맞춤!

[주의] 이것이 **Phase 2 버그의 단골 원인**: 가중치 주소를 1 클럭 잘못 advance 해서 “가중치가 입력보다 1 스텝 앞서가는” 버그가 있었습니다 (14 장 8.7, HISTORY). 타이밍 정렬은 한 클럭만 틀려도 결과가 완전히 망가지므로 가장 조심해야 합니다.

4.5 valid 신호—데이터가 진짜인지 표시

파이프라인에서는 “지금 이 전선의 값이 진짜 유효한 데이터인가, 아니면 쓰레기인가”를 구분해야 합니다. 그래서 데이터와 함께 **valid 신호**를 흘려보냅니다.

```
// 데이터가 4 단 파이프라인을 지나는 동안, valid 도 똑같이 4 단 지연
reg v1, v2, v3, v4;
always @(posedge clk) begin
    v1 <= vld_in; v2 <= v1; v3 <= v2; v4 <= v3;
end
// v4 = 출력 데이터가 유효한 시점
```

[핵심] mac_stack 의 vld_m1~m4, add_tree 의 vld_d1~d4 가 정확히 이 패턴입니다. 데이터와 valid 를 **같은 지연**으로 흘려, 결과가 나오는 정확한 클럭을 알아냅니다.

[비유] 비유: 공장 컨베이어 벨트에서 제품마다 “검사 통과”스티커 (valid) 를 함께 붙여 보내는 것. 스티커 없는 것은 빈 자리 (쓰레기).

4.6 signed 연산—부호를 잊으면 큰일

Verilog 는 기본적으로 비트를 부호 없는 수로 다룹니다. 음수를 다루려면 명시적으로 \$signed() 를 써야 합니다.

```
reg [7:0] a; // 8 비트
a = 8'b1111_1111; // 부호 없으면 255, 부호 있으면 -1

c = a * b; // 부호 없는 곱셈 (255 × ...)
c = $signed(a) * $signed(b); // 부호 있는 곱셈 (-1 × ...)
```

[핵심] mul.v 는 가중치·입력을 \$signed() 로 곱합니다. Phase 2 버그 중 하나가 입력을 \$signed({1'b0,x})(억지로 양수) 로 다뤄서, 음수 가중치와의 곱이 틀어진 것이었습니다 (2 장 2.3). 부호 처리는 양자화 정확도와 직결됩니다.

비슷하게, 산술 우측 시프트 >>> 는 부호를 보존하지만 (음수는 위를 1 로 채움), 논리 시프트 >> 는 0 으로 채웁니다. descaling 에서 >>> 를 쓰는 이유입니다 (2 장 2.6).

4.7 uninitialized X —시뮬레이션의 함정

Verilog 시뮬레이션에서 초기화하지 않은 레지스터/메모리는 **X(unknown, 알 수 없음)** 상태입니다. X 는 계산에 섞이면 결과도 X 로 전파됩니다.

$X + 5 = X$, $X > 3 = X$ (불확실)

실제 FPGA 의 BRAM 은 전원이 켜지면 0 으로 초기화되지만, 시뮬레이터는 X 로 둘 수 있습니다. 그리고 시뮬레이터 버전마다 X 를 다르게 처리하기도 합니다.

[핵심] 이것이 21 장의 “Vivado 2021 vs 2025” 문제입니다. 초기화 안 된 메모리의 X 때문에, 같은 RTL 인데 시뮬레이터 버전에 따라 결과가 달라졌습니다. 해결책은 시뮬용 메모리를 initial 로 0 초기화하는 것 (실제 BRAM 과 일치) —15 장. 이것은 RTL 버그가 아니라 시뮬레이션 환경 문제였습니다.

4.8 왜 이 설계의 타이밍이 특히 까다로운가

이 프로젝트는 여러 BRAM(가중치·라인 버퍼·출력) 을 동시에 쓰고, 각각 read latency 가 있습니다. 그리고 곱셈기·가산기 파이프라인까지 있습니다. 이 모든 지연을 한 클럭도 어긋나지 않게 맞춰야 정확한 결과가 나옵니다.

가중치 BRAM (latency 1)  라인 버퍼 BRAM (latency 2)  곱셈기 (4) → 가산트리 (4) → 누적 → 후처리 (1)
↑ 이 입력들이 정확히 같은 클럭에 곱셈기에 도착해야 함

그래서 코드 곳곳에: - 신호를 1~2 클럭 지연 (mac_vld_d, offset_r) - 주소를 미리 발사 (lah_* look-ahead) - valid 를 데이터와 같이 지연 (vld_m1~4)

이런 장치들이 보입니다. 18 장 12.4 에 정렬 장치 표가 정리되어 있습니다. 처음 보면 복잡하지만, 모두 “BRAM·파이프라인 지연을 맞추기 위한 것”이라고 생각하면 읽힙니다.

4.9 이 장의 요약

- RTL 은 조합 논리 (즉시) 와 순차 논리 (클럭마다 기억) 로 구성. “한 클럭”차이를 정확히 추적하는 것이 핵심.
- FSM 은 하드웨어의 프로그램 흐름 (상태 + 전이). yolo_engine 은 53-state 거대 FSM.

- 파이프라인은 계산을 쪼개 클럭을 빠르게. **latency**(지연) 와 **throughput**(매 클럭 처리) 을 구분.
- **BRAM read latency 1** 이 타이밍 정렬을 까다롭게 만듦 → 신호 지연 또는 주소 look-ahead 로 해결.
- **valid** 신호를 데이터와 같은 지연으로 흘려 유효 시점 추적. `$signed()` 로 음수 처리 (빠뜨리면 양자화 오류).
- 시뮬레이션의 uninitialized X 는 버전마다 다르게 처리될 수 있어 0 초기화 필요 (Vivado 2021/2025 이슈).
- 이 설계의 복잡한 정렬 코드는 모두“여러 지연을 한 클럭도 안 어긋나게 맞추기 위한 것”.

다음 장에서는 FPGA 가 외부 메모리·CPU 와 데이터를 주고받는 약속—**AXI 버스 프로토콜**을 배웁니다.

5 장. AXI 버스 프로토콜 기초

Part 0 기초 개념—사전 지식: 4 장의 valid/ready 개념.

가속기는 외부 DRAM 에서 가중치·입력을 읽고 결과를 써야 하고, MicroBlaze(CPU) 로부터“시작” 명령을 받아야 합니다. 이 데이터·명령이 오가는 **통로 (버스)** 의 약속이 **AXI** 입니다. 이 장을 읽으면 17 장 AXI/DMA 의 수많은 M_AR*, M_W* 신호가 무엇인지 알게 됩니다.

5.1 버스란 무엇인가

여러 하드웨어 블록이 데이터를 주고받으려면 **공통의 약속 (프로토콜)** 이 필요합니다. 이 약속을 따르는 신호선 묶음이 **버스 (bus)** 입니다. **AXI(Advanced eXtensible Interface)** 는 ARM 이 만든 표준 버스로, Xilinx FPGA 에서 가장 널리 쓰입니다.

[비유] **비유**: AXI 는“택배 시스템의 표준 규약”입니다. 보내는 사람·받는 사람·주소·물건을 어떤 양식으로 주고받을지 정해두면, 서로 다른 회사 (블록) 끼리도 문제없이 거래할 수 있습니다.

5.2 Master 와 Slave

AXI 에는 두 역할이 있습니다.

- **Master(주인)**: 거래를 시작하는 쪽. “이 주소에서 읽어줘”, “이 데이터를 저기 써줘”라고 요청.
- **Slave(하인)**: 요청에 **응답**하는 쪽.

[핵심] 이 프로젝트에서: - yolo_engine 은 DRAM 에 대해 **master** 입니다 (읽기·쓰기를 직접 요청) → axi_dma_rd/wr. - yolo_engine 은 MicroBlaze(CPU) 에 대해 **slave** 입니다 (CPU 가 제어 레지스터를 읽고 씀) → yolo_engine_axi. 즉 같은 모듈이 한쪽에선 주인, 다른 쪽에선 하인입니다.

5.3 Handshake —valid 와 ready 의 악수

AXI 의 가장 기본 동작은 **valid/ready 핸드셰이크**입니다. 보내는 쪽이 valid(준비됐어) 를 올리고, 받는 쪽이 ready(받을게) 를 올립니다. 둘 다 **1** 인 클럭에 전송이 성립합니다.

클럭:	0	1	2	3	
valid:	0	1	1	0	(보내는 쪽: 클럭 1~2 에 데이터 준비)
ready:	0	0	1	0	(받는 쪽: 클럭 2 에 받을 준비)
전송:	-	-	●	-	← 클럭 2 (둘 다 1) 에 전송 성립

[비유] 비유: 물건을 건네줄 때 “줄게 (valid)”와 “받을게 (ready)”가 동시에 맞아야 손이 오갑니다. 한 쪽만 준비되면 기다립니다. 이 방식의 장점은 **빠른 쪽이 느린 쪽을 기다려줘서** 데이터 유실이 없다는 것입니다.

[핵심] 17 장의 M_ARVALID/M_ARREADY, M_WVALID/M_WREADY 등이 모두 이 핸드셰이크 쌍입니다. 코드에서 if (M_ARVALID && M_ARREADY) 같은 조건이 “전송 성립”을 뜻합니다.

5.4 AXI4-Lite —간단한 제어용

AXI 에는 여러 변종이 있는데, 가장 단순한 것이 **AXI4-Lite** 입니다. 한 번에 한 워드 (32 비트) 만 읽거나 씁니다. 레지스터 몇 개를 제어할 때 씁니다.

[핵심] 이 프로젝트에서 MicroBlaze 는 AXI4-Lite 로 yolo_engine 의 제어 레지스터 4 개를 씁니다 (17 장 yolo_engine_axi): - ctrl_reg0: 시작 신호 (ap_start) 와 완료 신호 (network_done) - ctrl_reg1/2/3: 가중치·입력·출력의 DRAM 주소

CPU 가 하는 일 (AXI4-Lite):
 ctrl_reg1 ← 가중치 주소
 ctrl_reg2 ← 입력 주소
 ctrl_reg3 ← 출력 주소
 ctrl_reg0 ← 1 (시작!)
 ... 기다림 ...
 ctrl_reg0 읽기 → bit1 이 1 이면 완료

AXI4-Lite 는 채널이 5 개입니다: 쓰기 주소 (AW), 쓰기 데이터 (W), 쓰기 응답 (B), 읽기 주소 (AR), 읽기 데이터 (R). 각각 valid/ready 핸드셰이크를 씁니다.

5.5 AXI4 (Full) —대량 데이터용 burst

큰 데이터 (특징맵 수만 개) 를 한 워드씩 주고받으면 너무 느립니다. 그래서 **AXI4 Full** 은 **burst(연속 전송)** 를 지원합니다: “이 주소부터 256 개 연속으로 줘”라고 한 번 요청하면, 데이터가 줄줄이 나옵니다.

한 워드씩 (느림): [주소][데이터] [주소][데이터] [주소][데이터] ...
 burst (빠름): [주소 + 길이 256] [데이터][데이터]...[데이터 ×256]
 ↑ 주소 한 번 + 데이터 256 개 연속

[핵심] 이 프로젝트의 DMA 는 **256 워드 (=1KB) burst** 단위로 DRAM 을 읽고 씁니다. 주소를 매번 보내는 오버헤드를 줄여 대역폭을 높입니다. burst 길이는 M_ARLEN(읽기)·M_AWLEN(쓰기) 으로 지정하고, 마지막 워드에 M_RLAST/M_WLAST 신호가 붙습니다.

[비유] 비유: 택배를 한 개씩 주문하면 매번 주소를 적어야 하지만, “이 주소로 256 개 한꺼번에”라고 하면 주소는 한 번만 적고 물건이 줄줄이 옵니다.

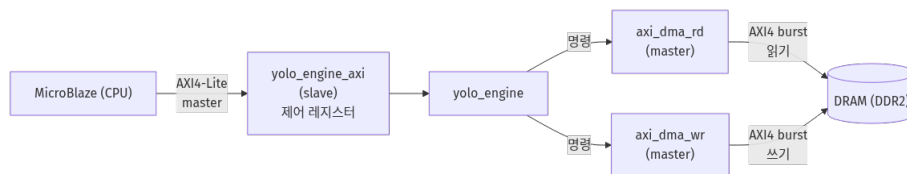
5.6 DMA —CPU 없이 메모리를 옮기기

DMA(Direct Memory Access) 는 CPU 를 거치지 않고 하드웨어가 직접 메모리를 읽고 쓰는 것입니다. CPU 가 일일이 옮기면 느리고 바쁘니, 전용 회로 (DMA 엔진) 가 알아서 옮깁니다.

[핵심] 이 프로젝트의 axi_dma_rd/axi_dma_wr 이 DMA 엔진입니다 (17 장). yolo_engine FSM 이 “이 주소에서 N 워드 읽어”(start_dma, start_addr, num_trans) 라고 시키면, DMA 가 AXI burst 로 DRAM 을 읽어 내부 버퍼에 채웁니다. 다 끝나면 done_o 로 알립니다.

```
yolo_engine FSM: "DRAM 0x1000 에서 768 워드 읽어와!" (start_dma)
↓
axi_dma_rd:      AXI burst 로 DRAM 접근 → 데이터를 line buffer 에 채움
↓
" 다 됐어" (done_o) → FSM 이 다음 단계로
```

5.7 이 프로젝트의 AXI 사용 정리



연결	프로토콜	yolo_engine 역할	용도
CPU ↔ yolo_engine	AXI4-Lite	slave	시작 명령·주소 설정·완료 확인
yolo_engine ↔ DRAM	AXI4 (burst)	master	가중치·입력 읽기, 출력 쓰기

이 두 인터페이스가 17 장의 세 모듈 (yolo_engine_axi, axi_dma_rd, axi_dma_wr) 로 구현됩니다.

5.8 이 장의 요약

- AXI 는 ARM 표준 버스 프로토콜. master(요청) 와 slave(응답) 로 역할이 나뉨.
- 기본 동작은 **valid/ready 핸드셰이크**: 둘 다 1 인 클럭에 전송 성립 (데이터 유실 없음).

- **AXI4-Lite** 는 한 워드씩 주고받는 간단한 제어용 (이 프로젝트의 제어 레지스터 4 개).
- **AXI4 Full** 은 **burst**(256 워드 연속 전송) 로 대량 데이터를 효율적으로 (이 프로젝트의 DMA).
- **DMA** 는 CPU 없이 하드웨어가 직접 메모리를 옮기는 것. `axi_dma_rd/wr` 이 담당.
- `yolo_engine` 은 CPU 에 slave, DRAM 에 master 로 동시에 동작.

다음 장에서는 이 가속기가 최종적으로 풀려는 문제—**YOLO 객체 탐지**의 기초를 배웁니다.

6 장. YOLO 객체 탐지 기초

Part 0 기초 개념—사전 지식: 1 장의 CNN.

이 가속기가 최종적으로 푸는 문제는 **객체 탐지 (object detection)** 입니다. “사진 안에 무엇이, 어디에 있는가”를 찾는 것이죠. 이 장은 YOLO 방식의 탐지 원리를 설명하고, 왜 출력 채널이 195 개 인지, 왜 L18 에서 업샘플하고 L8 과 합치는지를 풀어줍니다. 이것을 알아야 10 장 네트워크 구조의 뒷부분 (검출 헤드·Route) 이 이해됩니다.

6.1 객체 탐지란—분류와의 차이

	이미지 분류 (classification)	객체 탐지 (detection)
질문	“이 사진은 무엇인가?”	“무엇이, 어디에 있는가?”
출력	클래스 한 개 (예: “고양이”)	여러 개의 (박스 + 클래스)
예	“고양이 사진”	“(120,80)~(200,160) 에 고양이, (10,10)~(50,90) 에 개”

탐지는 분류보다 어렵습니다. **위치 (박스)** 까지 찾아야 하기 때문입니다. YOLO(“You Only Look Once”) 는 이를 한 번의 신경망 통과로 빠르게 해냅니다.

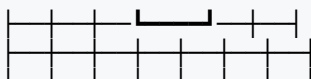
6.2 Grid 기반 탐지—이미지를 격자로

YOLO 의 핵심 아이디어: 이미지를 격자 (**grid**) 로 나누고, 각 격자 칸이 그 위치의 객체를 책임지고 예측하게 합니다.

입력 이미지를 8×8 격자로 (이 네트워크 검출 헤드 1)



각 칸: “ 내 위치에 객체가 있나?
있다면 어떤 박스? 무슨 클래스?”



신경망의 마지막 특징맵의 공간 크기가 곧 격자 크기입니다. 이 네트워크는 8×8(검출 헤드 1, L14)과 16×16(검출 헤드 2, L20) 두 격자를 씁니다.

6.3 Anchor Box —박스 모양의 후보

객체는 모양이 다양합니다 (가로로 긴 차, 세로로 긴 사람). 각 격자 칸이 아무 박스나 예측하면 학습이 어렵습니다. 그래서 미리 **여러 모양의 기준 박스 (anchor)** 를 정해두고, 신경망은 “이 anchor 를 얼마나 조정할지”만 예측합니다.

미리 정한 anchor 모양들 (cfg 의 anchors):
 □ (가로로 긴 것) ▢ (세로로 긴 것) ■ (정사각형) ...
 신경망: “2 번 anchor 를 약간 키우고 오른쪽으로 옮겨” → 실제 박스

이 네트워크는 anchor 가 6 개 있고, 검출 헤드마다 **3 개씩** 나눠 씁니다 (mask 로 지정). 그래서 헤드당 **anchor 3 개**입니다.

6.4 한 예측에 담기는 정보

격자 칸 하나가 anchor 하나에 대해 예측하는 값은 다음과 같습니다.

정보	개수	의미
박스 좌표	4	tx, ty(중심 위치 보정), tw, th(크기 보정)
objectness	1	“여기에 진짜 객체가 있을 확률”
클래스 확률	60	60 개 클래스 각각의 확률
합	65	(= 5 + 60)

즉 anchor 하나당 **65 개** 숫자 (5 + 클래스 60) 를 예측합니다.

6.5 출력 채널 195 의 정체

이제 10 장에서 보게 될 “출력 195 채널”이 어디서 나오는지 계산할 수 있습니다.

출력 채널 = (헤드당 anchor 수) × (5 + 클래스 수)
 = 3 × (5 + 60)
 = 3 × 65
 = 195

[핵심] 그래서 L14·L20 의 출력 채널이 **195** 입니다. 격자 칸 하나 (채널 방향으로 195 개 값) 가 “3 개 anchor × 각 (박스 4 + objectness1 + 클래스 60)”을 담습니다.

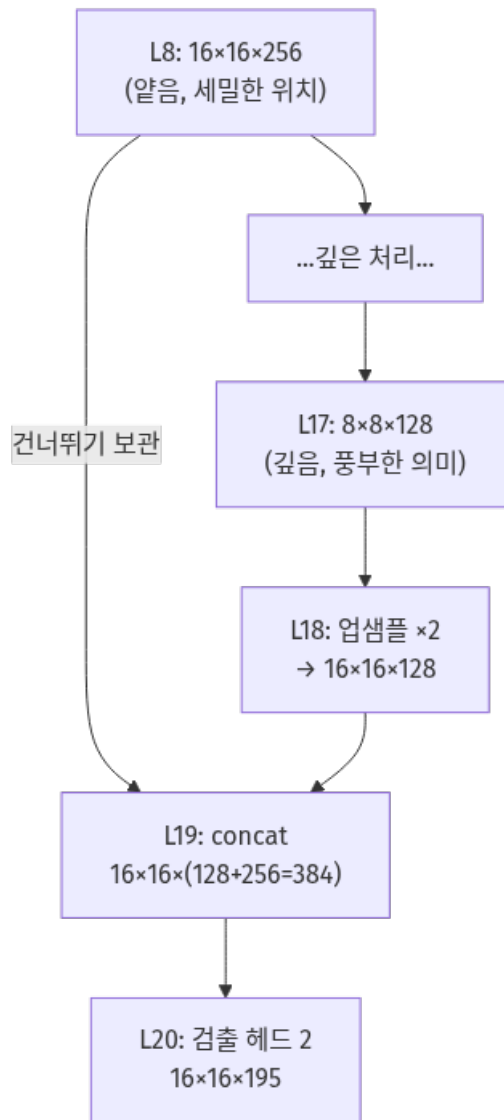
[주의] 하드웨어 관점: 195 는 4 의 배수가 아닙니다 ($195 = 48 \times 4 + 3$). 이 가속기는 데이터를 4 개 단위로 묶어 처리하는데 (12 장), 195 는 딱 떨어지지 않아 마지막에 특별 취급이 필요합니다. 또 검출 헤드는 좌표·확률 (음수 포함) 을 출력해야 해서 **ReLU** 를 **끄고** (linear) INT8 그대로 출력합니다 (2 장 2.8, 14 장).

6.6 2-Scale 탐지와 Skip Connection

이 네트워크는 검출 헤드를 두 개 (8×8 , 16×16) 둡니다. 왜일까요?

- **8×8 격자 (저해상도):** 칸이 커서 큰 객체 탐지에 유리.
- **16×16 격자 (고해상도):** 칸이 작아 작은 객체 탐지에 유리.

문제는, 신경망 깊은 곳 (8×8) 은 “의미 정보”는 풍부하지만 “세밀한 위치 정보”는 잃었다는 것입니다. 그래서 16×16 헤드를 만들 때, 깊은 특징 (L17 → 업샘플) 과 **얕은 곳의 세밀한 특징 (L8)** 을 합칩니다. 이것이 **skip connection**(건너뛰기 연결) 입니다.



[핵심] 이래서 **L18 업샘플** (8×8 깊은 특징을 16×16 으로 키움) 과 **L19 concat**(키운 것 + 보관해둔 L8) 이 필요합니다 (1 장 1.8, 10 장). L8 을 나중에 쓰려고 DRAM 에 보존하는 이유가 이것입니다 (12 장).

[비유] 비유: 멀리서 보면 (8×8) 큰 윤곽은 알지만 디테일이 흐릿하고, 가까이 보면 (L8 의 16×16) 디테일은 보이지만 전체 맥락이 약합니다. 둘을 합치면 “전체 맥락 + 디테일”을 모두 가진 판단이 가능합니다.

6.7 NMS —중복 박스 정리

신경망이 출력한 박스들은 보통 같은 객체에 여러 박스가 겹쳐서 나옵니다 (이웃 격자 칸들이 같은 고양이를 예측). 이걸 정리하는 후처리가 **NMS(Non-Maximum Suppression, 비최대 억제)** 입니다.

NMS 전: 한 고양이에 박스 5 개 겹침 ■■■■
NMS 후: 가장 확신 높은 박스 1 개만 남김 ■

방법: (1) objectness 가 가장 높은 박스를 고르고 (2) 그것과 많이 겹치는 다른 박스들을 제거 (3) 반복. 이 과정은 정렬·비교가 많아 하드웨어보다 소프트웨어가 적합합니다.

6.8 하드웨어 vs 소프트웨어 분담

YOLO 추론 전체에서 무엇을 FPGA 가 하고 무엇을 CPU 가 하는지 정리하면:

단계	담당	이유
합성곱·풀링·업샘플 (L0~L20)	FPGA(RTL)	규칙적·대량 연산 → 병렬 가속 효과 큼
검출 헤드 출력 (L14, L20 OFM)	FPGA	합성곱이므로
Sigmoid·Softmax (확률화)	소프트웨어	비선형 함수, 출력 격자가 작아 빠름
NMS (중복 제거)	소프트웨어	정렬·분기 많음
좌표 디코딩·그리기	소프트웨어	불규칙 처리

[핵심] 그래서 cfg 의 [yolo] 레이어 (L15, L21) 는 RTL 에 없습니다 (10 장). 가속 기는 검출 헤드의 raw 출력 (L14, L20 OFM) 까지만 만들고, 그 다음은 소프트웨어 (MicroBlaze + host.py) 가 처리합니다 (22 장).

6.9 이 장의 요약

- 객체 탐지는 “무엇이 어디에 있는가”를 찾는 것 (분류보다 어려움).
- YOLO 는 이미지를 격자로 나누고, 각 칸이 anchor 를 조정해 박스를 예측.
- 한 예측 = 박스 4 + objectness 1 + 클래스 60 = **65**. anchor 3 개 → 출력 채널 **195** (= 3×65).
- 검출 헤드는 ReLU 없이 (linear) 음수 포함 raw 출력 → 하드웨어에서 특별 취급.
- **2-scale**(8×8 큰 객체, 16×16 작은 객체) + **skip connection**(L18 업샘플 + L8 concat) 으로 정확도 향상 → L8 보존 이유.
- NMS-Sigmoid 등 후처리는 소프트웨어 담당 ([yolo] 레이어는 RTL 에 없음).

이것으로 **Part 0 기초 개념**을 마칩니다. 이제 Part 1 부터는 이 배경지식 위에서 실제 프로젝트와 RTL 을 본격적으로 파고듭니다.

7 장. 프로젝트 개요

이 장을 읽기 위한 준비: 1 장 CNN, 6 장 YOLO 를 보면 더 잘 이해됩니다.

Part 0 에서 배경 지식 (CNN·양자화·FPGA·RTL·AXI·YOLO) 을 익혔으니, 이제 이 프로젝트가 실제로 무엇을 만드는지 큰 그림을 그립니다. 이 장은“전체 지도”이고, 이후 장들이 각 영역을 확대합니다.

7.1 이 프로젝트를 한 문장으로

22-layer YOLOv2 객체 탐지 신경망 전체를, Nexys A7-100T(Artix-7 XC7A100T) FPGA 한 장에서 추론하는 하드웨어 가속기 SoC.

1 장에서 본 합성곱·풀링·업샘플과, 6 장에서 본 YOLO 탐지 구조를, 3 장의 FPGA 부품 (DSP48·BRAM) 으로 구현합니다. 학습은 PC 에서 끝났고, FPGA 는 **추론만** 합니다 (1 장 1.1).

처리 흐름을 거칠게 말하면: 1. 외부 DRAM 에 양자화된 가중치와 입력 이미지가 들어 있음. 2. 가속기가 그것을 읽어 L0 부터 L21 까지 합성곱·풀링·업샘플을 순서대로 계산. 3. 두 검출 헤드 (L14, L20) 의 출력을 DRAM 에 기록. 4. 소프트웨어가 그 출력을 후처리 (Sigmoid·NMS) 하여“무엇이 어디에”목록을 만듦.

7.2 목표와 점수 공식

이 프로젝트는“동작하는”가속기를 넘어 **점수를 최대화**하는 것이 목표입니다. 대회 점수는 다음과 같습니다.

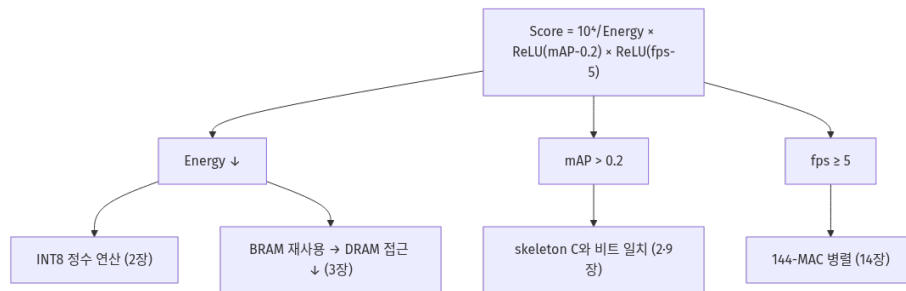
$$\text{Score} = 10^4 / \text{Energy} \times \text{ReLU}(\text{mAP} - 0.2) \times \text{ReLU}(\text{fps} - 5)$$

1 장에서 본 ReLU(음수면 0) 가 여기 쓰입니다. 세 항을 풀어보면:

항	의미	함의
$\text{ReLU}(\text{fps} - 5)$	속도가 5 미만이면 0 점	fps 는 통과 조건 이자, 넘으면 가산 점
$\text{ReLU}(\text{mAP} - 0.2)$	정확도가 0.2 미만이면 0 점	양자화 오차가 이 선을 넘으면 안 됨

항	의미	함의
$10^4 / \text{Energy}$	에너지가 분모	적은 전력으로 끝낼수록 고득점

[핵심] 이 공식이 모든 설계 결정의 뿌리입니다. 왜 INT8 정수를 쓰고 (2 장), 왜 BRAM 에 데이터를 재사용하고 (3 장 3.4), 왜 144 개 MAC 을 병렬로 돌리는지 (14 장) — 모두 “fps ↑, mAP 유지, Energy ↓”를 위한 것입니다.



7.3 타겟 하드웨어

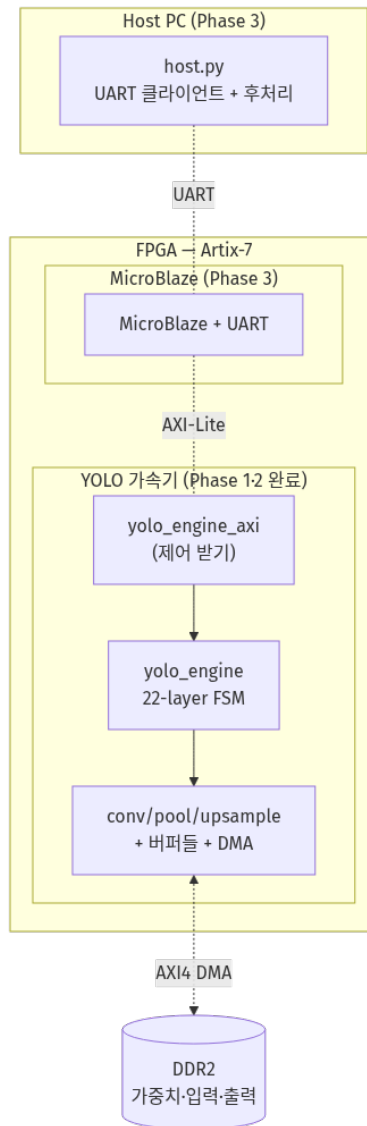
3 장 3.8 에서 본 FPGA 가 이 프로젝트의 무대입니다.

항목	사양	한 줄 설명
보드	Digilent Nexys A7-100T	교육용 Artix-7 보드
FPGA	Xilinx XC7A100T	DSP48 240 개, BRAM ~135 개
외부 메모리	DDR2 (4 MB 사용)	가중치·입력·출력 저장
통신	UART (USB)	PC ↔ MicroBlaze

DSP48 은 여유롭고 (240 개 중 144 개 사용), **BRAM 용량**과 **DDR2 대역폭**이 주요 제약입니다 (3 장 3.6). 이 제약이 “데이터를 잘게 나눠 흘려보내는 (streaming)”설계를 낳습니다 (15·18 장).

7.4 시스템 큰 그림

전체 SoC 를 가장 큰 단위로 그리면 이렇게합니다. 점선 박스는 아직 통합 중인 Phase 3 부분입니다.



5 장 5.7 에서 본 AXI 연결이 여기 나타납니다: CPU 는 AXI-Lite 로 가속기를 제어하고, 가속기는 AXI4 DMA 로 DRAM 과 데이터를 주고받습니다.

7.5 하드웨어와 소프트웨어의 역할 분담

6 장 6.8 에서 본 분담을 다시 정리합니다. 규칙적·대량 연산은 하드웨어, 불규칙·제어는 소프트웨어입니다.

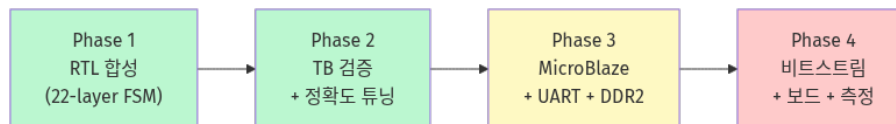
구분	담당	연산
하드웨어 (RTL)	yolo_engine + 서브모듈	합성곱 (3×3·1×1), bias· descaling, ReLU, 풀링 (/2· /1), 업샘플, Route(주소 제어)

구분	담당	연산
소프트웨어	MicroBlaze + host.py	가속기 제어, DMA 트리거, L19 concat 배치, 후처리 (Sigmoid·Softmax·NMS)

[왜?] 왜 후처리는 소프트웨어?: NMS 는 정렬·비교·분기가 많아 하드웨어로 만들면 복잡하고 비효율적입니다. 게다가 검출 헤드 출력 격자가 작아서 (8×8, 16×16) 소프트웨어로도 충분히 빠릅니다. “하드웨어가 잘하는 것 (대량 곱셈) 만 하드웨어로”가 원칙입니다.

7.6 4 단계 개발 로드맵

프로젝트는 4 단계로 진행되며, 현재 **Phase 1·2** 가 완료되었습니다.



Phase	내용	상태
1	yolo_engine 단독 22-layer 자동 추론 RTL 합성	[O] 완료
2	Testbench 검증 + shift 실측 (L0~L18 chain 0 mismatch)	[O] 완료
3	MicroBlaze + UART + DDR2 통합	코드 완성 / 보드 통합 대기
4	비트스트림 + 보드 데모 + fps/mAP/전력 측정	대기

본 해설서는 **Phase 2 완료 (L0~L18 검증)** 시점 기준입니다. L19~L21 과 보드 동작은 22 장 부록에 정리되어 있고, 완료되면 본문으로 승격합니다.

7.7 핵심 설계 결정 미리 보기

이후 장에서 자세히 볼 설계 결정들을 한눈에 모았습니다. “왜?”가 궁금하면 해당 장으로 가십시오.

결정	이유	자세히
INT8 정수 연산	전력·메모리 절약 (Energy↓)	2 장
144-MAC 병렬	한 클럭에 4 픽셀 동시 생성	14 장
가중치 streaming	가중치가 BRAM 보다 커서 필터별로 흘림	13·15 장
1×1 conv 는 3×3 엔진 재사용	모듈 추가 없이 면적 절약	14 장

결정	이유	자세히
L11 풀링 전용 모듈	stride=1 이라 일반 풀링과 다름	16 장
Route 는 모듈 없이 주소 제어	연산이 아니라 데이터 재배치	13 장
Vivado 2025 로 검증 통일	시뮬레이터 X 처리 차이 회피	21 장

7.8 이 장의 요약

- 22-layer YOLOv2 를 Artix-7 FPGA 에서 추론하는 가속기. 학습은 끝났고 추론만 함.
- 점수 = $10^4 / \text{Energy} \times \text{ReLU}(\text{mAP} - 0.2) \times \text{ReLU}(\text{fps} - 5) \rightarrow \text{fps} \cdot \text{mAP}$ 는 통과 조건, Energy 는 분모 $\rightarrow$ INT8·BRAM 재사용·144-MAC 의 뿌리.
- 규칙적 연산 (conv/pool/upsample) 은 하드웨어, 제어·후처리 (NMS 등) 는 소프트웨어.
- Phase 1·2(RTL+ 검증) 완료, Phase 3·4(보드) 대기. 본 문서는 L0~L18 기준.

다음 장에서는 이 시스템을 실제로 빌드·시뮬레이션하는 **개발 환경**을 봅니다.

8 장. 개발 환경과 빌드

이 장을 읽기 위한 준비: 3 장 3.6 합성/시물, 4 장 4.7 X 초기화.

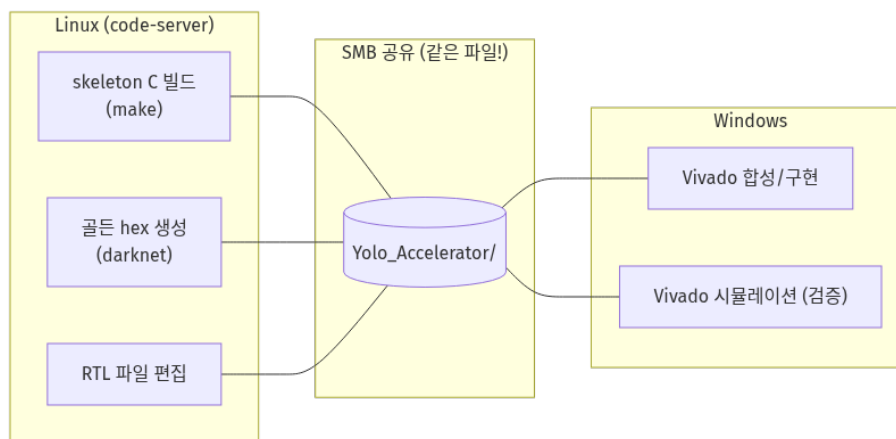
이 장은 프로젝트를 처음 받은 사람이 “무엇을 어디서 빌드하고 돌리는지” 막히지 않게 안내합니다. 이 프로젝트의 가장 독특한 점은 두 개의 **OS** 를 오가며 작업한다는 것입니다. 왜 그런지부터 설명합니다.

8.1 왜 두 개의 환경인가

신경망 가속기 개발에는 성격이 다른 두 가지 작업이 있습니다.

1. **소프트웨어 작업** (C 컴파일, 골든 데이터 생성): 리눅스가 편함.
2. **FPGA 작업** (Vivado 합성·시뮬레이션): Vivado 가 윈도우에서 잘 돌아감.

그래서 이 프로젝트는 **Linux(code-server)** 와 **Windows(Vivado)** 두 환경을 같이 씁니다.



[핵심] 핵심: 두 환경은 **SMB 공유로 같은 파일을 봅니다**. Linux 에서 RTL 을 편집하면 Windows 의 Vivado 가 그대로 읽습니다. 따라서 RTL 편집은 Linux 에서, 합성/시물은 Windows 에서 합니다.

[비유] 비유: 한 폴더 (Google Drive 같은) 를 두 사람이 공유하는 것과 같습니다. 한 명 (Linux) 은 문서를 작성하고, 다른 명 (Windows) 은 그 문서를 출력합니다.

8.2 SMB 공유와 경로 차이

같은 파일이지만 OS 마다 경로 표기가 다릅니다.

```
Linux:    /data/2026 CAU/AIX2026/git/Yolo_Accelerator/yolohw/src/yolo_engine.v
Windows:  Z:\2026 CAU\AIX2026\git\Yolo_Accelerator\yolohw\src\yolo_engine.v
```

[주의] **경로 하드코딩 함정**: 일부 Testbench 나 스크립트에 절대경로가 박혀 있을 수 있습니다. 이 프로젝트의 verify TB 들은 이를 피하려고 **상대경로** (.././.././.././../testbench/...) 로 hex 를 참조합니다 (20 장). Windows 에서 C:/yolohw/... 하드코딩이 필요하면 정션 (junction) 을 만듭니다:

```
mklink /J C:\yolohw "C:\AIX Project\yolohw"
```

8.3 프로젝트 디렉토리 구조

처음 보면 폴더가 많지만, 역할별로 묶으면 단순합니다.

```
Yolo_Accelerator/
├── README / ARCHITECTURE / CLAUDE / HISTORY .md  ← 프로젝트 문서
├── skeleton/                                     ← [소프트웨어] C 골든 레퍼런스 (9 장)
│   ├── src/                                     darknet 소스
│   └── bin/                                     실행파일 + cfg/weights + 생성된 hex
├── documents/                                  ← 강의 PDF + 이 해설서 (study_guide) + technical_reference
└── yolohw/                                     ← [하드웨어] 핵심
    ├── src/                                     * RTL 19 개 .v 파일 (13~17 장)
    ├── testbench/                             * TB + 골든 hex 데이터 (19~20 장)
    ├── sim/                                    시뮬레이션 출력 (자동 생성)
    ├── fpga/                                   Vivado 프로젝트 + IP 스크립트 (21 장)
    └── firmware/                              host.py (22 장)
```

합성·시물 대상은 **yolohw/src/(RTL)** 와 **yolohw/testbench/(TB)** 뿐입니다. 나머지는 데이터·문서·도구입니다.

8.4 skeleton C 레퍼런스 빌드 (Linux)

skeleton 은 “정답”을 만드는 소프트웨어입니다 (2 장에서 본 양자화 추론을 수행, 9 장에서 코드 해설). 빌드는 간단합니다.

```
cd skeleton
make # 컴파일 → ./bin/darknet 실행파일 생성

cd bin/dataset
python make_list_cur.py # 테스트 이미지 경로 목록 갱신 (최초 1 회)
```

make 는 src/의 C 파일들을 컴파일해 bin/darknet 을 만듭니다. 이 실행파일이 양자화 추론과 골든 hex 생성을 담당합니다.

8.5 골든 hex 파일 생성

검증에 쓸 “정답 데이터”를 만드는 단계입니다.

```
cd skeleton/bin

# 이미지 한 장 추론 + 파라미터/특징맵 hex 저장
./darknet detector test yolo11.names aix2024.cfg aix2024.weights \
  -thresh 0.24 test01.jpg -out_filename test01-det-quantized \
  -quantized -save_params
```

명령을 풀어보면: - detector test: 탐지 모드. - aix2024.cfg: 네트워크 구조 (10 장). - aix2024.weights: 학습된 가중치. - -quantized: INT8 양자화 추론 (2 장). - -save_params: 가중치·bias·scale 을 hex 로 저장 ← 검증용 핵심 플래그.

만들어지는 파일:

위치	파일	내용
bin/log_param/	CONVnn_param_weight/bias/scale.hex	양자화 가중치, bias, 시프트값
bin/log_featmap/	CONVnn_input/output.hex	각 레이어 입출력 특징맵 (검증 골든)

이 hex 들이 Testbench 의 입력과 정답이 됩니다 (19 장). scales.hex 의 값이 2 장 2.6 의 descaling shift 를 결정합니다.

8.6 Vivado 합성 (Windows)

RTL 을 실제 FPGA 회로로 만드는 단계입니다 (3 장 3.6). Vivado TCL 콘솔에서:

```
cd yolohw/fpga
source create_project_25.tcl # Vivado 2025 프로젝트 생성
source gen_bram_ips.tcl    # BRAM IP 생성

launch_runs synth_1 -jobs 4 # 합성
launch_runs impl_1 -to_step write_bitstream -jobs 4 # 구현 + 비트스트림
```

이때 **FPGA 매크로**가 켜진 상태여야 DSP48·BRAM IP 가 실제 부품으로 합성됩니다 (아래 8.8). 상세는 21 장.

8.7 Vivado 시뮬레이션 (검증)

실제 칩에 올리기 전, RTL 이 올바른지 PC 에서 확인합니다 (4 장 4.7, 19 장). 이때는 **FPGA 매크로를 꺼서** behavioral 메모리 모델을 씁니다.

```
# Testbench 를 top 으로 지정하고 시뮬레이션 실행
set_property top l18_verify_tb [get_filesets sim_1]
launch_simulation
```

[주의] **log_all_signals** 옵션은 **끄세요**. 커먼 파형 파일 (.wdb) 이 수백 MB 로 폭증해 디스크가 가득 찹니다 (CLAUDE.md 규칙 6).

8.8 FPGA 매크로—가장 자주 하는 실수

user_define_h.v 의 `define FPGA 한 줄이 합성/시뮬을 가릅니다. 이것을 잘못 두면 바로 실패하므로 반드시 기억하세요.

매크로 상태	용도	메모리·곱셈기
`define FPGA 켜짐	합성	실제 BRAM IP, DSP48 (3 장)
//`define FPGA 꺼짐 (주석)	시뮬	behavioral 모델 (4 장 4.7)

```
`ifdef FPGA
    // 합성: 진짜 BRAM/DSP IP
`else
    // 시뮬: reg 배열 + $signed 곱셈 (initial 로 0 초기화)
`endif
```

[주의] **흔한 실수 두 가지**: - 시뮬인데 매크로를 켜둠 → 존재하지 않는 IP 를 찾다가 elaboration 실패. - 합성인데 매크로를 꺼둠 → behavioral 메모리가 LUT 로 합성되어 자원 낭비.

참고: 실제 프로젝트 스크립트는 파일의 `define 은 주석으로 두고, **합성할 때만 Vivado 가 자동으로 매크로를 주입하도록** 되어 있어 파일 수정 없이 모드가 갈립니다 (21 장 15.2).

8.9 검증은 Vivado 2025 로

이것은 권장이 아니라 **검증 정확성의 전제**입니다.

4 장 4.7 에서 본 “uninitialized X”때문에, 같은 RTL 인데 Vivado 버전 (2021 vs 2025) 에 따라 검증 결과가 달라지는 문제가 있었습니다. 해결책은 (1) 시뮬용 메모리를 0 으로 초기화하고 (2) **모든 검증을 Vivado 2025 로 통일**하는 것입니다. 자세한 진단·해결 과정은 21 장에서 다룹니다.

8.10 흔한 함정 체크리스트

#	함정	회피
1	시물인데 FPGA 매크로 커둠	시물 전 주석 처리
2	TB 경로가 Windows 절대경로 하드코딩	상대경로/정선
3	log_all_signals ON → 디스크 폭증	OFF
4	Vivado 2021 로 chain 검증 → 비결정적 mismatch	2025 사용
5	SystemVerilog 구문 (fork 등)	plain Verilog 로
6	bias 를 zero-extend 로 적재	sign-extend (2 장 2.5)

이 함정들은 CLAUDE.md §4 의“치명적 에러 방지 규칙”에서 왔습니다. 한 번씩 당해본 실수들이니 미리 알아두면 시간을 아낍니다.

8.11 이 장의 요약

- **Linux(C 빌드·hex 생성·RTL 편집) + Windows(Vivado 합성·시물)** 두 환경을 SMB 공유로 같은 파일을 보며 사용.
- make → darknet ... -save_params 로 골든 hex(가중치·특징맵) 생성 → 검증 데이터.
- 합성은 FPGA 매크로 ON + create_project_25.tcl, 시물은 매크로 OFF.
- 검증은 **Vivado 2025** 로 통일 (X 처리 차이 회피), log_all_signals OFF.
- 6 가지 함정 (매크로·경로·디스크·버전·구문·sign-extend) 을 미리 숙지.

다음 장에서는 골든 데이터를 만드는 **skeleton C 코드**를 직접 들여다봅니다.

9 장. skeleton C 골든 레퍼런스

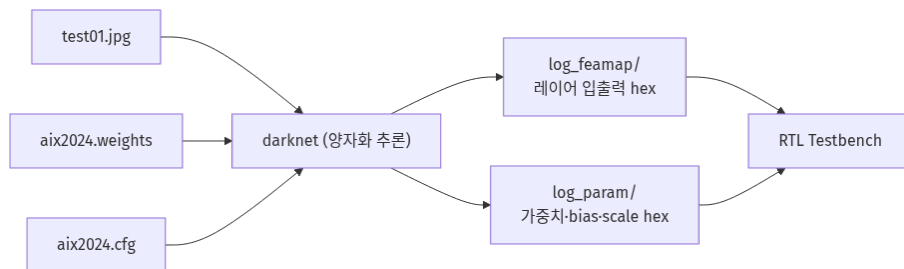
이 장을 읽기 위한 준비: 2 장 양자화 전체. 이 장은 2 장의 내용을 실제 C 코드로 확인합니다.

검증의 기준은“RTL 이 skeleton C 와 같은 결과를 내는가”입니다 (2 장 2.10). 그러니 이 C 코드를 이해하는 것이 검증을 이해하는 출발점입니다. 이 장은 2 장에서 개념으로 배운 양자화 추론을 실제 C 코드로 한 줄씩 따라갑니다.

9.1 skeleton 의 두 가지 역할

skeleton 은 Darknet(YOLO 원저자의 C 프레임워크) 을 변형한 것으로, 두 가지를 합니다.

1. 양자화 추론: 입력을 INT8 로 바꿔 22-layer 를 정수로 계산 → FPGA 가 흉내 낼 “정답”.
2. 골든 hex 덤프: 각 레이어 입출력과 양자화 파라미터를 hex 파일로 저장 → Testbench 가 읽음.



핵심 파일은 **yolov2_forward_network_quantized.c**(정수 추론 + 덤프) 와 **additionally.c**(양자화 계수 + 파라미터 저장) 입니다.

9.2 핵심 함수—forward_convolutional_layer_q()

한 합성곱 레이어의 정수 추론 전체가 이 함수에 있습니다. 2 장 2.9 의 흐름도와 단계마다 1:1 로 대응하니, 비교하며 읽으세요.

[코드] 코드 해설 (1) —입력 양자화

```
state.input_uint8 = (uint8_t*)calloc(l.inputs, sizeof(uint8_t));
for (z = 0; z < l.inputs; ++z) {
    int16_t src = state.input[z] * l.input_quant_multiplier; // 실수 × scale
    state.input_uint8[z] = max_abs(src, MAX_VAL_UINT8); // 0~255 로 클립
}
```

- state.input[z]: 실수 입력값 (이전 레이어의 출력).
- × l.input_quant_multiplier: 2 장 2.3 의 input scale(L0=128, 나머지 =8) 을 곱함.
- max_abs(src, 255): 0~255 범위로 클립 (2 장 2.2). 이것이 UINT8 입력.

[코드] 코드 해설 (2) —im2col + GEMM (MAC 의 본체)

```
im2col_cpu_int8(state.input_uint8, l.c, l.h, l.w, l.size, l.stride, l.pad, b);
// ...
gemm_nn_int8_int32(1, n, k, 1, a + t*k, k, b, n, c + t*n, n);
```

- im2col: 합성곱을 행렬 곱셈으로 바꾸기 위해, 각 출력 위치가 보는 입력 윈도우를 **한 줄로 펼칩니다** (1 장 1.3 의 3×3 윈도우를 1 차원으로). 소프트웨어의 흔한 기법입니다.
- gemm_nn_int8_int32: 펼친 입력과 가중치를 곱해 더하는 행렬 곱셈 (General Matrix Multiply). 이름 그대로 **INT8 × INT8 → INT32** 누적입니다 (2 장 2.4).

[주의] **하드웨어와의 차이**: 소프트웨어는 im2col 로 메모리에 펼치지만, FPGA 는 메모리가 비싸서 **라인 버퍼**가 윈도우를 측석에서 만듭니다 (15 장). 결과 (MAC 누적) 는 같습니다.

gemm 내부를 보면 누적이 INT32 인 이유가 드러납니다:

```
int32_t *c_tmp = calloc(N, sizeof(int32_t)); // INT32 누적기
register int16_t A_PART = ALPHA*A[i*lda + k]; // 가중치 (INT16 여유)
c_tmp[j] += A_PART*B[k*ldb + j]; // 곱 누적 → INT32
```

곱셈 하나는 작지만, 수천 개를 더하면 넘치므로 INT32 에 모읍니다 (2 장 2.4).

[코드] 코드 해설 (3) —bias, ReLU, descaling

```
// bias 덧셈
output_q[fil*out_size + j] = output_q[...] + l.biases_quant[fil];

// ReLU (활성화가 RELU 일 때만)
if (l.activation == RELU)
    output_q[i] = (output_q[i] > 0) ? output_q[i] : 0;

// descaling (부동소수점으로 복원)
float ALPHA1 = 1 / (l.input_quant_multiplier * l.weights_quant_multiplier);
l.output[i] = output_q[i] * ALPHA1;
```

- bias 를 더하고 (2 장 2.5), ReLU 로 음수를 0 으로 (2 장 2.7→ 1 장 1.6).
- **descaling**: C 는 input_scale × weight_scale 로 나뉘서 **부동소수점으로 복원**합니다.

[핵심] 여기가 **C** 와 **RTL** 의 가장 큰 표현 차이입니다. C 는 float 으로 복원했다가 다음 레이어에서 다시 정수화합니다. RTL 은 float 이 없으므로 이“복원 후 재양자화”를 **하나의 시프트**로 합칩니다 (2 장 2.6). 그 시프트량이 다음에 볼 scale 입니다.

[코드] 코드 해설 (4) —골든 덤프 (NCHW 순서)

```
// 출력 특징맵을 hex 로 저장
for (int chn = 0; chn < l.n; chn++) { // 채널 바깥 루프
    for (int idx = 0; idx < out_size; idx++) { // 한 채널 안의 픽셀
        int16_t src = l.output[i] * next_input_quant_multiplier; // 다음 레이어 scale 로 재양자화
        uint8_t pixel = max_abs(src, MAX_VAL_UINT_8);
        fprintf(fp, "%02x\\n", pixel); // 1 바이트씩 기록
    }
}
```

- 채널마다 한 특징맵 전체 (**H×W**) 를 쪽 쓰고 다음 채널로 → 이것이“NCHW byte order”(12장).
- $\times$ next_input_quant_multiplier: 이 레이어 출력을 다음 레이어 입력 **scale** 로 재양자화해서 저장 → 그래서 한 레이어의 output.hex 가 그대로 다음 레이어의 input.hex 가 됩니다.

[주의] 검출 헤드 예외: L14·L20 은 activation=linear 라 ReLU 를 건너뛰고, 다음 conv 가 없어 next_input_quant_multiplier=1 이 됩니다. 그래서 INT8 raw 출력입니다 (6 장 6.5).

9.3 양자화 계수—do_quantization()

2 장 2.3 의 multiplier 표가 어디서 오는지 코드로 확인합니다.

```
float weight_quant_multiplier[11] = { 16, 64, 64, 64, 64, 64, 64, 64, 64, 64, 64 };
float input_quant_multiplier [11] = { 128, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8 };
```

- 11 개 conv 레이어 각각의 scale. L0 만 input 128/weight 16, 나머지는 input 8/weight 64.

```
// 가중치 양자화
float w = l->weights[...] * l->weights_quant_multiplier; // 실수  $\times$  scale
l->weights_int8[...] = max_abs(w, MAX_VAL_8); // -128~127 클립

// bias 양자화 —두 scale 의 곱으로
float biases_multiplier = (l->weights_quant_multiplier * l->input_quant_multiplier);
l->biases_quant[fil] = max_abs(b, MAX_VAL_16); // INT16 클립
```

bias 가 weight_scale $\times$ input_scale 로 커지는 이유는 2 장 2.5 에서 설명했습니다—MAC 결과와 스케일을 맞추기 위함입니다.

9.4 scale → RTL shift —save_quantized_model()

이 함수가 만드는 scale 값이 **RTL** 의 **descaling shift** 를 직접 결정합니다 (2 장 2.6).

[코드] 코드 해설—scale 계산

```
int scale = (l->input_quant_multiplier * l->weights_quant_multiplier)
            / next_input_quant_multiplier;
fprintf(fp_s, "%04x\n", (uint16_t) scale);
```

- $\text{input_scale} \times \text{weight_scale}$: MAC+bias 결과가 부풀어 있는 배율.
- $/ \text{next_input_quant_multiplier}$: 다음 레이어 입력 scale 로 되돌리기 위한 나눗셈.
- 이 scale 이 **2** 의 거듭제곱이라, RTL 은 $\log_2(\text{scale})$ 만큼 시프트하면 됩니다.

직접 계산해보면 (2 장 2.6):

레이어	input × weight / next	scale	RTL shift = $\log_2$
L0	128×16/8	256	8
L2~L13, L17	8×64/8	64	6
L14	8×64/8 (다음 conv = L17)	64	6
L20	8×64/1 (다음 conv 없음)	512	9

[핵심] 이 shift 값들이 13 장의 L0_SHIFT=8, L2_SHIFT=6 같은 RTL 파라미터입니다.

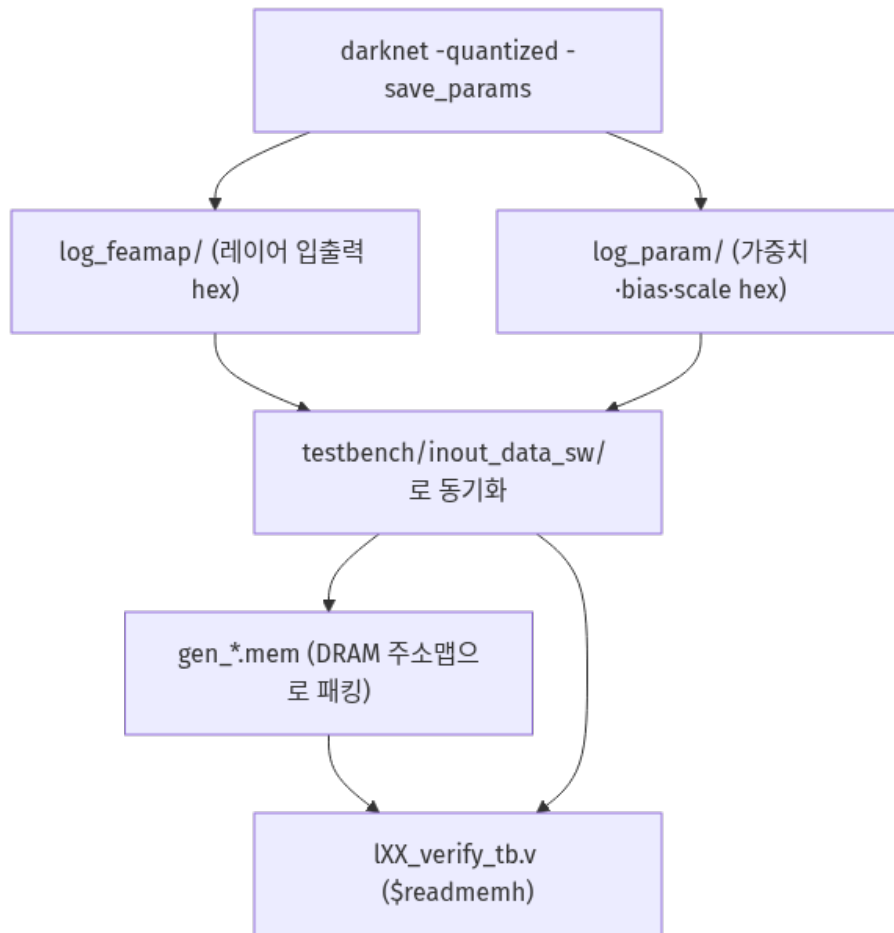
Phase 2 에서 이 scales.hex 를 실측해 RTL 에 반영했습니다.

가중치·bias 도 같은 함수에서 저장합니다.

```
fprintf(fp_w, "%02x\n", (uint8_t) l->weights_int8[w_index]); // INT8 가중치, 1 바이트
fprintf(fp_b, "%04x\n", (uint16_t) l->biases_quant[f]);       // INT16 bias, 2 바이트
```

9.5 골든 데이터 워크플로우

skeleton 출력이 Testbench 에 도달하는 전체 경로입니다.



- gen_wgt_dram.mem, gen_bias_dram.mem, gen_ifm_dram.mem: DRAM에 적재할 형태로 패킹된 입력.
- log_feamap/CONVnn_output.hex: 비교용 정답 (골든).

TB는 .mem을 DRAM 모델에 넣고 RTL을 돌린 뒤, 결과를 output.hex와 대조합니다 (19장).

9.6 이 장의 요약

- skeleton C(특히 yolov2_forward_network_quantized.c)는 RTL이 따라야 할 정수 추론 골든.
- forward_convolutional_layer_q()는 2장의 흐름을 코드로 구현: 입력양자화 → im2col → INT32 GEMM → bias → ReLU → descaling → NCHW 덤프.
- do_quantization()이 레이어별 scale을, save_quantized_model()이 scale = input×weight/next를 계산 → 이것이 **RTL shift**(L0=8, 대부분 =6, 헤드 =9).
- 출력은 다음 레이어 scale로 재양자화되어 저장 → 레이어 간 자연 연결.
- 이 hex들이 inout_data_sw/로 동기화되어 Testbench의 입력·정답이 됨.

다음 장에서는 이 코드가 정의하는 **22-layer 네트워크의 구조**를 봅니다.

10 장. 22-Layer 네트워크 구조

이 장을 읽기 위한 준비: 1 장 CNN, 6 장 YOLO.

이 장은 가속기가 추론하는 신경망의 “지도”를 그립니다. 어떤 레이어가 몇 번째에 있고, 무슨 일을 하며, 데이터가 어떻게 흐르는지를 aix2024.cfg 파일에 근거해 봅니다. 이 지도가 머릿속에 있으면 13 장의 FSM 이 “지금 몇 번 레이어를 처리 중”인지 따라갈 수 있습니다.

10.1 cfg 파일 읽는 법

Darknet 은 네트워크 구조를 .cfg 텍스트 파일로 정의합니다. [블록] 하나가 레이어 하나이고, 파일에 적힌 순서가 곧 레이어 번호 (**layer_idx**) 입니다.

[convolutional]	← 합성곱 레이어
batch_normalize=1	배치 정규화 사용
filters=16	출력 채널 16 개
size=3	3×3 커널
stride=1	보폭 1
pad=1	패딩 1
activation=relu	ReLU 활성화
[maxpool]	← 풀링 레이어
size=2	2 칸씩 (크기 절반)
stride=2	

1 장에서 배운 개념들 (filters= 출력 채널, size= 커널, stride= 보폭, pad= 패딩, activation=ReLU) 이 그대로 적혀 있습니다.

[비유] cfg 를 위에서 아래로 읽으면 데이터가 신경망을 통과하는 순서입니다. 첫 블록이 L0, 다음이 L1, ...식입니다. 주석 처리된 #[convolutional] 블록은 번호에서 빠집니다.

10.2 cfg 를 레이어로 펼치기

aix2024.cfg 의 블록을 순서대로 레이어 번호와 맞추면:

```

[net] width=256 height=256 channels=3      → 입력 256×256×3

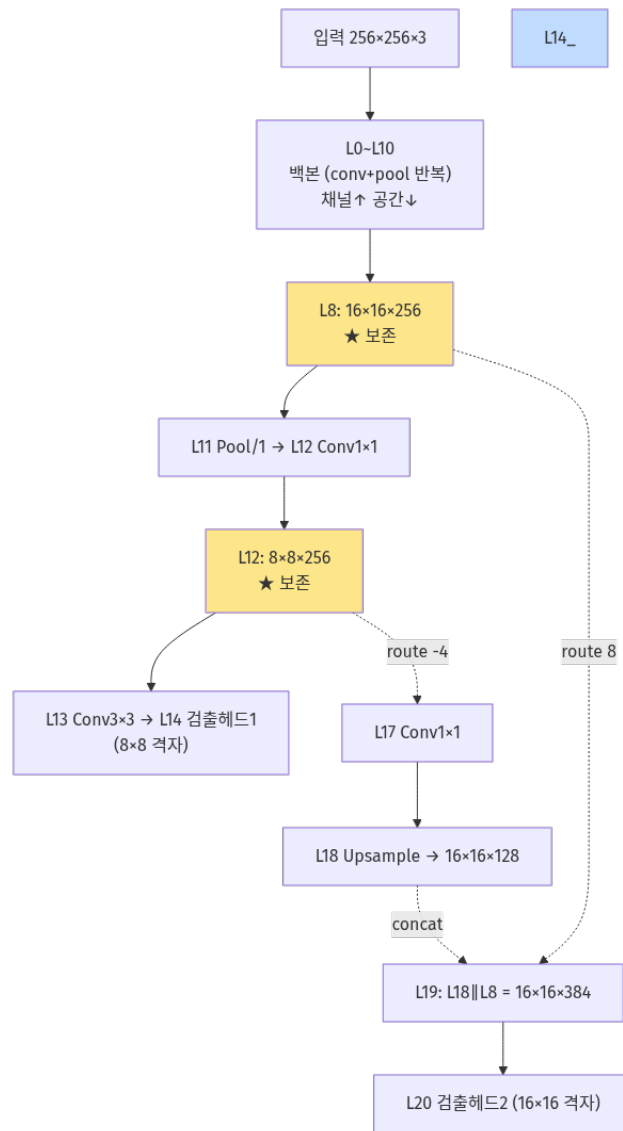
[convolutional] filters=16 size=3 → L0 Conv3×3, 출력 16 채널
[maxpool] stride=2 → L1 Pool/2 (크기 절반)
[convolutional] filters=32 size=3 → L2 Conv3×3, 32 채널
[maxpool] stride=2 → L3 Pool/2
[convolutional] filters=64 size=3 → L4 Conv3×3, 64 채널
[maxpool] stride=2 → L5 Pool/2
[convolutional] filters=128 size=3 → L6 Conv3×3, 128 채널
[maxpool] stride=2 → L7 Pool/2
[convolutional] filters=256 size=3 → L8 Conv3×3, 256 채널 *(나중에 재사용)
[maxpool] stride=2 → L9 Pool/2
[convolutional] filters=512 size=3 → L10 Conv3×3, 512 채널
[maxpool] stride=1 → L11 Pool/1 *(stride=1! 크기 유지)
[convolutional] filters=256 size=1 → L12 Conv1×1, 256 채널 *(나중에 재사용)
[convolutional] filters=512 size=3 → L13 Conv3×3, 512 채널
[convolutional] filters=195 size=1 activation=linear → L14 검출 헤드 1
[yolo] → L15 (소프트웨어 후처리)
[route] layers=-4 → L16 (L12 를 가리킴: 16-4=12)
[convolutional] filters=128 size=1 → L17 Conv1×1, 128 채널
[upsample] stride=2 → L18 Upsample×2 (크기 2 배)
[route] layers=-1,8 → L19 (L18 과 L8 을 합침)
[convolutional] filters=195 size=1 activation=linear → L20 검출 헤드 2
[yolo] → L21 (소프트웨어 후처리)

```

[핵심] **[route]** 읽는 법: layers=-4 는“4 칸 뒤 레이어”(16-4=12) 를 가리킵니다. layers=-1,8 은“직전 (18) 과 8 번”을 합치라는 뜻입니다 (6 장 6.6 skip connection).

10.3 네트워크 토폴로지 (데이터 흐름)

위 레이어들의 연결을 그림으로 그리면 skip connection 이 보입니다.



- **L0~L10**: 백본. 합성곱과 풀링을 번갈아 하며 채널을 늘리고 공간을 줄입니다 ($256^2 \rightarrow 8^2$).
- **L8, L12**: 나중에 재사용되므로 DRAM에 보존합니다 (6 장 6.6).
- 두 검출 헤드: L14(8×8, 큰 객체) + L20(16×16, 작은 객체).

10.4 레이어별 상세 표

각 레이어의 형상과 양자화 파라미터입니다. 형상은 stride·pad 로 계산했고 (1 장 1.4), shift 는 9 장 9.4 에서 구한 값입니다.

idx	연산	입력 H×W×C	출력 H×W×C	커널	활성화	shift
L0	Conv3×3	256×256×3	256×256×16	3/1/1	relu	8
L1	Pool/2	256×256×16	128×128×16	2/2	-	-
L2	Conv3×3	128×128×16	128×128×32	3/1/1	relu	6

idx	연산	입력 H×W×C	출력 H×W×C	커널	활성화	shift
L3	Pool/2	128×128×32	64×64×32	2/2	-	-
L4	Conv3×3	64×64×32	64×64×64	3/1/1	relu	6
L5	Pool/2	64×64×64	32×32×64	2/2	-	-
L6	Conv3×3	32×32×64	32×32×128	3/1/1	relu	6
L7	Pool/2	32×32×128	16×16×128	2/2	-	-
L8	Conv3×3	16×16×128	16×16×256	3/1/1	relu	6
L9	Pool/2	16×16×256	8×8×256	2/2	-	-
L10	Conv3×3	8×8×256	8×8×512	3/1/1	relu	6
L11	Pool/1	8×8×512	8×8×512	2/1	-	-
L12	Conv1×1	8×8×512	8×8×256	1/1	relu	6
L13	Conv3×3	8×8×256	8×8×512	3/1/1	relu	6
L14	Conv1×1	8×8×512	8×8×195	1/1	linear	6
L15	YOLO	-	(검출)	-	-	-
L16	Route(=L12)	-	8×8×256	-	-	-
L17	Conv1×1	8×8×256	8×8×128	1/1	relu	6
L18	Upsample×2	8×8×128	16×16×128	-	-	-
L19	Route concat	-	16×16×384	-	-	-
L20	Conv1×1	16×16×384	16×16×195	1/1	linear	9
L21	YOLO	-	(검출)	-	-	-

[주의] **L11** 이 유일하게 **stride=1** 풀링입니다. 크기가 안 줄고 (8×8 유지) 윈도우가 겹쳐서, 일반 풀링과 동작이 달라 전용 하드웨어가 필요합니다 (1 장 1.7, 16 장).

10.5 연산은 세 종류로 나뉜다

22 개 레이어는 하드웨어 관점에서 세 부류입니다. 이 분류가 11 장의 모듈 배정을 결정합니다.

분류	레이어	하드웨어
합성곱 (3×3, 1×1)	L0,2,4,6,8,10,12,13,14,17,20	conv_top + 144-MAC (14 장)
형상 변환 (pool, upsample)	L1,3,5,7,9 (/2), L11 (/1), L18 (×2)	max_pool_unit / _s1 / upsample_unit (16 장)
재배치 (route, yolo)	L15,16,19,21	연산 없음 (주소 제어 / 소프트웨어)

합성곱이 11 개로 가장 많고, 연산량의 대부분 (88%) 을 차지합니다.

10.6 출력 채널 195 의 의미 (복습)

L14~L20 의 195 채널은 6 장 6.5 에서 계산했습니다.

$$195 = 3 \text{ (헤드당 anchor)} \times 65 \text{ (= 5 박스 \cdot objectness + 60 클래스)}$$

검출 헤드는 `activation=linear` 라 ReLU 없이 음수 포함 INT8 로 출력하고, 195 가 4 의 배수가 아니라 하드웨어에서 특별 취급합니다 (12 장, 14 장).

10.7 연산량—어디에 시간이 쓰이나

각 합성곱의 곱셈 (MAC) 횟수를 계산하면 (1 장 1.3: 출력공간 $\times$ 출력채널 $\times k^2 \times$ 입력채널):

레이어	MAC	비중
L0	28.3 M	5%
L2,4,6,8,10,13 (각)	75.5 M	각 15%
L12	8.4 M	2%
L14	6.4 M	1%
L17	2.1 M	0.4%
L20	19.2 M	4%
합계	$\approx 517 \text{ M}$	100%

[핵심] 흥미로운 균형: L2~L4~L6~L8~L10~L13 이 모두 75.5 M 으로 같습니다. 공간을 절반으로 줄이면서 채널을 두 배로 늘리니, 합성곱 비용이 일정하게 유지되는 것입니다. 이 6 개 레이어가 전체의 88% 를 차지하므로, 이들을 빠르게 처리하는 것이 **fps** 의 핵심입니다 (18 장).

메모리 관점에서는 **L0** 의 출력 ($256 \times 256 \times 16 = 1 \text{ MB}$) 이 가장 큼니다. 그래서 큰 레이어는 온 칩 버퍼 (256 KB) 에 한 번에 못 담고 잘게 나눠 처리합니다 (15~18 장).

10.8 이 장의 요약

- `cfg` 의 [블록] 순서가 곧 레이어 번호. 합성곱·풀링·route·yolo 로 구성.
- 백본 (L0~L10) 이 채널 $\uparrow$ ·공간 $\downarrow$, 두 검출 헤드 (L14 8×8 , L20 16×16) 가 큰/작은 객체 담당.
- L8~L12 는 skip/route 로 재사용되어 DRAM 보존. L11 은 유일한 stride=1 풀링 (전용 HW).
- 연산은 합성곱 (11 개, 88%) / 형상변환 (pool·upsample) / 재배치 (route·yolo) 로 갈림.

- 출력 195 = 3 anchor × 65. 총 ≈517 M MAC, 6 개 3×3 레이어 (각 75.5 M) 가 88%.

다음 장부터 본격적으로 하드웨어로 들어가, 이 레이어들을 처리하는 **모듈 구조**를 봅니다.

11 장. 하드웨어 아키텍처 개요

이 장을 읽기 위한 준비: 3 장 FPGA, 1 장 1.10 MAC.

이제 RTL 의 세계로 들어갑니다. 이 장은“숲”을 봅니다—어떤 모듈들이 있고, 무엇이 무엇을 품고, 데이터가 어디로 흐르는지. 개별 모듈의“나무”는 13~17 장에서 하나씩 봅니다. 먼저 전체 지도를 머리에 넣어야 길을 잃지 않습니다.

11.1 19 개의 RTL 파일

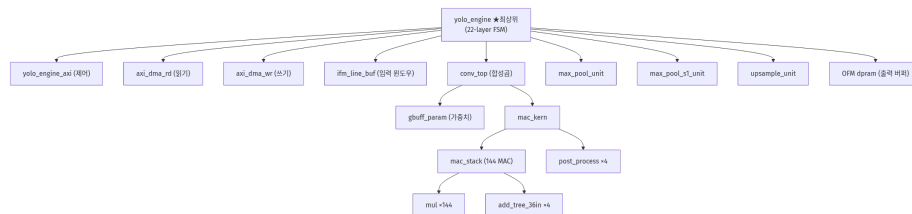
합성·시뮬 대상은 yolohw/src/의 **19 개.v** 파일입니다. 역할별로 묶으면 외울 만합니다.

묶음	파일	한 줄 역할	자세히
최상위	yolo_engine.v	22-layer 를 지휘하는 거대 FSM	13 장
연산 코어	conv_top.v	합성곱 한 호출을 제어	14 장
	mac_kern.v	144-MAC + 누적 + 후처리	14 장
	mac_stack.v	144 개 곱셈 + 4 개 가산트리	14 장
	mul.v	INT8×INT8 곱셈기	14 장
	add_tree_36in.v	36 입력 덧셈 트리	14 장
	post_process.v	bias·ReLU·시프트·클립	14 장
메모리	ifm_line_buf.v	입력 윈도우 만들기	15 장
	gbuff_param.v	가중치·bias 저장	15 장
	dpram_wrapper.v	BRAM 래퍼	15 장
	spram_wrapper.v		
특수 연산	max_pool_unit.v	풀링 (/2, /1)	16 장
	upsample_unit.v	업샘플 ×2	16 장

목록	파일	한 줄 역할	자세히
외부 통신	axi_dma_rd.v	DRAM 읽기/쓰기 DMA	17 장
	axi_dma_wr.v		
설정	yolo_engine_axi.v	CRU 제어 받기	17 장
	user_define.h.v	매크로	17 장
	/ define.v		

11.2 모듈은 어떻게 포개져 있나 (계층)

yolo_engine 이 모든 것을 품고, 연산 코어는 인형 안의 인형처럼 4 단으로 중첩됩니다.

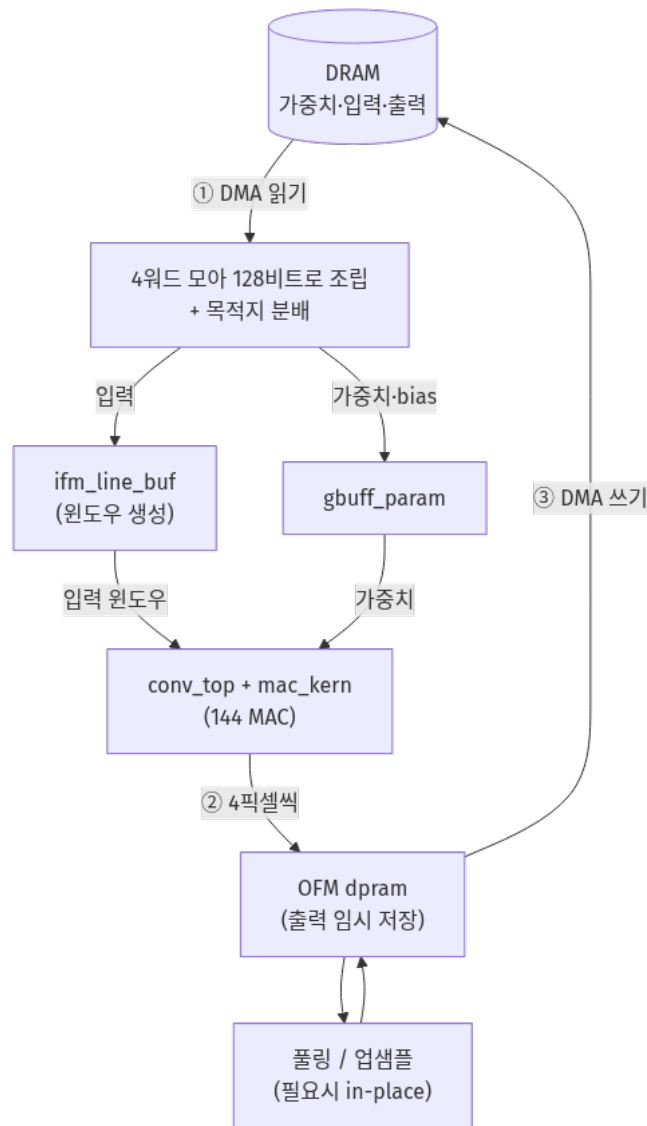


[핵심] **핵심:** 연산 유닛 (conv_top, 풀링, 업샘플) 은 각각 딱 하나씩만 존재합니다. yolo_engine 이 22 개 레이어를 돌며 같은 유닛을 **재사용**합니다. 매 레이어마다 다른 파라미터 (크기·채널·shift) 를 끼워주는 식입니다. 이것이 면적·전력을 아끼는 핵심입니다 (3 장 3.5).

[비유] **비유:** 요리사 (conv_top) 한 명이 22 개 요리 (레이어) 를 순서대로 합니다. 요리마다 레시피 (파라미터) 만 바뀌어서 같은 주방 도구를 씁니다. 주방을 22 개 만들 필요가 없습니다.

11.3 데이터는 어떻게 흐르나

한 레이어를 처리할 때 데이터가 도는 경로입니다. DRAM ↔ 가속기 사이는 5 장에서 배운 AXI DMA 가 담당합니다.



순서를 말로 풀면: 1. 읽기: DRAM 에서 가중치·입력을 DMA 로 읽어, 가중치는 gbuff_param 에, 입력은 ifm_line_buf 에 넣습니다. 2. 연산: mac_kern 이 144 개 곱셈으로 한 클럭에 **4 픽셀 (2×2 블록)** 을 만들어 OFM 버퍼에 씁니다. 3. 형상 변환: 풀링·업샘플 레이어는 이 OFM 버퍼를 직접 읽고 씁니다. 4. 쓰기: 레이어가 끝나면 OFM 버퍼를 DRAM 에 DMA 로 기록합니다.

이“읽기 → 연산 → 쓰기”가 22 번 반복되면 추론 한 장이 끝납니다 (18 장).

11.4 144-MAC —가속의 심장 (미리 보기)

1 장 1.10 에서“합성곱은 MAC 의 산”이라고 했습니다. 이 가속기는 그 MAC 을 **144 개 동시에** 합니다.

$$144 = 36 \times 4$$

$\begin{array}{l} \text{└─ 4 개 " 공간 위치" (출력 2×2 블록의 네 칸)} \\ \text{└─ 36 개 곱셈 (= 4 입력채널 × 3×3 커널)} \end{array}$

- **36 개**: 한 출력점은 $3 \times 3 \times$ 입력채널개 곱셈이 필요한데 (1 장 1.3), 입력채널 4 개씩 묶어 한 번에 처리 $\rightarrow 4 \times 9 = 36$.

- **4 벌:** 같은 가중치로 인접한 2×2 출력 위치 4 개를 동시에 계산 → 한 클럭에 4 픽셀.

한 클럭에: 입력 윈도우 → [36 곱셈] ×4 위치 → [덧셈 트리] ×4 → 4 픽셀

입력 채널이 4 보다 많으면 (거의 모든 레이어), 4 개씩 나눠 여러 클럭에 걸쳐 누적합니다. 예를 들어 입력 채널 256 개면 $256/4 = 64$ 번 누적. 자세한 동작과 코드는 14 장에서 한 줄씩 봅니다.

[핵심] 10 장 10.7 에서 본 총 517 M MAC 을 144 개씩 처리하면 이론상 약 360 만 클럭입니다. 실제로는 DMA·제어 오버헤드가 더해집니다 (18 장).

11.5 어느 레이어를 어느 모듈이 처리하나

10 장 10.5 의 분류를 모듈 관점으로 다시 보면:

모듈 (1 개씩만 존재)	처리 레이어	호출 횟수
conv_top (+ mac_kern 등)	L0,2,4,6,8,10,13 (3×3) ·L12,14,17,20 (1×1)	11
max_pool_unit	L1,3,5,7,9	5
max_pool_s1_unit	L11	1
upsample_unit	L18	1
(모듈 없음)	L15,16,19,21 (route/yolo)	—

3×3 과 1×1 합성곱은 같은 **conv_top** 을 씁니다. 차이는 입력을 모아주는 방식뿐입니다 (14 장 14.9).

11.6 온칩 메모리—무엇을 어디에 두나

3 장 3.4 에서 본 BRAM 을 세 용도로 씁니다.

버퍼	크기	용도
가중치 버퍼 (gbuff_param)	36 KB + bias 10 KB	현재 레이어의 가중치
입력 라인 버퍼 (ifm_line_buf)	128 비트 × 2048 × 4 벌 (~1 MB)	합성곱 윈도우용 입력 행들
출력 버퍼 (OFM dpram)	32 비트 × 65536 (256 KB)	합성곱 출력 + 풀링/업샘플 작 업장

전체 입력·출력·가중치는 너무 커서 DRAM 에 두고 (10 장 10.7: L0 출력만 1 MB), 현재 처리 중인

부분만 **BRAM** 에 올립니다. DRAM 접근을 줄이는 것이 Energy 절약의 핵심이기 때문입니다 (3 장 3.4, 7 장 7.2). 정확한 비트 배치는 12 장에서.

11.7 이 장의 요약

- 활성 RTL 은 19 개. yolo_engine 이 모든 모듈을 품고 FSM 으로 지휘.
- 연산 코어는 4 단 중첩: conv_top → mac_kern → mac_stack → (mul ×144 + add_tree ×4) + post_process.
- 데이터 흐름: **DMA** 읽기 → (라인버퍼·가중치버퍼) → **144-MAC** → 출력버퍼 → **DMA** 쓰기를 레이어마다 반복.
- 연산 유닛은 1 개씩만 두고 22-layer 에 재사용 (파라미터 mux) → 면적·전력 절약.
- 144-MAC = 36(4 채널 ×3×3) × 4(2×2 출력) → 한 클럭 4 픽셀. 채널 4 초과는 누적.
- BRAM 은 가중치·입력라인·출력 버퍼로 사용, 큰 데이터는 DRAM 에 두고 일부만 올림.

다음 장에서는 이 데이터가 비트 단위로 어떻게 표현되고 메모리에 배치되는지 정밀하게 봅니다.

12 장. 데이터 표현과 메모리 맵

이 장을 읽기 위한 준비: 2 장 양자화, 1 장 1.2 특징맵.

하드웨어 디버깅에서 mismatch 가 났을 때 가장 먼저 의심하는 것이 데이터 포맷입니다. 비트 하나만 어긋나도 결과가 완전히 달라지기 때문입니다. 이 장은 가속기를 흐르는 데이터가 비트 단위로 어떻게 생겼고 메모리 어디에 놓이는지를 그림으로 설명합니다. 처음엔 지루해 보여도, 14~16 장 코드를 읽을 때 계속 되돌아올 장입니다.

12.1 수 표현—비트폭이 단계마다 커진다

2 장에서 배운 양자화 수들이 하드웨어에서 갖는 비트폭입니다.

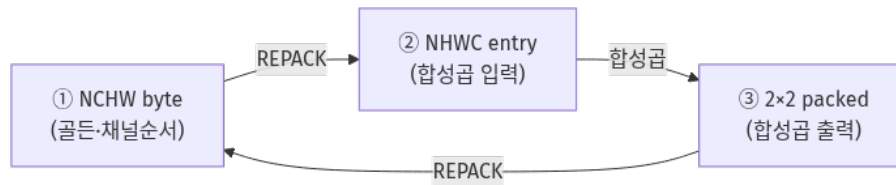
데이터	비트	부호	어디서
입력 (활성)	8	(저장은 unsigned, 곱셈은 signed)	라인 버퍼
가중치	8	signed (INT8)	가중치 버퍼
곱셈 결과	16	signed	mul 출력
가산 트리 합	22	signed	add_tree
누적기	32	signed	mac_kern
bias	16→32	signed (sign-extend)	post_process 입력
출력 픽셀	8	UINT8 또는 INT8	OFM

[핵심] 2 장 2.4 에서 설명한“곱은 작지만 수천 개 더하면 넘친다”가 비트폭 증가로 나타납니다: 곱셈 16 비트 → 가산 22 비트 → 누적 32 비트. 이 숫자들이 14 장의 mul(16) → add_tree_36in(22) → accumulator(32) 에 그대로 있습니다.

[주의] 입력의 부호 함정: 입력은 0~255(UINT8) 로 저장되지만 곱셈할 때는 **signed** 로 다룹니다. 이걸 헛갈려서 Phase 2 에 버그가 있었습니다 (2 장 2.3, 14 장).

12.2 특징맵의 세 가지 옷

같은 특징맵이 처리 단계에 따라 세 가지 다른 포맷으로 변신합니다. 이걸 이해하는 것이 이 장의 핵심입니다.



왜 세 가지나 필요할까요? 각 포맷이 다른 일에 편하기 때문입니다.

(1) NCHW byte order —채널 순서로 죽 늘어놓기

skeleton 이 골든을 저장하는 포맷 (9 장 9.2-(4)) 이자, 풀링 출력의 포맷입니다.

주소 증가 →
[채널 0 의 모든 픽셀 H×W] [채널 1 의 모든 픽셀] ... [채널 C-1 의 모든 픽셀]
└─ 한 채널 통째로 ─┘
1 byte = 1 픽셀

채널 하나를 통째로 쓰고 다음 채널로. 단순하고 사람이 읽기 쉽습니다.

(2) NHWC 16-byte entry —합성곱이 좋아하는 포맷

ifm_line_buf 가 합성곱에 공급하는 포맷입니다. **128 비트 (16 바이트) = 4 개 열 (col) × 4 개 채널 (ch)**.

한 entry (128 비트 = 16 바이트):
byte 위치 = col×4 + ch (col 0~3, ch 0~3)

MSB
[col3ch3][col3ch2]...[col0ch1][col0ch0] LSB
byte15 byte14 ... byte1 byte0

[왜?] 왜 이렇게 섞나?: 합성곱은 한 클럭에 “여러 채널 × 여러 위치”를 동시에 곱해야 합니다 (11 장 11.4 의 144-MAC). 그래서 채널과 위치를 섞어 한 덩어리 (16 바이트) 에 담아두면, 한 번 읽어서 곧장 곱셈기에 넣을 수 있습니다. 채널 4 개·열 4 개를 묶은 이유입니다.

(3) 2×2 packed word —합성곱이 토해내는 포맷

합성곱이 한 클럭에 만드는 2×2 출력 블록 (4 픽셀) 을 담는 32 비트입니다.

32 비트 word = 한 2×2 출력 블록:
[픽셀 (1,1)][픽셀 (1,0)][픽셀 (0,1)][픽셀 (0,0)]
byte3 byte2 byte1 byte0

11 장 11.4 에서 본 “한 클럭 4 픽셀”이 이 32 비트로 나옵니다. 풀링도 이 포맷을 입력으로 받습니다 (16 장).

12.3 REPACK —옷 갈아입히기

문제는, 합성곱 출력 (2x2 packed 또는 풀링 후 채널순서) 을 다음 합성곱 입력 (NHWC entry) 으로 바꿔야 한다는 것입니다. 이 변환이 **REPACK** 입니다.

풀링 출력 (채널순서) —REPACK—> NHWC entry (다음 합성곱 입력)
합성곱 출력 (2x2 packed) —REPACK—> NHWC entry

REPACK 은 새 계산이 아니라 **메모리에서 데이터를 재배치**하는 것입니다. 곱셈이 없습니다. yolo_engine 이 DMA 와 작은 버퍼로 수행합니다 (16 장 16.5).

[비유] **비유**: 같은 옷가지 (데이터) 를 서랍에“색깔별로 정리”(채널순서) 했다가, 입을 때는“상하의 세트로 묶기”(NHWC entry) 로 다시 정리하는 것. 옷 자체는 안 바꿉니다.

REPACK 이 일어나는 곳: 풀링 → 합성곱 전환 (L1→L2 등), 1x1 합성곱 전환 (L11→L12 등). 어떤 레이어에서 일어나는지는 16 장에 정리되어 있습니다.

12.4 가중치는 어떻게 저장되나

11 장 11.4 의 36 개 곱셈기는 한 클럭에 가중치 36 개를 받아야 합니다. 그래서 가중치 버퍼는 **한 번 읽으면 36 개 (288 비트)** 가 나옵니다.

가중치 버퍼 한 줄 읽기 = 288 비트 = 36 개 가중치 (INT8)
→ mul[0]~mul[35] 에 각각 1 개씩

3x3 합성곱의 36 개 배치는“입력채널 0 의 3x3(9 개), 채널 1 의 3x3, 채널 2, 채널 3”순서입니다.

가중치 인덱스 i = 입력채널 $\times$ 9 + (커널행 $\times$ 3 + 커널열)
i= 0~8 : 채널 0 의 3x3
i= 9~17: 채널 1 의 3x3
i=18~26: 채널 2 의 3x3
i=27~35: 채널 3 의 3x3

[주의] **1x1 의 미묘함**: 1x1 은 채널당 가중치가 1 개뿐인데, 가중치 슬롯은 채널을 byte 0/9/18/27 위치에 둡니다 (3x3 과 같은 레이아웃 재사용). 그래서 입력도 같은 위치에 맞춰야 곱셈이 짝이 맞습니다. 이 정렬을 놓쳐서 Phase 2 에 1x1 버그가 있었습니다 (15 장, HISTORY).

12.5 bias 는 sign-extend 로

2 장 2.5 에서 강조한 sign-extend 를 다시 짚습니다. bias 는 16 비트인데 32 비트 누적값에 더해야 하므로, 부호를 보존하며 위를 채웁니다.

```
i_bias = { {16{bias[15]}}, bias }; // bias[15](부호비트) 를 위 16 비트에 복제
```

```
16 비트 음수 bias 0xFFFF(=-1)
sign-extend → 0xFFFFFFFF (= -1) [0]
zero-extend → 0x0000FFFF (= 65535) [X] 완전히 틀림
```

이걸 틀리면 음수 bias 가 거대한 양수가 되어 출력이 망가집니다 (CLAUDE.md 규칙 7).

12.6 descaling 과 clamp (코드 미리보기)

2 장 2.6~2.8 의 descaling·clamp 가 하드웨어 post_process 에서 어떻게 생겼는지 미리 봅시다 (자세히는 14 장).

```
biased    = acc_result + bias;                // (1) bias
activated = (i_relu_en && biased<0) ? 0 : biased; // (2) ReLU
round_bias = activated[31] ? ((1<<shift)-1) : 0; // 음수 toward-zero 보정
scaled    = (activated + round_bias) >>> shift_amount; // (3) descaling (시프트)
clamped   = i_relu_en ? UINT8(0~255) : INT8(-128~127); // (4) clamp
```

2 장에서 개념으로 배운 5 단계 (bias→ReLU→round→shift→clamp) 가 거의 그대로입니다. i_relu_en 이 일반 레이어 (UINT8) 와 검출 헤드 (INT8) 를 가릅니다 (6 장 6.5).

12.7 DRAM 메모리 맵—무엇이 어디에

외부 DRAM 은 세 구역으로 나뉘고, 시작 주소는 CPU 가 제어 레지스터로 알려줍니다 (5 장 5.4, 13 장).

레지스터	구역	내용
ctrl_reg1	가중치 + bias	모든 레이어 가중치 (bias 는 +0xA00000)
ctrl_reg2	입력 이미지	L0 입력
ctrl_reg3	출력	모든 레이어 출력 (레이어마다 다른 위치)

출력 구역 안에서 각 레이어는 고정된 위치 (offset) 를 갖습니다 (일부):

```
L0 출력 : offset 0      크기 262144 word (1 MB, 가장 큼)
L2 출력 : offset 327680 크기 131072
...
L8 출력 : offset 614400 * L19 에서 재사용하므로 보존
L12 출력 : offset 651264 * L16 에서 재사용하므로 보존
L14 출력 : offset 663552 크기 3120 (검출 헤드, 195 채널)
...
```

[핵심] 각 레이어 출력 위치가 미리 정해져 있어 (결정론적), FSM 이 레이어마다 정확한 주소로 읽고 씁니다. L8·L12 는 나중에 재사용되므로 (6 장 6.6, 10 장) 덮어쓰지 않습니다. 전체 표는 기술 레퍼런스 6 장에 있습니다.

12.8 온칩 메모리 맵

11 장 11.6 의 BRAM 버퍼들의 정확한 모양입니다.

버퍼	모양	포맷
가중치	4096 × 72 비트 (읽기 1024 × 288 비트)	36 개 묶음
bias	2560 × 32 비트	{bias16, shift16}
라인 버퍼	2048 × 128 비트 × 4 벌	NHWC entry
출력 (OFM)	65536 × 32 비트 (256 KB)	2×2 packed

출력 버퍼 (OFM dpram) 는 한 레이어 동안 여러 용도로 돌려 씁니다. 예: L11 풀링은 입력을 앞쪽 ([0..8191]), 출력을 뒤쪽 ([8192..]) 에 두어 충돌을 피합니다 (16 장). 이렇게 하나의 버퍼를 알뜰하게 재사용하는 것이 면적 절약입니다.

12.9 이 장의 요약

- 비트폭: 입력/가중치 8 → 곱셈 16 → 가산 22 → 누적 32 → bias(sign-extend) → 출력 8. (2 장 연결)
- 특징맵은 세 옷: (1) NCHW byte(골든·채널순서) (2) NHWC 16-byte entry(합성곱 입력) (3) 2×2 packed(합성곱 출력).
- **REPACK** = 출력 포맷을 다음 합성곱 입력 포맷으로 재배치 (곱셈 없는 메모리 정리).
- 가중치는 36 개 묶음 (288 비트) 으로 읽힘, 1×1 은 채널을 byte 0/9/18/27 에 정렬. bias 는 **sign-extend** 필수.
- DRAM 은 가중치·입력·출력 3 구역, 출력은 레이어별 고정 위치 (L8·L12 보존). 온칩 OFM 버퍼 는 in-place 재사용.

이제 배경과 데이터 포맷이 갖춰졌으니, 다음 장부터 실제 **RTL 코드를 한 줄씩** 읽습니다. 먼저 모든 것을 지휘하는 yolo_engine 입니다.

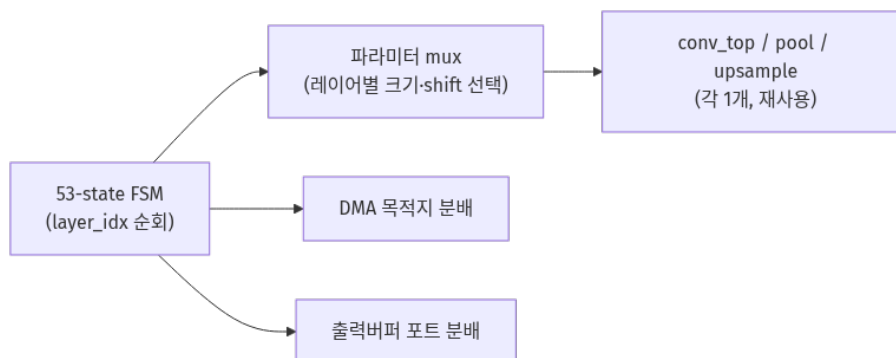
13 장. RTL —yolo_engine (최상위 FSM)

이 장을 읽기 위한 준비: 4 장 FSM, 5 장 AXI, 10 장 네트워크, 12 장 메모리맵.

yolo_engine.v(2365 줄) 는 시스템의 두뇌입니다. 22 개 레이어를 자동으로 순회하면서 적절한 파라미터를 골라 모든 모듈을 지휘합니다. 코드가 길지만 구조는 명확합니다: 레이어 파라미터 표 + 거대한 FSM + 자원 분배 (mux) 세 부분입니다. 이 장은 그 세 부분을 코드로 봅니다.

13.1 핵심 사상—하나의 일꾼, 22 번 재사용

11 장 11.2 에서 본 “요리사 한 명이 22 개 요리”가 yolo_engine 의 본질입니다. 합성곱 유닛·풀링·업샘플은 각 하나뿐이고, FSM 이 레이어를 진행하며 파라미터를 바꿔 끼웁니다.



13.2 포트—무엇이 드나드나

```
module yolo_engine #(...) (  
    input  clk, rstn,  
    // CPU 제어 (AXI4-Lite slave)  
    input  S_AXI_AWADDR, S_AXI_WDATA, ...,  
    // DRAM 읽기 (AXI4 master)  
    output M_ARVALID, M_ARADDR, ..., input M_RDATA, ...,  
    // DRAM 쓰기 (AXI4 master)  
    output M_AWVALID, M_WDATA, ...,
```

```
// 완료
output o_network_done, network_done_led
);
```

5 장 5.7 에서 본 두 인터페이스가 보입니다: S_AXI_*(CPU 가 제어하는 slave), M_*(DRAM 에 접근하는 master). CPU 는 4 개 제어 레지스터로 시작 명령과 주소를 줍니다 (13.7).

13.3 레이어 파라미터 테이블

yolo_engine.v 의 약 122~610 행은 레이어별 상수의 거대한 표입니다. 각 레이어가 필요로 하는 모든 수치가 localparam 으로 정의됩니다.

[코드] 코드 해설—L0 와 L2 의 파라미터

```
// L0 (CONV3x3, 256x256x3 → 256x256x16)
localparam L0_W=256, L0_W_HALF=128, L0_W_BLOCKS=64;
localparam L0_C0=16, L0_ACC_LEN=1, L0_SHIFT=8;

// L2 (CONV3x3, 128x128x16 → 128x128x32)
localparam L2_W=128, L2_C0=32;
localparam L2_ACC_LEN=4, L2_SHIFT=6; // scale 0x40=64=2^6
localparam L2_OFM_BYTE_BASE = 32'h00180000;
```

- L0_W=256: 입력 너비. L0_W_HALF=128: 출력 2x2 블록 개수 (너비 ÷ 2). L0_W_BLOCKS=64: 한 행의 entry 수 (너비 ÷ 4, 12 장 NHWC entry).
- L0_C0=16: 출력 채널 (필터) 수.
- L0_ACC_LEN=1: 누적 횟수. L0 은 입력 채널이 3 개라 4 개 묶음 1 번이면 끝 → 1. L2 는 입력 16 채널 → 16/4 = 4.
- L0_SHIFT=8: descaling 시프트. 9 장 9.4 에서 구한 값 (L0=8, 나머지 =6).
- L2_OFM_BYTE_BASE: L2 출력이 DRAM 어디에 저장될지 (12 장 12.7).

[핵심] 이 상수들이 모든 레이어에 대해 정의되어 있고, 다음에 볼 mux 가“지금 레이어”에 맞는 값을 골라 연산 유닛에 전달합니다.

13.4 “지금 몇 번 레이어?”—phase 레지스터

FSM 은 두 레지스터로 현재 처리 중인 레이어를 추적합니다.

[코드] 코드 해설

```
reg [4:0] conv_phase_r; // 합성곱 레이어 번호
wire is_conv_l0 = (conv_phase_r == 5'd0);
wire is_conv_l12 = (conv_phase_r == 5'd12);
// ...
wire is_conv_lx1 = is_conv_l12 || is_conv_l14 || is_conv_l17 || is_conv_l20;

reg [4:0] pool_phase_r; // 풀링 레이어 번호
wire is_pool_l1 = (pool_phase_r == 5'd1);
```

- conv_phase_r 이 12 면 “지금 L12 처리 중”이라는 뜻.
- is_conv_1x1: L12/14/17/20 을 하나로 묶어 “1x1 모드”임을 표시 → 합성곱 유닛과 라인 버퍼에 “1x1 로 동작해” 신호를 보냄 (14 장, 15 장).

[비유] 이 wire 들이 파라미터 mux 의 선택 신호가 됩니다. 예: cur_shift = is_conv_l0 ? L0_SHIFT : (is_conv_l2 ? L2_SHIFT : ...).

13.5 53-state FSM —전체 지도

FSM 은 53 개 상태를 가지며, 기능별로 7 그룹입니다. 상태가 많지만 그룹으로 보면 단순합니다.

```
localparam S_IDLE=0, S_LOAD_BIAS=1, S_LOAD_BIAS_WAIT=2,
           S_RB_DMA_IFM=3, ..., S_CONV_DONE=12, // 합성곱 그룹
           S_L1_FI_LOAD=13, ..., S_L1_NEXT_FI=19, // 풀링 그룹
           S_RP_LOAD=20, ..., S_RP_NEXT_CIG=25, // REPACK 그룹
           S_L11_LOAD=26, ..., S_L11_STORE_WAIT=32, // L11 풀링
           S_L12_RP_LOAD=33, ..., S_L12_RP_NEXT=38, // 1x1 REPACK
           S_L18_LOAD=39, ..., S_L18_STORE_WAIT=44, // 업샘플
           S_L19_RP_LOAD_A=45, ..., S_L19_RP_NEXT=52; // route concat
```

그룹	담당	패턴
합성곱 (0~12)	모든 conv	bias→IFM→가중치 → 연산 → 저장
풀링/2 (13~19)	L1,3,5,7,9	필터별 적재 → 풀링 → 저장
REPACK (20~25)	conv 전 포맷변환	적재 → 변환 → 저장
L11 풀링/1 (26~32)	L11	stride=1 풀링
1x1 REPACK (33~38)	L12,13,14,17 전	conv 출력 → entry
업샘플 (39~44)	L18	적재 → 복제 → 저장
route concat (45~52)	L20 전	L18+L8 합치기

[주의] 상태 번호가 기능 순서와 안 맞습니다. 예: 종료 상태 S_DONE 이 28 번. 이는 개발이 레이어 단위로 점진적으로 진행되며 (HISTORY) 추가된 순서로 번호가 붙었기 때문입니다. 번호가 아니라 이름과 그룹으로 읽으세요.

13.6 합성곱 레이어 실행 흐름 (가장 중요)

여기가 핵심입니다. yolo_engine 이 합성곱을 어떻게 돌리는지 봅니다.

[코드] 코드 해설—conv_top 을 어떻게 부르나

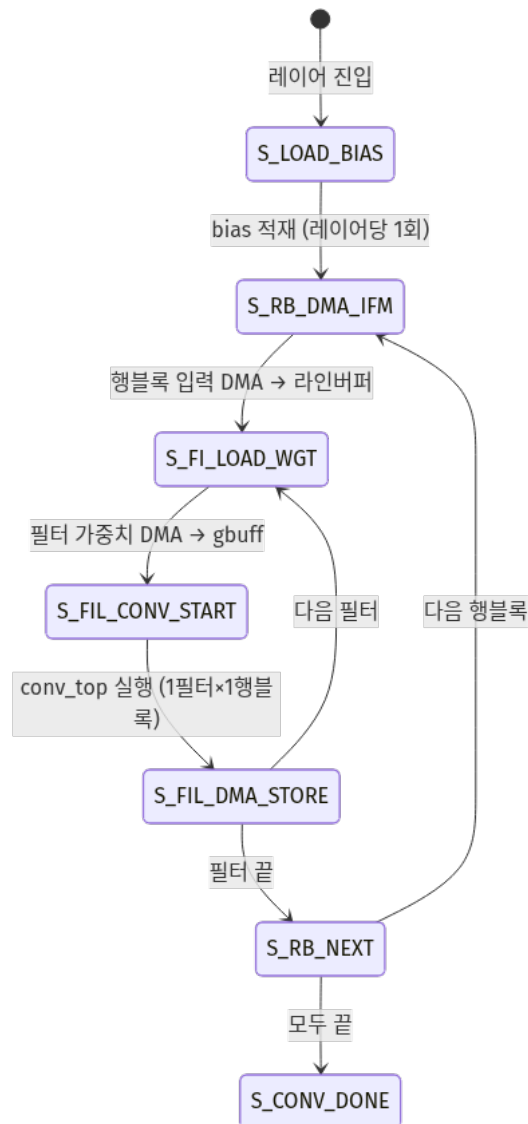
```
conv_top u_conv (
    .i_stream_wgt_mode(1'b1),           // 가중치 streaming 모드
    .i_mode(is_conv_1x1),               // 1x1 이면 1, 3x3 이면 0
    .i_relu_en(!is_conv_l14 && !is_conv_l20), // 검출 헤드만 ReLU 끄
    .i_ofm_h_half(12'd1),               // ← 한 번에 1 행블록만!
    .i_co_total(12'd1),                 // ← 한 번에 1 필터만!
    .i_row_start(rb_r), .i_acc_len(cur_acc_len), .i_shift(cur_shift),
    ...
);
```

여기서 가장 중요한 두 줄: - i_ofm_h_half(12'd1): 한 번 호출에 출력 **1 행블록 (row-block)** 만. - i_co_total(12'd1): 한 번 호출에 **1 필터 (출력채널 1 개)** 만.

[왜?] 왜 한 번에 **1 필터·1 행블록만?**: 가중치 전체가 BRAM(작음) 보다 커서, 필터마다 새 가중치를 **DMA** 로 가져와야 합니다 (15 장 streaming). 입력도 라인 버퍼 용량 때문에 행블록 단위로 가져옵니다. 그래서 yolo_engine 이 바깥에서 “필터 루프”와 “행블록 루프”를 직접 돌리고, conv_top 은 한 조각만 처리합니다.

- i_relu_en(!is_conv_l14 && !is_conv_l20): L14·L20(검출 헤드) 만 ReLU 를 끄니다 (6 장 6.5, 2 장 2.8).

합성곱 FSM 흐름



말로 풀면:

```

for 행블록 rb:
    입력 DMA → 라인버퍼
    for 필터 fi:
        가중치 DMA → gbuff_param
        conv_top 실행 → 출력버퍼
        출력버퍼 → DRAM 저장
  
```

13.7 제어 레지스터—CPU 와의 약속

5 장 5.4 에서 본 AXI-Lite 제어 레지스터입니다.

레지스터	의미
ctrl_reg0[0]	ap_start (1 쓰면 시작)
ctrl_reg0[1]	network_done (읽으면 완료 여부)

레지스터	의미
ctrl_reg1	가중치 DRAM 주소
ctrl_reg2	입력 DRAM 주소
ctrl_reg3	출력 DRAM 주소

CPU 는 주소 3 개를 쓰고 ap_start 를 1 로 만든 뒤, ctrl_reg0 을 읽으며 완료 (bit 1) 를 기다립니다 (22 장 host.py).

13.8 DMA 목적지 분배

DMA 읽기 엔진은 하나인데, 읽은 데이터를 여러 곳 (가중치·bias·입력·각종 버퍼) 에 나눠 보내야 합니다. dma_target_r 로 분배합니다.

[코드] 코드 해설

```
localparam DMA_TGT_WGT=1,      // → gbuff_param 가중치
            DMA_TGT_BIAS=2,     // → gbuff_param bias
            DMA_TGT_IFM=3,      // → ifm_line_buf
            DMA_TGT_L1_IFM=4,   // → 출력버퍼 (폴링/REPACK 입력)
            DMA_TGT_L11_IFM=5,  // → 출력버퍼 (L11 입력)
            DMA_TGT_L12_RP_IN=6, // → 출력버퍼 (1x1 REPACK 입력)
            DMA_TGT_L18_IFM=7;  // → 출력버퍼 (업샘플 입력)
```

dma_target_r 이 현재 무엇을 읽는 중인지 기억하고, 도착한 데이터를 맞는 버퍼에 씁니다. 예를 들어 DMA_TGT_WGT 면 가중치를 4 개 모아 128 비트로 만들어 가중치 버퍼에 기록합니다.

13.9 출력 버퍼 5 방향 분배

출력 버퍼 (OFM dpram, 256 KB) 는 여러 주체가 공유합니다 (12 장 12.8). 두 포트 (쓰기 A, 읽기 B) 에 5 가지 경로를 mux 합니다.

```
쓰기 포트 (A): (1) 합성곱 출력 (2) DMA 적재 (폴링 입력)
               (3) 폴링 출력 (4) stride1 폴링 출력 (5) 업샘플 출력
읽기 포트 (B): (1) DMA 저장 (DRAM 으로) (2) 폴링 읽기
               (3) stride1 폴링 읽기 (4) 업샘플 읽기
```

하나의 버퍼를 이렇게 돌려 써서, 합성곱 출력 임시저장·폴링/업샘플 작업·DRAM 저장을 모두 256 KB 로 처리합니다. 현재 FSM 그룹에 따라 mux 가 자동으로 경로를 고릅니다.

13.10 Route 와 레이어 스킵

L15(yolo)·L16(route)·L19(route)·L21(yolo) 은 연산 모듈이 없습니다 (10 장 10.5). FSM 이 이렇게 처리합니다.

- **L15, L21 (yolo):** 아무것도 안 함. 검출 헤드 출력이 DRAM 에 있고, 소프트웨어가 후처리 (6 장 6.8).
 - **L16 (route -4):** L12 출력을 가리키는 별칭. FSM 이 L14 다음에 **L15·L16** 을 건너뛰고 **L17** 로 직행 (cp==14 done → cp=17). L17 의 REPACK 이 L12 출력을 입력으로 읽습니다.
 - **L19 (route concat):** L18 과 L8 을 합치는 단계. FSM 이 두 출력을 출력버퍼에 적재한 뒤 entry 로 묶어 L20 입력을 만듭니다 (16 장 16.5).
-

13.11 이 장의 요약

- yolo_engine 은 “일꾼 하나를 22 번 재사용”하는 지휘자: 레이어 파라미터 표 + 53-state FSM + 자원 mux.
- 파라미터 표 (122~610 행) 를 conv_phase_r/pool_phase_r 로 골라 연산 유닛에 전달.
- 합성곱은 yolo_engine 이 필터·행블록 루프를 직접 돌리고 conv_top 은 “1 필터 × 1 행블록”만 처리 (가중치/입력 streaming 때문).
- DMA 읽기는 8 종 목적지로 분배, 출력버퍼는 5 방향 포트 mux 로 공유.
- route/yolo 는 모듈 없이 FSM 스킵 + 주소 별칭으로 처리.

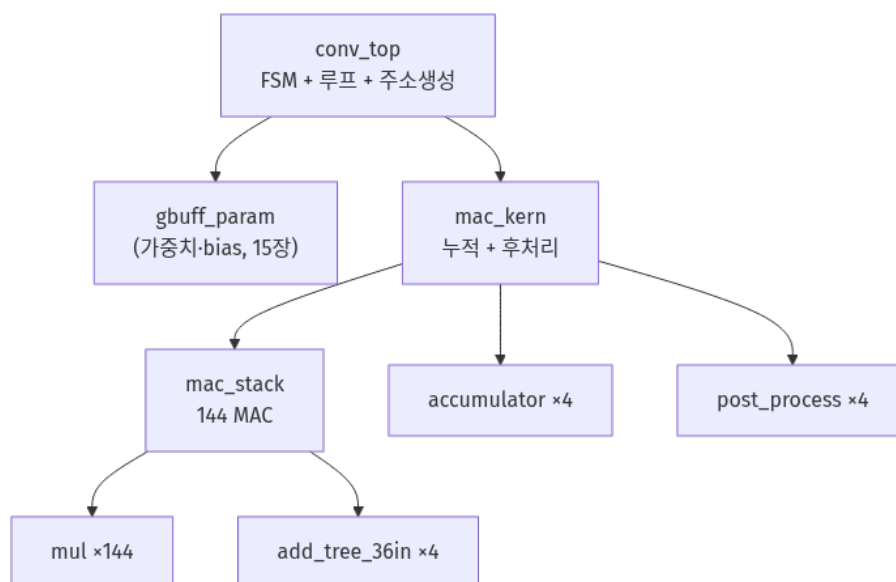
다음 장에서는 이 FSM 이 호출하는 합성곱 엔진을 가장 아래 mul 부터 한 줄씩 봅니다.

14 장. RTL —Convolution 엔진 (코드 해설)

이 장을 읽기 위한 준비: 1 장 1.10 MAC, 2 장 양자화, 4 장 파이프라인·타이밍, 12 장 데이터포맷.

이 장은 실제 곱셈-누적이 일어나는 연산 코어를 가장 아래 (**mul**) 부터 위 (**conv_top**) 까지 코드로 봅니다. 이 5 개 모듈이 전체 연산의 88% 를 담당하므로 (10 장 10.7), 가장 중요하고 가장 길게 다룹니다. 코드를 한 줄씩 짚으니 천천히 따라오세요.

14.1 5 단 계층—곱셈기부터 제어까지



모듈	하는 일	latency
mul	INT8 곱셈 1 개	4
add_tree_36in	36 개를 더해 1 개	4
mac_stack	144 곱셈 + 4 덧셈트리	8 (4+4)

모듈	하는 일	latency
mac_kern	+ 누적 + 후처리	8 + 누적 + 1
post_process	bias·ReLU·시프트·클립	1
conv_top	루프 제어	(FSM)

아래부터 하나씩 코드로 봅니다.

14.2 mul.v —INT8 곱셈기 한 개

mul.v 는 가중치 하나와 입력 하나를 곱합니다. 가장 작은 부품이며 144 개가 병렬로 쓰입니다.

[코드] 코드 해설—시뮬레이션 경로

```
module mul(input clk, input [7:0] w, input [7:0] x, output [15:0] y);
`ifdef FPGA
    // 합성: DSP48 사용 (아래)
`else
    reg [15:0] dsp_P[0:3]; // 4 단 파이프라인 레지스터
    always@(posedge clk) begin
        dsp_P[0] <= $signed(w) * $signed(x); // (1) 곱셈 (부호 있는!)
        dsp_P[1] <= dsp_P[0]; // (2) 지연
        dsp_P[2] <= dsp_P[1]; // (3) 지연
        dsp_P[3] <= dsp_P[2]; // (4) 지연
    end
    assign y = dsp_P[3]; // 4 클럭 후 출력
`endif
endmodule
```

- `$signed(w) * $signed(x)`: 가중치와 입력을 **부호 있는 수**로 곱합니다 (4 장 4.6). `$signed` 를 빼면 음수 가중치가 큰 양수가 되어 결과가 틀립니다—Phase 2 의 실제 버그였습니다 (2 장 2.3).
- `dsp_P[0~3]`: 곱셈 결과를 4 단 레지스터로 흘려보냅니다. 왜 4 단일까요? 실제 FPGA 의 DSP48 곱셈기가 **latency 4** 라서, 시뮬레이션도 똑같이 4 단으로 맞춥니다 (4 장 4.3 파이프라인). 그래야 FPGA 와 시뮬 타이밍이 일치합니다.

[코드] 코드 해설—합성 경로 (DSP48)

```
`ifdef FPGA
    assign dsp_A = w[7] ? {10'b11_1111_1111, w} : {10'b0, w}; // INT8 → INT18 부호 확장
    assign dsp_B = x[7] ? {10'b11_1111_1111, x} : {10'b0, x};
    xbip_dsp48_macro_0 u_dsp(.CLK(clk), .A(dsp_A), .B(dsp_B), .C(48'b0), .P(dsp_P));
    assign y = dsp_P[15:0];
`endif
```

- `w[7] ? {10'b1...1, w} : {10'b0, w}`: 8 비트를 18 비트로 **부호 확장** (2 장 2.5). 최상위 비트 (`w[7]`) 가 1(음수) 이면 위를 1 로, 0(양수) 이면 0 으로 채웁니다.
- `xbip_dsp48_macro_0`: Xilinx DSP48 곱셈기 IP(3 장 3.3). `.C(48'b0)` 은“누산은 안 함”(곱셈만). 누산은 다음의 가산 트리가 합니다.

곱의 최댓값은 $(-128) \times (-128) = 16384$ 라 16 비트면 충분합니다 (2 장 2.4).

14.3 add_tree_36in.v —36 개를 하나로

add_tree_36in.v 는 곱셈 결과 36 개를 더해 1 개로 만듭니다. 2 장 2.4 에서 본 “비트폭이 단계마다 커지는” 가산 트리입니다.

[코드] 코드 해설—4 단 트리

```
// Level 1: 36 개 → 12 개 (3 개씩 더함, 18 비트)
l1_00 <= $signed(m[0]) + $signed(m[1]) + $signed(m[2]);
// ... l1_11 까지 12 개

// Level 2: 12 개 → 4 개 (3 개씩, 20 비트)
l2_00 <= $signed(l1_00) + $signed(l1_01) + $signed(l1_02);
// ... l2_03 까지

// Level 3: 4 개 → 2 개 (2 개씩, 21 비트)
l3_00 <= $signed(l2_00) + $signed(l2_01);
l3_01 <= $signed(l2_02) + $signed(l2_03);

// Level 4: 2 개 → 1 개 (22 비트)
l4 <= $signed(l3_00) + $signed(l3_01);
```

```
36 개 (16 비트)
→ Level1: 12 개 (18 비트)   3 개씩 묶어 더함
→ Level2:  4 개 (20 비트)
→ Level3:  2 개 (21 비트)
→ Level4:  1 개 (22 비트)   ← 최종 합
```

- 각 레벨이 한 클럭씩 걸려 **총 4 클럭 (latency 4)**. 각 레벨 출력이 레지스터 (l1_xx 등) 라 한 클럭 소요.
- 비트폭이 16→18→20→21→22 로 커집니다. 더할수록 값이 커지니 넘치지 않게 늘립니다 (2 장 2.4). 최댓값 $36 \times (127 \times 255) \approx 117$ 만이라 22 비트면 안전.

[코드] 코드 해설—valid 신호도 같이 지연

```
always @(posedge clk) begin
    vld_d1 <= vld_i; vld_d2 <= vld_d1; vld_d3 <= vld_d2; vld_d4 <= vld_d3;
end
assign vld_o = vld_d4;    // 데이터와 똑같이 4 클럭 지연
```

4 장 4.5 에서 본 valid 파이프라인입니다. 데이터가 4 단을 지나는 동안 valid 도 4 단 지연시켜, 출력이 유효한 정확한 클럭을 알려줍니다.

14.4 mac_stack.v —144 개 MAC 한꺼번에

mac_stack.v 는 11 장 11.4 의 144-MAC 을 실제로 인스턴스화합니다.

[코드] 코드 해설—144 개 곱셈기 생성

```
genvar i;
generate
  for (i=0; i<36; i=i+1) begin: g_mul_arr
    mul u_mul_00 (.clk(clk), .w(wgt[i*8 +: 8]), .x(ifm_00[i*8 +: 8]), .y(muls_00[i*16 +: 16])
    mul u_mul_01 (.clk(clk), .w(wgt[i*8 +: 8]), .x(ifm_01[i*8 +: 8]), .y(muls_01[i*16 +: 16])
    mul u_mul_10 (.clk(clk), .w(wgt[i*8 +: 8]), .x(ifm_10[i*8 +: 8]), .y(muls_10[i*16 +: 16])
    mul u_mul_11 (.clk(clk), .w(wgt[i*8 +: 8]), .x(ifm_11[i*8 +: 8]), .y(muls_11[i*16 +: 16])
  end
endgenerate
```

- for (i=0; i<36; ...) × 4 개 인스턴스 = **36 × 4 = 144 개 곱셈기** (11 장 11.4).
- wgt[i*8 +: 8]: 288 비트 가중치에서 i 번째 8 비트 (가중치 1 개) 를 꺼냄 (12 장 12.4).
- 핵심: 같은 wgt 가 4 개 인스턴스 (u_mul_00/01/10/11) 모두에 들어갑니다 → **가중치 broadcast**. 인접한 2×2 출력 위치 4 개는 같은 필터를 쓰기 때문입니다.
- ifm_00/01/10/11: 4 개 출력 위치 (2×2 블록의 네 칸) 에 해당하는 입력.

```

      wgt (36 개, 공유)
      |   |   |   |
ifm_00 ifm_01 ifm_10 ifm_11 (4 개 위치의 입력)
      |   |   |   |
      [36 곱셈]×4 → [덧셈트리]×4 → acc_00/01/10/11 (4 개 부분합)

```

[코드] 코드 해설—4 개 가산 트리

```
add_tree_36in u_at_00 (.clk(clk), .rstn(rstn), .vld_i(vld_m4),
                      .muls(muls_00), .acc_o(acc_00), .vld_o(vld_at_00));
// u_at_01, u_at_10, u_at_11 동일
```

4 개 출력 위치 각각에 가산 트리가 하나씩. vld_m4 는 곱셈기 4 단 지연 후의 valid(4 장 4.5). 곱셈 (4) + 가산 (4) = **총 latency 8**.

14.5 mac_kern.v —누적과 후처리를 붙이다

mac_kern.v 는 mac_stack 의 부분합을 여러 클럭 누적하고 post_process 로 최종 픽셀을 만듭니다.

[코드] 코드 해설—누적기

```
always @(posedge clk or negedge rstn) begin
  acc_done <= 1'b0;
  if (mac_vld) begin
    if (cyc_cnt == 8'd0) begin
      psum_00 <= $signed(mac_00); // 첫 클럭: 초기화
      // psum_01/10/11 동일
    end else begin
      psum_00 <= psum_00 + $signed(mac_00); // 이후: 누적
    end
    if (cyc_cnt == i_len - 8'd1) begin
      acc_done <= 1'b1; // 마지막: 완료 펄스
      cyc_cnt <= 8'd0;
    end else cyc_cnt <= cyc_cnt + 8'd1;
  end
end
```

```
end
end
```

- `cyc_cnt`: 지금 몇 번째 누적인지 세는 카운터.
- `cyc_cnt == 0` 이면 첫 값으로 초기화, 아니면 더하기. 입력 채널이 4 개 초과면 4 개씩 나눠 여러 클럭 누적합니다 (11 장 11.4).
- `i_len`: 누적 횟수. 예를 들어 입력 256 채널이면 $256/4 = 64$ 번. 이 값이 13 장의 `cur_acc_len` 입니다.
- `cyc_cnt == i_len-1` 이면 누적 끝 → `acc_done` 펄스 → `post_process` 가 받습니다.
[비유] 22 비트 부분합을 32 비트 `psum` 에 누적합니다. 64 번 더해도 넘치지 않게 32 비트를 씁니다 (2 장 2.4).

[코드] 코드 해설—4 개의 `post_process`

```
post_process u_pp_00 (.clk(clk), .rstn(rstn),
    .acc_done(acc_done), .acc_result(psum_00),
    .bias(i_bias), .shift_amount(i_shift), .i_relu_en(i_relu_en),
    .pixel_out(px_00), .output_valid(vld_00));
// u_pp_01/10/11 동일

assign o_pixel = {px_11, px_10, px_01, px_00}; // 4 픽셀을 32 비트로 묶음
```

- 4 개 출력 위치 각각에 후처리하기. **bias·shift** 는 4 개가 공유합니다 (같은 필터 = 같은 출력 채널이라 동일).
- `o_pixel = {px_11, px_10, px_01, px_00}`: 4 픽셀을 12 장 12.2-(3) 의 “2×2 packed” 32 비트로 묶어 출력.

14.6 `post_process.v` —bias·ReLU·시프트·클립

`post_process.v` 는 2 장 2.5~2.8 에서 개념으로 배운 후처리를 그대로 구현합니다. 누적 결과를 최종 8 비트 픽셀로 만듭니다.

[코드] 코드 해설—4 단계 (조합 논리)

```
// (1) bias 덧셈
wire signed [31:0] biased = acc_result + bias;

// (2) ReLU (i_relu_en=1 이고 음수면 0)
wire signed [31:0] activated = (i_relu_en && biased[31]) ? 32'd0 : biased;

// (3) descaling —음수 toward-zero 보정 후 시프트
wire signed [31:0] round_bias = activated[31] ?
    (($signed(32'sd1) <<< shift_amount) - 32'sd1) : 32'sd0;
wire signed [31:0] scaled = (activated + round_bias) >>> shift_amount;

// (4) clamp
wire [7:0] clamped = i_relu_en ?
    ( (scaled[31]) ? 8'd0 : (|scaled[31:8]) ? 8'hFF : scaled[7:0] ) : // UINT8 0~255
    ( (scaled > 127) ? 8'h7F : (scaled < -128) ? 8'h80 : scaled[7:0] ); // INT8 -128~127
```

한 줄씩: - **(1) bias**: 누적값에 bias 를 더함 (2 장 2.5). bias 는 sign-extend 된 32 비트 (12

장 12.5). - **(2) ReLU**: `i_relu_en` 이 1 이고 `biased[31]`(부호비트 = 음수) 이면 0. 검출 헤드는 `i_relu_en=0` 이라 통과 (1 장 1.6, 6 장 6.5). - **(3) descaling**: `round_bias` 가 핵심입니다. `activated[31]`(음수) 이면 $(1 \ll \text{shift}) - 1$ 을 더한 뒤 시프트 → C 의 toward-zero 와 일치 (2 장 2.7). 양수면 `round_bias=0` 이라 영향 없음. - **(4) clamp**: `i_relu_en` 이면 0~255(UINT8), 아니면 -128~127(INT8). 검출 헤드는 음수 유지 (2 장 2.8).

[코드] 코드 해설—결과 래치

```
always @(posedge clk or negedge rstn) begin
    output_valid <= acc_done;          // acc_done 다음 클럭에 유효
    if (acc_done) pixel_out <= clamped;
    else          pixel_out <= 8'd0;
end
```

`acc_done` 이 오면 다음 클럭에 결과를 레지스터에 저장 (latency 1).

[핵심] **shift 값의 중요성**: `shift_amount(=i_shift)` 는 레이어마다 다릅니다 (2 장 2.6): L0=8, 대부분 =6, **L14=6, L20=9**. 검출 헤드인데도 L14(6) 와 L20(9) 가 다른 이유는 다음 합성곱 유무 때문입니다 (2 장 2.6, 9 장 9.4). 이 toward-zero 보정과 정확한 shift 가 검출 헤드 정확도를 결정합니다 (20 장의 L14 디버깅 사례).

14.7 conv_top.v —합성곱 한 호출 제어

`conv_top.v` 는 위 모듈들을 인스턴스화하고, 출력 고정 (**output-stationary**) 4 중 루프로 합성곱 한 조각을 수행합니다.

루프 구조

```
for 필터:
  for 행:
    for 열:          ← 한 좌표 = 2x2 출력 블록
      for 누적:      ← 입력채널 4 개씩 (acc_len 회)
        mac_kern 에 입력 + 가중치 흘려보내기
```

13 장 13.6 에서 봤듯, `yolo_engine` 은 `i_co_total=1`, `i_ofm_h_half=1` 로 호출해 “1 필터 × 1 행블록”만 시킵니다.

[코드] 코드 해설—FSM

```
localparam ST_IDLE=0, ST_LOAD=1, ST_RUN=2, ST_DRAIN=3, ST_NEXT=4, ST_DONE=5;
always @(*) case (cstate)
    ST_IDLE : if (i_start)          nstate = ST_LOAD;
    ST_LOAD :                       nstate = ST_RUN;
    ST_RUN  : if (spatial_last)      nstate = ST_DRAIN;
    ST_DRAIN: if (out_cnt == q_total) nstate = ST_NEXT; // 파이프라인 비우기
    ST_NEXT : if (!i_conv_pause)    nstate = fil_last ? ST_DONE : ST_LOAD;
endcase
```

- ST_LOAD: 가중치·bias 의 첫 주소를 BRAM 에 발사 (4 장 4.4 의 latency 함수).
- ST_RUN: 매 클럭 `mac_kern` 에 입력·가중치를 streaming.
- ST_DRAIN: 파이프라인에 남은 픽셀을 다 받을 때까지 기다림 (4 장 4.3 latency). 곱셈·가산·누

적의 지연 때문에 마지막 입력 후에도 결과가 몇 클럭 더 나옵니다.

[코드] 코드 해설—타이밍 정렬 (자주 틀리는 곳)

```
// BRAM read latency 1 을 흡수하려고 mac_kern 의 valid 를 1 클럭 늦춤
reg mac_vld_d;
always @(posedge clk) mac_vld_d <= (cstate == ST_RUN);
```

가중치 BRAM 은 주소를 준 다음 클럭에 데이터가 나옵니다 (4 장 4.4). 그래서 mac_kern 을 시작하는 valid(mac_vld_d) 를 1 클럭 늦춰, 데이터 도착과 맞춥니다.

```
// 라인 버퍼 (latency 2) 를 위해 다음 좌표를 미리 발사 (look-ahead)
wire [11:0] lah_col = lah_acc_last ? (lah_col_last ? 0 : col_idx + 1) : col_idx;
assign o_ifm_col = (cstate == ST_RUN) ? lah_col : col_idx;
```

라인 버퍼는 latency 2 라 (15 장) 다음에 필요할 좌표를 미리 보냅니다 (4 장 4.4 look-ahead).

[주의] **Phase 2 버그**: 가중치 주소를 1 클럭 잘못 advance 해서 “가중치가 입력보다 1 스텝 앞서가는” 버그가 있었습니다. ST_LOAD 에서 wgt_addr 를 +1 하지 않는 것이 핵심입니다 (HISTORY, 메모리 conv_top_pipeline). 타이밍은 한 클럭만 틀려도 결과가 망가집니다 (4 장 4.8).

[코드] 코드 해설—가중치 streaming 모드

```
if (!i_stream_wgt_mode)
    wgt_base_r <= wgt_base_r + {2'b00, i_acc_len}; // 일반: 다음 필터 가중치로 이동
// streaming 모드 (=1) 면 wgt_base_r 를 그대로 둬 → 모든 필터가 BRAM[0] 부터 읽음
```

i_stream_wgt_mode=1(yolo_engine 이 항상 1 로 호출) 이면, 필터마다 새 가중치를 BRAM 처음에 DMA 로 채우므로 base 주소를 0 으로 고정합니다 (15 장, 13 장 13.6).

14.8 전체 latency 정리

한 출력 픽셀이 나오기까지:

```
입력 → [곱셈 4] → [가산트리 4] → [누적 i_len] → [후처리 1] → 출력
           |               |
           |               | mac_stack 8
총 latency = 8 + i_len + 1
```

[핵심] **중요**: 이 latency 는 파이프라인 채움/비움에만 보입니다 (4 장 4.3). 정상 운전 중에는 i_len(=acc_len) 클럭마다 4 픽셀이 쏟아집니다. 이것이 진짜 처리량입니다 (18 장).

14.9 3×3 과 1×1 이 같은 엔진을 쓰는 비결

1 장 1.5 에서 “1×1 과 3×3 은 한 출력점에 입력을 몇 개 모으냐만 다르다”고 했습니다. 코드로 보면 mac_stack·mac_kern·mul 은 mode 를 전혀 신경 쓰지 않습니다.

항목	3×3	1×1
i_mode	0	1
36 개 입력의 의미	4 채널 × 3×3	4 채널 × 1 (나머지는 다른 채널)
다른 점	입력 패킹 (라인 버퍼) 과 acc_len 뿐	

차이는 전적으로 15 장의 ifm_line_buf 가 입력을 어떻게 묶느냐에 있습니다. 곱셈기는 받은 36 개를 그냥 곱할 뿐입니다. 이것이 “1×1 을 위해 새 하드웨어를 안 만든다”는 7 장 7.7 의 설계 결정입니다.

14.10 이 장의 요약

- 연산 코어는 conv_top → mac_kern → mac_stack → (mul ×144 + add_tree ×4) + post_process ×4.
- mul: \$signed INT8 곱셈, latency 4(DSP48 과 일치). add_tree: 4 단 트리, 16→22 비트, latency 4.
- mac_stack: 가중치 broadcast 로 144 곱셈 → 4 개 부분합 (latency 8).
- mac_kern: 부분합을 acc_len 번 누적 (32 비트), 4 개 post_process 로 4 픽셀 생성.
- post_process: bias → ReLU → (음수 toward-zero 보정) descaling → clamp. shift 는 L0=8/대부분 =6/**L14=6, L20=9**.
- conv_top: 4 중 루프 FSM + 타이밍 정렬 (가중치 valid 지연, 라인버퍼 look-ahead, streaming).
- 전체 latency 8+acc_len+1 이지만 처리량은 4 픽셀/acc_len 클럭. 3×3·1×1 은 입력 패킹만 다름.

다음 장에서는 이 엔진에 데이터를 공급하는 **메모리 버퍼** (라인 버퍼·가중치 버퍼) 를 봅니다.

15 장. RTL —메모리 버퍼 (코드 해설)

이 장을 읽기 위한 준비: 3 장 3.4 BRAM, 4 장 4.4 BRAM latency, 12 장 데이터 포맷.

14 장의 144-MAC 이 매 클럭 굵지 않으려면, 버퍼가 정확한 포맷·타이밍으로 데이터를 공급해야 합니다. 이 장은 네 버퍼를 다룹니다: 입력 윈도우를 만드는 `ifm_line_buf`(가장 복잡), 가중치를 담은 `gbuff_param`, 그리고 그 기반인 `dpram_wrapper/spram_wrapper` 입니다.

15.1 ifm_line_buf.v —입력 윈도우 공장

`ifm_line_buf.v` 는 이 프로젝트에서 가장 까다로운 데이터 정렬을 합니다. 역할: DRAM 에서 받은 NHWC entry(12 장 12.2-(2)) 를 4 행으로 쌓고, 합성곱이 요청하는 3×3 윈도우를 36 바이트씩 4 벌 뽑아주는 것.

[비유] 비유: 합성곱은 이미지 위를 슬라이딩하며 3×3 창문으로 봅니다 (1 장 1.3). 라인 버퍼는“지금 필요한 4 줄”을 들고 있다가, 요청이 오면 그 위치의 창문 (윈도우) 을 잘라 건네주는 역할입니다.

15.1.1 왜 4 줄만 들고 있나

3×3 합성곱은 인접 3 줄을 동시에 봅니다. 2×2 출력 블록 (아래 줄도 필요) 을 위해 1 줄 더, 총 4 줄이면 충분합니다. 줄이 진행되면 가장 오래된 줄을 새 줄로 덮어씹습니다 (**cyclic**, 순환).

4 개 bank(줄 저장소): bank0, bank1, bank2, bank3
줄이 진행 → 가장 오래된 bank 를 새 줄로 덮어씹 (원형으로 순환)
어느 bank 에 어느 줄? → " 입력행 mod 4"

[코드] 코드 해설—cyclic 매핑

```
wire [1:0] base_line = i_rb[0] ? 2'd1 : 2'd3; // (2*Rb-1) mod 4
wire [1:0] line_sel_0 = base_line;
wire [1:0] line_sel_1 = base_line + 2'd1; // mod 4 (2 비트라 자동 wrap)
```

출력 행블록 `i_rb` 가 1 진행하면 입력 줄은 2 진행하므로, `base` 가 토글됩니다. 이 매핑으로“새 줄을 어느 bank 에 쓸지”와“윈도우를 읽을 때 어느 bank 를 볼지”가 일치합니다.

[코드] 코드 해설—3×3 윈도우 주소

```

wire signed [13:0] col_start_s = $signed({1'b0, i_cb, 1'b0}) - 14'sd1; // 2*Cb - 1
wire signed [13:0] base_entry_s = col_start_s >>> 2; // entry 인덱스
wire [1:0] offset_3x3 = col_start_s[1:0]; // entry 내 위치
wire [10:0] rd_addr_a_3x3 = row_off[10:0] + base_entry_u; // entry A
wire [10:0] rd_addr_b_3x3 = row_off[10:0] + base_entry_u + 1; // entry B

```

- col_start = 2*Cb - 1: 2×2 출력 블록이 보는 입력의 왼쪽 열 (패딩 -1 포함, 1 장 1.4).
- entry 하나가 4 열을 담으므로 (12 장 12.2-(2)), 원하는 열이 entry 경계를 걸치면 두 entry(A, B)를 읽어 offset으로 잘라 붙입니다.

[코드] 코드 해설—경계 패딩

```

if (!i_mode) begin // 3×3 모드만 패딩
    if (row_inv_r[rr]) begin window[rr][0..3] = 0; end // 행이 경계 밖이면 0
    else begin
        if (col_inv_r[0]) window[rr][0] = 0; // 열이 경계 밖이면 0
        ...
    end
end
end

```

pad=1 합성곱의 가장자리 처리 (1 장 1.4). 윈도우가 이미지 밖으로 나가면 그 칸을 0으로 채웁니다.

[코드] 코드 해설—36 바이트로 패킹 (3×3)

```

for (cl_loc=0; cl_loc<4; cl_loc++) // 입력 채널 4개
    for (kh=0; kh<3; kh++) // 커널 행
        for (kw=0; kw<3; kw++) begin // 커널 열
            idx = cl_loc*9 + kh*3 + kw; // 0~35
            ifm00_w[idx*8 +: 8] = win_byte(kh, kw, cl_loc); // 출력 (0,0) 용
            ifm01_w[idx*8 +: 8] = win_byte(kh, kw+1, cl_loc); // 출력 (0,1) 용
            ifm10_w[idx*8 +: 8] = win_byte(kh+1, kw, cl_loc); // 출력 (1,0) 용
            ifm11_w[idx*8 +: 8] = win_byte(kh+1, kw+1, cl_loc); // 출력 (1,1) 용
        end
end

```

- idx = cl_loc*9 + kh*3 + kw: 12 장 12.4에서 본 가중치 배치 (채널 ×9 + 커널위치)와 똑같은 순서로 입력을 배치합니다. 그래야 mac_stack의 곱셈기 i번이 “가중치 i × 입력 i”로 짝이 맞습니다 (14 장 14.4).
- ifm00/01/10/11: 2×2 출력의 네 위치는 윈도우가 1칸씩 어긋납니다 (kw vs kw+1, kh vs kh+1).

[코드] 코드 해설—1×1 패킹 (채널 정렬)

```

// 1×1 모드: 채널 0~3을 byte 0, 9, 18, 27 위치에 배치
ifm00_w[7:0] = window[1][col_inblk_r][7:0]; // ch0 @ byte0
ifm00_w[79:72] = window[1][col_inblk_r][15:8]; // ch1 @ byte9
ifm00_w[151:144] = window[1][col_inblk_r][23:16]; // ch2 @ byte18
ifm00_w[223:216] = window[1][col_inblk_r][31:24]; // ch3 @ byte27

```

- 1×1은 커널이 점 하나라 채널당 입력 1개. 가중치 슬롯이 채널을 byte 0/9/18/27에 두므로 (12 장 12.4), 입력도 같은 위치에 맞춥니다.
- [주의] 이 정렬을 빠뜨려 ch1~3 곱셈이 무시되던 것이 Phase 2의 1×1 버그였습니다 (수정 후)

L12 mismatch 10997→0, HISTORY).

15.1.2 타이밍—latency 2

i_rd_en → (BRAM 읽기 1 클럭) → 윈도우 추출 (조합) → 출력 레지스터 (1 클럭) → 총 2 클럭

4 장 4.4 에서 본 BRAM latency 때문에, conv_top 이 **look-ahead** 로 다음 좌표를 미리 보냅니다 (14 장 14.7). 라인 버퍼 내부에서도 offset 경계 판정을 1 클럭 지연 (offset_r 등) 시켜 BRAM 출력과 맞춥니다.

15.2 gbuff_param.v —가중치·bias 창고

gbuff_param.v 는 현재 레이어의 가중치와 bias 를 담습니다.

[코드] 코드 해설—비대칭 가중치 메모리

```
// 읽기: 288 비트 entry = 4 개의 72 비트를 합침
rd_wgt_data_r <= { wgt_mem[{rd_wgt_addr, 2'd3}], wgt_mem[{rd_wgt_addr, 2'd2}],
                  wgt_mem[{rd_wgt_addr, 2'd1}], wgt_mem[{rd_wgt_addr, 2'd0}] };
```

- 쓰기는 72 비트씩, 읽기는 **288 비트 (=4×72)** 입니다 (12 장 12.4). 한 번 읽으면 36 개 가중치가 나와 mac_stack 의 36 곱셈기에 바로 들어갑니다.
- DMA 로 들어오는 가중치는 좁게 (72 비트) 쓰고, 곱셈기에는 넓게 (288 비트) 한 번에 공급—이 비대칭이 효율의 핵심입니다.

[코드] 코드 해설—bias 메모리

```
reg [31:0] bias_mem [0:2559]; // 32 비트 × 2560 칸
rd_bias_data_r <= bias_mem[rd_bias_addr];
```

- entry 당 32 비트에 {bias16, shift16} 를 담습니다 (12 장 12.5). 전체 레이어 필터 수 합 (~2294) 이 2560 안에 들어가 한 번에 적재합니다.
- 읽기 latency 1 클럭. conv_top 의 ST_LOAD 가 이를 흡수합니다 (14 장 14.7).

15.2.1 가중치 streaming 의 그림

13 장 13.6·14 장 14.7 에서 본 streaming 을 메모리 관점으로 보면:

전체 가중치 (수 MB) → BRAM(36KB) 에 다 못 담음
해결: 필터마다 그 필터 가중치만 DMA 로 BRAM[0] 부터 채움 → 합성곱 → 다음 필터로

[왜?] 왜 **streaming**?: L6 이후 레이어는 가중치가 BRAM(4096 칸) 을 초과합니다. 그래서 전체를 올릴 수 없고, **필터 하나 분량만** 그때그때 가져옵니다. 대신 DRAM 접근이 늘지만, 가중치는 한 번만 읽으면 되므로 감당할 만합니다 (3 장 3.4).

15.3 dpram_wrapper.v / spram_wrapper.v —BRAM 껍데기

dpram_wrapper.v:spram_wrapper.v 는 3 장 3.4 의 BRAM 을 추상화한 래퍼입니다. 출력 버퍼 (OFM) 와 라인 버퍼 bank 의 기반입니다.

- **dpram**(dual-port): 쓰기 포트 (A) 와 읽기 포트 (B) 가 따로 있어 동시에 가능. OFM 버퍼 (65536×32) 와 라인 버퍼 bank(2048×128).
- **spram**(single-port): 한 포트로 읽기/쓰기 시분할. gbuff bias 등.

[코드] 코드 해설—파라미터로 여러 크기 지원

```
parameter DW=64, AW=8, DEPTH=256, N_DELAY=1;
`ifdef FPGA
    if((DEPTH==65536) && (DW==32)) begin // OFM 버퍼
        dpram_65536x32 u_dpram_65536x32 (...);
    end else if((DEPTH==2048) && (DW==128)) begin // 라인 버퍼
        dpram_2048x128 u_dpram_2048x128 (...);
    end
    // ...
`else
    reg [DW-1:0] ram[0:DEPTH-1]; // 시뮬: reg 배열
`endif
```

(DEPTH, DW) 조합에 따라 맞는 BRAM IP 를 고릅니다 (3 장 3.4 BMG). 같은 래퍼 코드로 여러 크기 메모리를 만듭니다.

[코드] 코드 해설—시뮬 메모리 0 초기화 (중요)

```
`else // 시뮬레이션 경로
    reg [DW-1:0] ram[0:DEPTH-1];
    integer dpram_init_idx;
    initial begin // * uninitialized X 방지
        for (dpram_init_idx=0; dpram_init_idx<DEPTH; dpram_init_idx++)
            ram[dpram_init_idx] = {DW{1'b0}};
    end
    always @(posedge clk) if(ena && wea) ram[addra] <= dia; // 쓰기
    // 읽기: N_DELAY=1 → 1 클럭 지연
    always @(posedge clk) if(enb) rdata <= ram[addrb];
`endif
```

- **initial** 로 메모리를 0 으로 채웁니다. 4 장 4.7 에서 본 “uninitialized X”가 시뮬레이터 버전마다 다르게 처리되어 검증이 비결정적이 되던 문제를 막습니다 (21 장).
- **initial** 은 시뮬 전용 (합성에는 BMG IP 경로라 무관) 이고, 실제 FPGA BRAM 의 0 초기화와 일치합니다 → **fps·Energy·정확도에 영향 없음**, 검증 재현성만 확보.
- 읽기 N_DELAY=1 → BRAM read latency 1 클럭 (3 장 3.4, 4 장 4.4). BMG 의 “No Output Register” 설정과 같습니다.

15.4 네 버퍼 비교

버퍼	모양	포트	특이점
ifm_line_buf (×4 bank)	2048×128	dual	cyclic, 윈도우 추출, latency 2
gbuff_param 가중 치	4096×72 (읽기 288)	dual 비대칭	쓰기 72/읽기 288
gbuff_param bias	2560×32	single	{bias16,shift16}
OFM dpram	65536×32	dual	5 방향 공유, latency 1

모두 `ifdef FPGA 로 합성 (BMG IP)/시물 (reg+0 초기화) 두 경로를 가집니다.

15.5 이 장의 요약

- ifm_line_buf: 4 줄을 cyclic 으로 쌓고 3×3/1×1 윈도우를 36 바이트씩 4 번 추출 (latency 2, look-ahead 로 보상).
- 3×3 은 entry 경계를 offset 으로 잘라 붙이고 가장자리 0 패딩, 1×1 은 채널을 byte 0/9/18/27 에 정렬 (Phase 2 버그 수정점).
- gbuff_param: 가중치는 쓰기 72/읽기 288 비대칭 (한 번에 36 개 공급), bias 는 {bias16,shift16}. 가중치는 필터별 streaming.
- dpram/spram_wrapper: 파라미터로 여러 BRAM 크기 지원, 시물은 reg+initial 0 초기화 (X 방지, 검증 재현성).
- 모든 버퍼가 합성/시물 두 경로, read latency 1(라인버퍼는 윈도우 + 레지스터로 2).

다음 장에서는 합성곱이 아닌 풀링·업샘플·REPACK 특수 유닛을 봅니다.

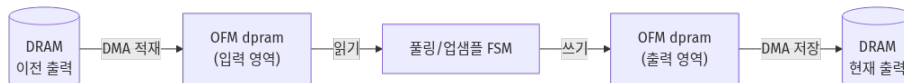
16 장. RTL —특수 연산 유닛 (코드 해설)

이 장을 읽기 위한 준비: 1 장 1.7~1.8 풀링·업샘플, 12 장 12.2~12.3 포맷·REPACK.

합성곱 외의 형상 변환을 담당하는 세 모듈 (`max_pool_unit`, `max_pool_sl_unit`, `upsample_unit`) 과, 데이터 포맷을 바꾸는 REPACK 엔진을 코드로 봅니다. 이들의 공통점은 출력 버퍼 (**OFM dpram**) 를 직접 읽고 쓰는 단순 **FSM** 이라는 것입니다—합성곱보다 훨씬 간단합니다.

16.1 공통 패턴—load → process → store

세 유닛 모두 11 장 11.3 의 출력 버퍼에 붙어 다음 패턴으로 동작합니다.



공통 인터페이스: `i_start/o_done`(시작·완료), `o_rd_*/i_rd_data`(읽기), `o_wr_*`(쓰기). 데이터는 모두 12 장 12.2-(3) 의 `2x2 packed`(32 비트 = 4 픽셀) 입니다.

16.2 `max_pool_unit.v` —stride-2 풀링

`max_pool_unit.v` 는 L1/3/5/7/9 의 `2x2 stride-2` 풀링 (1 장 1.7) 입니다.

핵심 통찰—합성곱 `2x2` 블록 = 풀링 윈도우

합성곱이 만든 32 비트 (`2x2` 블록 4 픽셀) 가 곧 풀링의 **`2x2` 윈도우**입니다! 그래서 입력 32 비트의 4 바이트 최댓값을 구하면 풀링 출력 1 픽셀이 됩니다. 윈도우 슬라이딩이 필요 없습니다.

입력 word = {b3, b2, b1, b0} (`2x2` 블록)
→ `max(b0,b1,b2,b3)` = 풀링 출력 1 바이트
→ 4 개 모으면 다음 `2x2` 출력 블록 (32 비트)

[코드] 코드 해설—max-of-4

```
wire [7:0] b0 = i_rd_data[7:0], b1 = i_rd_data[15:8],  
          b2 = i_rd_data[23:16], b3 = i_rd_data[31:24];  
wire [7:0] m01 = (b0 > b1) ? b0 : b1; // 둘 중 큰 것  
wire [7:0] m23 = (b2 > b3) ? b2 : b3;  
wire [7:0] in_max = (m01 > m23) ? m01 : m23; // 넷 중 큰 것
```

비교 3 번으로 4 개 중 최대값. 1 장 1.7 의 max pooling 을 하드웨어로 만들면 이렇게 단순합니다.

[코드] 코드 해설—4 개 모아 저장

```
case (buf_cnt_r)  
  2'd0: buf_b0 <= in_max;  
  2'd1: buf_b1 <= in_max;  
  2'd2: buf_b2 <= in_max;  
  2'd3: begin // 4 번째 → 32 비트로 묶어 쓰기  
    packed_word_r <= {in_max, buf_b2, buf_b1, buf_b0};  
    packed_vld_r <= 1'b1;  
    out_addr_r <= out_addr_r + 1;  
  end  
endcase
```

출력 픽셀 4 개를 모아 32 비트로 묶어 씁니다 (12 장 12.2-(3)).

[비유] **in-place** 안전성: 출력 word K 는 입력 word 4K+3 까지 읽은 뒤 씁니다. 읽기 주소 (4K+3) 가 쓰기 주소 (K) 보다 항상 앞서므로, 같은 버퍼에 덮여써도 안전합니다. 별도 출력 버퍼가 필요 없어 메모리를 아킵니다.

16.3 max_pool_s1_unit.v —stride-1 풀링 (L11 전용)

max_pool_s1_unit.v 는 L11 전용입니다. stride=1 이라 크기가 유지되고 (1 장 1.7) 윈도우가 겹쳐서, stride-2 모듈과 완전히 다른 동작이 필요합니다. 그래서 전용 모듈입니다.

왜 어려운가—출력 한 픽셀이 4 개 입력 블록에 걸침

stride-1 2×2 윈도우는 packed 블록 경계를 가로지릅니다. 출력 블록의 네 픽셀이 **인접한 4 개 입력 블록**의 특정 바이트에서 옵니다.

[코드] 코드 해설—4-way max

```
wire [7:0] out_pix_00 = max4(rc_b0, rc_b1, rc_b2, rc_b3); // 블록 RC 안에서  
wire [7:0] out_pix_01 = max4(rc_b1, rc1_b0, rc_b3, rc1_b2); // RC + 오른쪽 블록 RC1  
wire [7:0] out_pix_10 = max4(rc_b2, rc_b3, r1c_b0, r1c_b1); // RC + 아래 블록 R1C  
wire [7:0] out_pix_11 = max4(rc_b3, rc1_b2, r1c_b1, r1c1_b0); // RC + 세 이웃 블록
```

rc(현재), rc1(오른쪽), r1c(아래), r1c1(우하단) —네 블록에서 골라 max. 경계 밖 (C+1≥W 등) 이면 0 으로 채웁니다 (same-padding, 1 장 1.7).

[코드] 코드 해설—7-phase FSM

```
case (cnt_phase)
  // phase 0~3: 4 개 입력 블록 읽기 주소 발사
  // phase 2~5: 도착한 데이터 샘플 (BRAM latency 1 보상)
  3'd2: cache_rc <= i_rd_data;
  3'd3: cache_rc1 <= valid_rc1 ? i_rd_data : 0;
  3'd4: cache_rlc <= valid_rlc ? i_rd_data : 0;
  3'd5: cache_rlc1 <= valid_rlc1 ? i_rd_data : 0;
  // phase 6: 출력 쓰기 + 다음 블록으로
endcase
```

출력 블록 하나당 7 클럭 (phase 0~6). 4 개 블록을 순차로 읽어야 해서 (4 장 4.4 BRAM latency), 발사 (0~3) 와 샘플 (2~5) 을 어긋나게 배치합니다. L11 전체는 약 512 채널 × 16 블록 × 7 ≈ 57,000 클럭.

[핵심] **dpram 분할**: 입력은 [0..8191], 출력은 [8192..] 에 둡니다. 입력과 출력 영역을 분리해 in-place 충돌을 피합니다 (12 장 12.8).

16.4 upsample_unit.v —2× 업샘플 (L18 전용)

upsample_unit.v 는 L18 에서 8×8 을 16×16 으로 키웁니다 (1 장 1.8). 곱셈 없이 값을 복제합니다.

[코드] 코드 해설—한 픽셀을 2×2 로 복제

```
wire [7:0] pix_00 = cache_in[7:0];
wire [31:0] word_00 = {pix_00, pix_00, pix_00, pix_00}; // 같은 값 4 번 (2×2 블록)
```

입력 한 픽셀 (pix_00) 을 32 비트 (2×2 블록, 같은 값 4 번) 로 만듭니다. 입력 블록의 4 픽셀 각각이 출력에서 2×2 블록이 되어, 8×8 → 16×16(1 장 1.8).

[코드] 코드 해설—6-phase FSM

```
case (cnt_phase)
  3'd0: begin rd_en_r<=1; rd_addr_r<=addr_in; end // 읽기 발사
  3'd1: ; // 대기 (BRAM latency)
  3'd2: begin cache_in<=i_rd_data; wr_data_r<=word_00; end // 샘플 + 좌상 쓰기
  3'd3: wr_data_r <= word_01; // 우상 쓰기
  3'd4: wr_data_r <= word_10; // 좌하 쓰기
  3'd5: begin wr_data_r<=word_11; /* 다음 블록 */ end // 우하 쓰기 + 진행
endcase
```

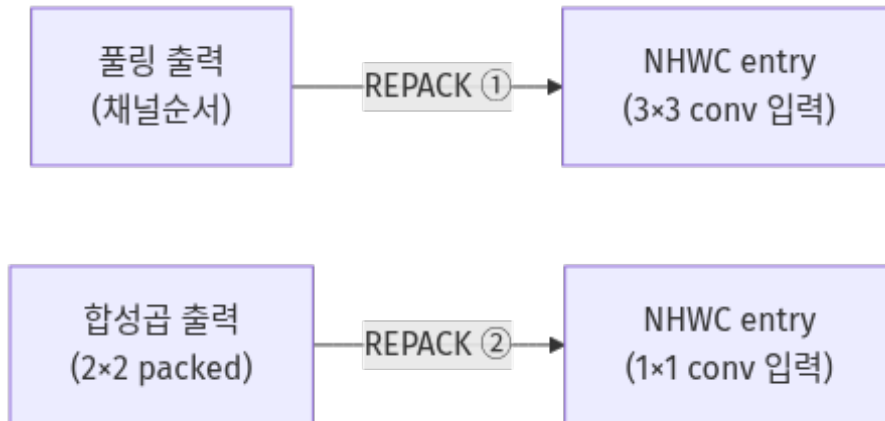
입력 블록 하나당 6 클럭: 1 읽기 + 1 대기 + 4 쓰기 (2×2 출력 블록 4 개). L18 전체는 128 채널 × 16 블록 × 6 ≈ 12,000 클럭.

업샘플은 곱셈이 전혀 없어 단순한 복사 FSM 입니다. 7 장 7.7 에서 “Route 와 달리 주소 패턴이 규칙적이라 전용 모듈화가 깔끔”한 경우입니다.

16.5 REPACK 엔진—포맷 갈아입히기

REPACK 은 별도.v 파일이 아니라 yolo_engine FSM 에 내장된 변환입니다. 12 장 12.3 에서 본 “옷 갈아입히기”(채널순서/2×2 packed → NHWC entry) 입니다. 곱셈이 없고 메모리 재배치만 합니다.

두 종류



- **(1) 풀링 → 합성곱 (L1→L2 등):** 채널순서 데이터를 작은 버퍼 (scratch) 에 적재 → 열별로 4 채널을 묶어 (transpose) entry 형성 → DRAM 저장.
- **(2) 합성곱 → 합성곱 (L11→L12 등):** 2×2 packed 에서 바이트를 추출해 entry 로 재배치. entry 당 13 클럭 (8 읽기 + 4 쓰기).

L19 Route concat (2-source REPACK)

L20 입력은 L18(128 채널) 과 L8(256 채널) 을 합친 384 채널입니다 (6 장 6.6, 10 장). FSM 이:
 1. S_L19_RP_LOAD_A: L18 출력을 버퍼 앞쪽에 적재
 2. S_L19_RP_LOAD_B: L8 출력을 버퍼 뒤쪽에 적재
 3. S_L19_RP_GEN: 둘을 묶어 NHWC entry 로 변환 → L20 입력

이것이 6 장 6.6 의 skip connection 을 하드웨어로 구현한 것입니다. L8 을 위해 DRAM 에 보존해둔 이유가 이것입니다 (12 장 12.7).

16.6 특수 유닛 비교

유닛	레이어	동작	클럭/단위	dpram 분할
max_pool_unit	1,3,5,7,9	2×2 블록 = 윈도우, max-of-4	~1/word	in-place
max_pool_slit	1,11	stride-1, 4 블록 걸침	7/block	0 / 8192

유닛	레이어	동작	클럭/단위	dpram 분할
upsample_unit(18)		1 픽셀 → 2×2 복제	6/block	0 / 2048
REPACK (1)	L2,4,6,8,10 전	채널순서 → entry	~9/열	scratch
REPACK (2)	L12,13,14,17 전	2×2 packed → entry	13/entry	dpram 영역

16.7 이 장의 요약

- 특수 유닛은 모두 출력 버퍼에 붙어 적재 → 처리 → 저장 패턴, 2×2 packed(32 비트) 데이터를 다룸.
- max_pool_unit(stride-2): 합성곱 2×2 블록이 곧 윈도우 → max-of-4 로 끝, in-place 안전.
- max_pool_s1_unit(L11): stride-1 이라 출력 픽셀이 4 개 입력 블록에 걸침 → 7-phase FSM + same-padding.
- upsample_unit(L18): 한 픽셀을 2×2 로 복제, 6-phase FSM, 곱셈 없음.
- REPACK 은 yolo_engine 내장 FSM: 채널순서/2×2 packed → NHWC entry 재배치. L19 는 L18+L8 concat(skip connection).

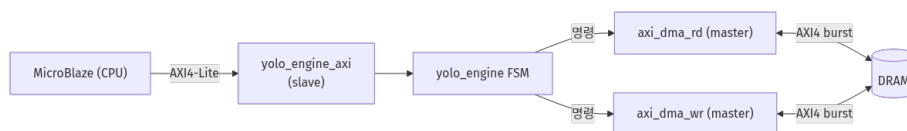
다음 장에서는 외부 DRAM·CPU 와 통신하는 **AXI/DMA** 인터페이스를 봅니다.

17 장. RTL —AXI / DMA 인터페이스 (코드 해설)

이 장을 읽기 위한 준비: 5 장 AXI 프로토콜, 4 장 FSM.

가속기가 외부 세계와 통신하는 경계입니다: DRAM 과 데이터를 주고받는 **DMA** 한 쌍 (axi_dma_rd, axi_dma_wr), CPU 명령을 받는 **AXI-Lite slave**(yolo_engine_axi), 그리고 합성/시뮬을 가르는 매크로 파일. 5 장에서 배운 AXI 개념이 실제 코드로 어떻게 나타나는지 봅니다.

17.1 통신 경계 전경



5 장 5.2 에서 본 대로, yolo_engine 은 CPU 에 **slave**(명령 받음), DRAM 에 **master**(직접 접근) 입니다.

17.2 axi_dma_rd.v —DRAM 읽기 DMA

axi_dma_rd.v 는 DRAM 에서 가중치·bias·입력을 읽어옵니다. 5 장 5.6 의 DMA 엔진입니다.

기능 인터페이스—yolo_engine 과 약속

```
input  start_dma;           // 1 클럭 펄스: 시작
input  [BITS_TRANS-1:0] num_trans; // 읽을 워드 수
input  [AXI_WIDTH_AD-1:0] start_addr; // DRAM 시작 주소
output data_o, data_vld_o;   // 받은 데이터 (1 클럭 지연)
output done_o;               // 완료 펄스
```

yolo_engine 이 “이 주소에서 N 워드 읽어”(start_dma, start_addr, num_trans) 라고 시키면, DMA 가 알아서 AXI burst 로 읽어 data_o 로 흘려줍니다. 다 되면 done_o(5 장 5.6).

[코드] 코드 해설—5-state FSM

```
localparam RD_IDLE=0, RD_PRE=1, RD_START=2, RD_SEQ=3, RD_WAIT=4;

case(st_rdx)
  RD_IDLE: if(start_dma_d) next = RD_PRE;           // 시작 대기
  RD_PRE:  if(q_burst_cnt_rd == num_trans_d) next = RD_IDLE; // 다 읽음 → 종료
           else next = RD_START;                   // 아니면 burst 시작
  RD_START: if(ext_arready) begin                  // AR 핸드셰이크
              ext_arvalid = 1'b1;
              ext_arlen    = q_burst_size_rd;      // burst 길이
              ext_arsize   = 3'b010;              // 4 바이트/beat
              ext_arburst  = 2'b01;               // INCR
              next = RD_SEQ;
            end
  RD_SEQ:  if(ext_rlast_r) next = RD_WAIT;         // 마지막 beat 수신
  RD_WAIT: begin d_burst_cnt += burst_size; next = RD_PRE; end // 다음 burst
endcase
```

5 장 5.3 handshake 와 5 장 5.5 burst 가 보입니다: - RD_START: AR 채널로 “이 주소에서 burst 길이만큼 줘” 요청. ext_arready(slave 준비) 를 확인하고 ext_arvalid(요청 유효) 를 올림 = 핸드셰이크. - ARSIZE=4 바이트, ARBURST=INCR(주소 자동 증가). - RD_SEQ: R 채널에서 데이터가 줄줄이 들어옴. ext_rlast(마지막 beat) 까지 받음. - RD_WAIT → RD_PRE: 다음 burst 로. num_trans 를 다 읽을 때까지 반복.

[코드] 코드 해설—burst 분할

```
localparam FIXED_BURST_SIZE = 256; // 한 burst 에 256 워드 (1KB)

if(q_burst_cnt_rd + FIXED_BURST_SIZE > num_trans_d)
  q_burst_size_rd <= num_trans_d[7:0] - 1; // 마지막 burst: 남은 만큼
else
  q_burst_size_rd <= FIXED_BURST_SIZE - 1; // 중간 burst: 256 개
```

큰 데이터를 **256 워드 (1KB)** 단위로 나눠 여러 burst 로 읽습니다 (5 장 5.5). 마지막 burst 는 남은 워드만큼만. 주소를 매번 보내는 오버헤드를 줄여 대역폭을 높입니다.

M_RREADY(받을 준비) 는 항상 1 입니다—데이터가 오는 대로 즉시 받습니다 (back-pressure 없음).

17.3 axi_dma_wr.v —DRAM 쓰기 DMA

axi_dma_wr.v 는 출력 버퍼의 결과를 DRAM 에 씁니다. 읽기와 대칭이지만, 데이터를 yolo_engine 에서 받아야 하므로 요청 신호가 추가됩니다.

[코드] 코드 해설—데이터 요청

```
output indata_req_o; // " 다음 데이터 줘 " 요청
input  indata;       // yolo_engine 이 공급하는 데이터
```

indata_req_o=1 인 클럭의 다음 클럭에 indata 가 유효해야 합니다 (1 클럭 look-ahead). yolo_engine 은 이 요청에 맞춰 출력 버퍼 읽기 주소를 진행시켜 데이터를 공급합니다 (13 장

13.9).

6-state FSM

WR_IDLE → WR_PRE → WR_START → WR_SEQ → WR_WAIT → (반복)

- WR_START: AW 채널로 주소·길이 전송 (5 장 5.3).
- WR_SEQ: W 채널로 데이터 beat 전송, M_WSTRB=4'b1111(4 바이트 모두 유효), 마지막에 M_WLAST=1.
- WR_WAIT: B 채널 응답 (BRESP=OKAY) 대기. AXI 쓰기는 응답 채널 (B) 이 있어“잘 썼는지”확인 합니다 (5 장 5.4).

17.4 yolo_engine_axi.v —CPU 제어 받기

yolo_engine_axi.v 는 5 장 5.4 의 AXI4-Lite slave 로, CPU 가 읽고 쓰는 제어 레지스터 4 개를 제공합니다.

레지스터 맵

주소	레지스터	의미
0x0	ctrl_reg0	[0]=ap_start(쓰기), 읽으면 [1]=network_done
0x4	ctrl_reg1	가중치 DRAM 주소
0x8	ctrl_reg2	입력 DRAM 주소
0xC	ctrl_reg3	출력 DRAM 주소

[코드] 코드 해설—완료 polling 트릭

```
// ctrl_reg0 를 읽을 때, bit[1] 에 network_done 을 끼워 넣음
reg_data_out = { slv_reg0[31:2], network_done, slv_reg0[0] };
```

CPU 는 ctrl_reg0 을 반복해서 읽으며 bit1이 1 이 되기를 기다립니다. RTL 이 추론을 끝내면 network_done 을 1 로 올리고, CPU 가 그것을 읽어“완료”를 압니다.

```
CPU: ctrl_reg1/2/3 ← 주소 쓰기
ctrl_reg0 ← 1 (ap_start)
loop { ctrl_reg0 읽기 → bit1==1 ? 완료 : 계속 대기 }
→ 출력 DRAM 읽어 후처리
```

이 흐름이 22 장 host.py 의 추론 시퀀스입니다.

17.5 user_define_h.v / define.v —매크로

user_define_h.v 는 8 장 8.8 에서 본 `define FPGA 를 정의합니다.

```

`define NUM_BRAMS 16
`define BRAM_WIDTH 128
//`define FPGA 1 // ← 합성 시 주석 해제 / 시뮬 시 주석 (현재)

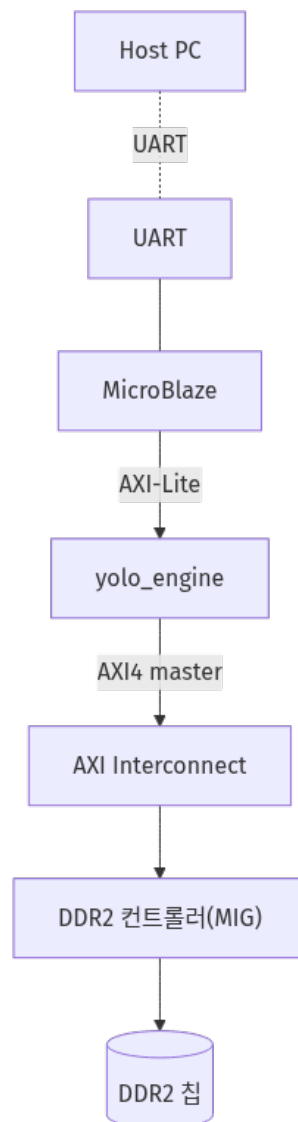
```

`define FPGA 가 켜지면 합성용 IP(DSP48·BRAM) 경로, 꺼지면 시뮬용 behavioral 경로가 컴파일됩니다 (3 장 3.6, 4 장 4.7). 합성에 필요한 IP 목록도 이 파일 주석에 정리되어 있습니다 (21 장).

define.v 는 mul.v 가 참조하는 빈 stub 입니다. 실제 매크로는 모두 user_define_h.v 에 있고, 이 파일은 컴파일 호환성용으로 비어 있습니다.

17.6 Phase 3 —실제 SoC 통합 (전망)

현재 Phase 1·2에서는 Testbench 가 가짜 DRAM 모델로 AXI 에 응답합니다 (19 장). Phase 3에서는 실제 SoC 에 통합됩니다 (22 장).



yolo_engine 의 master 2 개 (읽기·쓰기) 가 interconnect 를 거쳐 MicroBlaze 와 DDR2 대역

꼭을 공유합니다. DMA 코드는 그대로 재사용됩니다.

17.7 이 장의 요약

- 데이터 평면: `axi_dma_rd`(5-state, 256 워드 burst) 와 `axi_dma_wr`(6-state, indata 1 클럭 look-ahead 공급) 의 master 한 쌍.
- 두 DMA 모두 5 장의 `handshake·burst·INCR` 을 코드로 구현, `done_o` 로 완료 알림.
- 제어 평면: `yolo_engine_axi`(AXI-Lite slave) —`ctrl_reg0~3`(`ap_start` 주소), 읽기 시 `bit[1]` 에 `network_done` 끼워 polling 지원.
- `user_define_h.v` 의 ``define` FPGA 가 합성/시물 전환, `define.v` 는 빈 호환 stub.
- Phase 3 에서 MicroBlaze + interconnect + DDR2 와 통합 예정.

이것으로 모든 RTL 모듈 (13~17 장) 을 다뤘습니다. 다음 장에서는 이들이 시간 축에서 어떻게 협력하는지 **동작·타이밍**을 종합합니다.

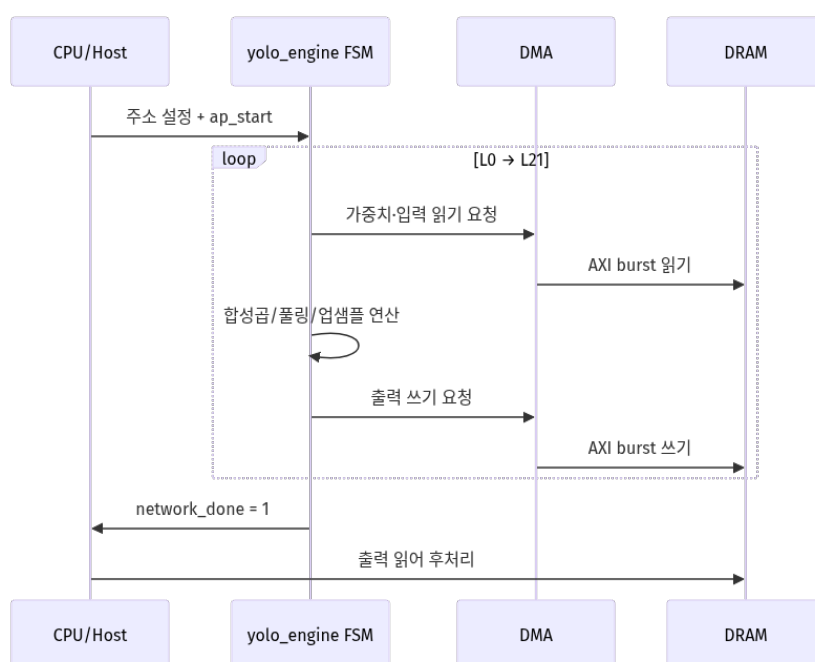
18 장. 동작 과정과 타이밍

이 장을 읽기 위한 준비: 4 장 4.3 파이프라인, 13~17 장 RTL 모듈.

13~17 장에서 모듈을 하나씩 봤다면, 이 장은 그것들이 시간 축에서 어떻게 협력해 한 장의 추론을 완성하는지 종합합니다. 전체 시퀀스 → 레이어 내부 → MAC 파이프라인 → 성능 추정 순으로 좁혀 들어갑니다.

18.1 전체 추론 시퀀스

ap_start 한 번으로 yolo_engine FSM 이 L0 부터 L21 까지 자동 순회합니다.



“읽기 → 연산 → 쓰기”가 22 번 반복됩니다 (11 장 11.3). 시간 비중은 합성곱이 압도적입니다 (10 장 10.7).

합성곱 (88%, 특히 6 개 3×3 레이어)
 1×1 합성곱 + 검출 헤드
 풀링/업샘플/REPACK (제어 오버헤드)

18.2 합성곱 한 레이어의 cycle 흐름

13 장 13.6 의 이중 루프를 시간으로 펼치면:

```

레이어 시작
├─ bias 읽기 (레이어당 1 회)
├─ for 행블록 rb:
│   └─ 입력 읽기 (DMA) → 라인 버퍼          ← DMA 시간
│       └─ for 필터 fi:
│           └─ 가중치 읽기 (DMA) → gbuff      ← DMA 시간
│               └─ conv_top 실행 (1 필터 x1 행블록) ← 연산 시간
│                   └─ 출력 쓰기 (DMA)        ← DMA 시간

```

핵심 관찰: - **DMA 와 연산이 순차로 일어납니다.** 현재 구조는“읽고 → 계산하고 → 쓰기”가 직렬이라 단순하지만, DMA 시간이 연산 시간에 더해집니다. - 작은 레이어 (L17) 는 DMA·제어 오버헤드 비중이, 큰 레이어 (L2~L10) 는 연산 비중이 큼니다.

18.3 MAC 파이프라인 타이밍—latency vs throughput

4 장 4.3 에서 배운 두 개념이 여기서 빛납니다. conv_top 의 ST_RUN 구간을 보면:

입력 (매 클럭) ─┐
 ├─ 곱셈 4 ─┐
 ├─ 가산트리 4 ─┐
 ├─ 누적 acc_len ─┐
 └─ 후처리 1 → 출력

총 latency = 8 + acc_len + 1

중요—latency 는 보이지 않고 throughput 이 진짜 속도

```

클럭:  0   1   2   ...  acc_len  ...
입력:  ●   ●   ●   ...  (다음 픽셀 그룹)
출력:           (채움 후)  매 acc_len 클럭마다 4 픽셀 ✓

```

4 장 4.3 의 공장 비유처럼, 파이프라인이 차면 **acc_len 클럭마다 4 픽셀**이 나옵니다. latency(8+acc_len+1) 는 파이프라인 채움·비움 (ST_LOAD/ST_DRAIN) 에만 보입니다.

레이어	acc_len	처리량
L0 (Ci=3)	1	1 클럭마다 4 픽셀 (가장 빠름)
L10 (Ci=256)	64	64 클럭마다 4 픽셀

[핵심] 채널이 깊을수록 (acc_len↑) 한 출력당 오래 걸리지만, 출력 공간이 작아 (8×8) 균형이 맞습니다. 10 장 10.7 에서 본“6 개 3×3 레이어가 모두 75.5 M MAC 으로 같은”구조적 균형과 같은 이치입니다.

18.4 BRAM latency 정렬—종합

이 설계에서 가장 섬세한 부분은 4 장 4.4·4.8 에서 본 여러 **BRAM**·파이프라인 지연을 한 클럭도 안 어긋나게 맞추는 것입니다. 곳곳의 정렬 장치를 한 표에 모으면:

위치	장치	목적
conv_top → gbuff	mac_vld_d(1 클럭 지연)	가중치 BRAM latency 1 흡수
conv_top → 라인버퍼	look-ahead 좌표	라인버퍼 latency 2 보상
라인버퍼 내부	offset_r 등 pipeline	BRAM 출력과 판정 정렬
max_pool	issued_d 지연 플래그	읽기 결과 샘플 타이밍
max_pool_s1	phase 0~3 발사 / 2~5 샘플	4-read 순차 정렬
DMA	start_dma_d, ext_rlast_r 래치	AXI 핸드셰이크 정렬

[주의] Phase 2 버그 상당수가 이 정렬 문제였습니다. 한 클럭만 틀려도 결과가 완전히 망가지므로 가장 조심하는 부분입니다 (14 장 14.7, 4 장 4.8).

18.5 특수 레이어 타이밍

16 장에서 본 유닛들의 cycle 비용:

레이어	유닛	클럭/단위	총 클럭 (대략)
L1/3/5/7/9	max_pool	~1/word	입력 word 수
L11	max_pool_s1	7/block	~57,000
L18	upsample	6/block	~12,000
REPACK	FSM	9~13/단위	(작음)

합성곱에 비하면 작지만, 각 레이어 전후로 DMA 가 직렬로 붙어 무시 못 할 오버헤드가 됩니다.

18.6 DMA 타이밍

17 장의 DMA 는 256 워드 (1KB) burst 단위입니다 (5 장 5.5).

start_dma → [준비 → AR 핸드셰이크 → 256 beat 수신 → 응답] × burst 수 → done

- 한 burst = AR 핸드셰이크 (수 클럭) + 256 beat(이상적 256 클럭) + 응답.
- 큰 레이어의 입력·가중치는 여러 burst 로 나뉘고, burst 사이 오버헤드가 누적됩니다.
- 실제 DDR2 대역폭·지연은 Phase 3 에서 확정됩니다 (22 장).

18.7 성능 추정

[주의] 아래는 이상적 합성곱 처리량 기반 거친 추정입니다. DMA 대기·제어 오버헤드·실제 DDR2 대역폭이 빠져 있어, 확정 fps·전력은 Phase 4 보드 측정에서 결정됩니다.

합성곱 클럭 (이상적)

레이어 클럭 $\approx$ (출력너비/2) $\times$ (출력높이/2) $\times$ 출력채널 $\times$ acc_len:

레이어	클럭 (대략)
L0	262 K
L2,4,6,8,10,13 (각)	524 K
L12	524 K
L14	399 K
L17	131 K
L20	1198 K
합계	$\approx$ 5.7 M 클럭

fps 추정

이상적 합성곱: 5.7 M 클럭 @ 100 MHz = 57 ms $\rightarrow$ ~17 fps (합성곱만)
오버헤드 포함: DMA · 풀링 · REPACK · 파이프라인 $\rightarrow$ 더 느려짐
목표: 5 fps = 200 ms = 20 M 클럭 @ 100 MHz

- 이상적으로는 ~17 fps 이지만, DMA·제어 오버헤드를 더하면 낮아집니다.
- 목표 **5 fps(20 M 클럭 예산)** 안에 들 가능성이 높습니다. 합성곱 5.7 M 에 오버헤드를 더해도 여유가 있습니다.
- 정확한 값은 실제 클럭 주파수 (timing closure 후, 3 장 3.7) 와 DDR2 대역폭에 좌우됩니다.

18.8 Energy —점수의 분모

7 장 7.2 에서 점수 분모가 Energy 임을 봤습니다. 이 설계가 Energy 를 줄이는 선택들:

선택	효과	배운 곳
INT8 정수 연산	곱셈가·스위칭 에너지 ↓	2 장
144-MAC 병렬 + 빠른 완료	짧은 시간 $\rightarrow$ 누설 에너지 ↓	14 장

선택	효과	배운 곳
온칩 버퍼 재사용 (streaming, in-place)	DRAM 접근 ↓ (최대 에너지 지원)	3 장 3.4, 15·16 장
연산 유닛 1 개 재사용	유휴 면적 ↓	11 장

[핵심] DRAM 접근이 온칩 연산보다 훨씬 큰 에너지를 씁니다. 그래서 **streaming** 가 **중치 + in-place 풀링 + 출력 버퍼 재사용**이 Energy 점수에 직접 기여합니다. 실측 전력은 Phase 4 보드 측정으로 확정됩니다.

18.9 이 장의 요약

- ap_start 하나로 FSM 이 L0~L21 을 순회, 레이어마다 읽기 → 연산 → 쓰기 반복.
- 합성곱 한 (필터, 행블록) 은 채움 (ST_LOAD) + 운전 (ST_RUN) + 비움 (ST_DRAIN); 처리량은 4 픽셀/acc_len 클럭.
- MAC latency 8+acc_len+1 은 채움·비움에만 보이고, 정상 운전은 acc_len 주기로 4 픽셀 (throughput 이 진짜 속도).
- 여러 BRAM·파이프라인 지연을 look-ahead·지연 플래그로 정밀 정렬 (Phase 2 버그 영역).
- 이상적 합성곱 ~5.7 M 클럭 ($\approx 17 \text{ fps}@100 \text{ MHz}$), 오버헤드 포함해도 목표 5 fps 예산 내 가능—확정은 Phase 4.
- Energy 는 INT8·버퍼 재사용으로 절감 (DRAM 접근 최소화가 핵심).

다음 장부터는 이 동작이 “정확한지” 확인하는 검증 (Testbench) 을 봅니다.

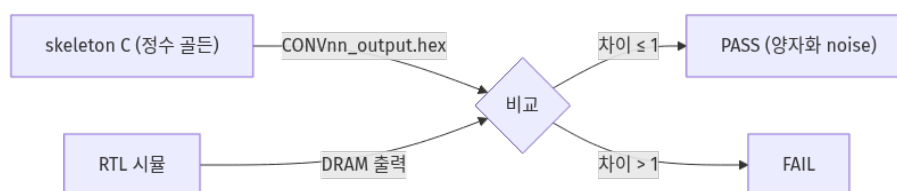
19 장. 검증 전략과 인프라

이 장을 읽기 위한 준비: 9 장 skeleton 골든, 2 장 2.10 tolerance.

“RTL 이 정확한가?”를 객관적으로 판정하는 방법을 다룹니다. 검증을 이해하면 20 장의 결과 (mismatch 수) 를 올바르게 읽을 수 있습니다.

19.1 검증 철학—골든과 비교

모든 검증의 기준은 하나입니다: “RTL 출력이 skeleton C(정수 골든) 와 같은가?” (9 장)

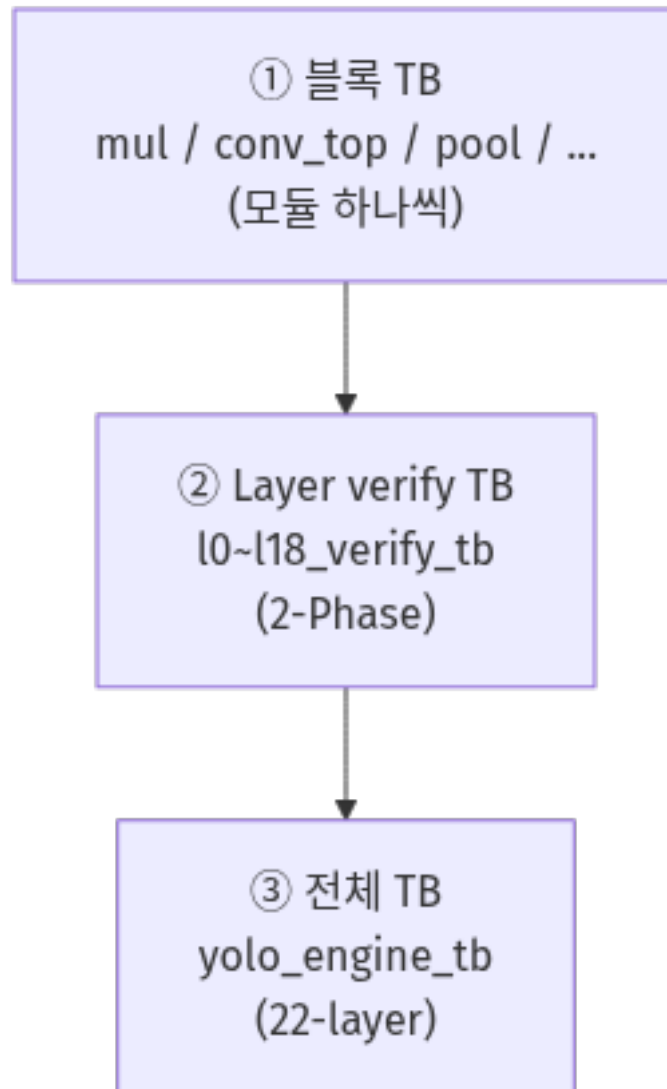


- **PASS:** 골든과 정확히 같거나 (exact), $|RTL - 골든| \leq 1$. 1 LSB 차이는 2 장 2.10 에서 본 양자화/반올림 noise 로 간주.
- 이 정의 덕분에 “정확함”이 주관이 아니라 **측정 가능한 수치** (mismatch 개수) 가 됩니다.

[비유] 비유: 채점 기준표가 있는 시험과 같습니다. “정답 (골든) 과 한 자리 차이까지는 정답 처리”라는 명확한 기준이 있어, 누가 채점해도 같은 결과가 나옵니다.

19.2 3 계층 검증—작은 것부터

검증은 작은 단위부터 전체까지 쌓아 올립니다.



계층	TB	검증 대상
(1) 블록	mul_tb, conv_top_tb, pool_tb, ...	단일 모듈
(2) 레이어	l0~l18_verify_tb	한 레이어 (단독 + chain)
(3) 전체	yolo_engine_tb	22-layer 완주

작은 단위가 맞아야 큰 단위로 올라갑니다. 자세한 내용은 20 장.

19.3 2-Phase 검증—같은 레이어를 두 방식으로

Layer verify TB 의 핵심입니다. 같은 레이어를 두 방식으로 검증해 신뢰도를 높입니다.

Phase A —Standalone (단위 테스트)

해당 레이어만 단독 실행. golden 입력을 직접 넣으므로 그 레이어 **RTL** 의 정확성만 분리 검증합니다.

```
// golden 입력을 REPACK 해서 DRAM 에 배치
software_repack_l13_ifm;
// 레이어 진입점으로 강제 점프 (force)
force u_yolo_engine.layer_idx    = 5'd13;
force u_yolo_engine.conv_phase_r = 5'd13;
force u_yolo_engine.state_r      = 6'd1;    // S_LOAD_BIAS
// release 후 실행 → 완료까지 대기 → 비교
release ...;
wait (u_yolo_engine.layer_idx == 5'd14);
compare_l13_ofm(mm_A, tf_A);
```

[비유] 비유: force/release 는 게임의“치트키로 특정 스테이지로 점프”하는 것입니다. L0 부터 다 거치지 않고 곧장 L13 으로 가서 L13 만 테스트합니다.

Phase B —Chain (통합 테스트)

ap_start 만 주고 L0 부터 자연 진행. 이전 레이어 출력이 누적되어 들어오므로 **REPACK·레이어 연결·FSM** 전체까지 검증합니다.

```
force slv_reg0 = 1; // ap_start 만
wait (u_yolo_engine.layer_idx == 5'd14); // L0~L13 자연 진행
compare_l13_ofm(mm_B, tf_B);
```

[핵심] 두 **Phase** 가 모두 **PASS** 여야 신뢰합니다. Phase A 만 보면“레이어는 맞지만 연결이 틀렸을”수 있고, Phase B 만 보면“어느 레이어가 틀렸는지”모릅니다. 둘을 비교 해 이중 확인합니다.

19.4 통일 로그 포맷

모든 verify TB 는 같은 형식의 로그를 출력합니다 (canonical = l13_verify_tb). 그래야 결과를 한눈에 비교할 수 있습니다.

```
[L13V-TB] ===== Phase A : Standalone L13 =====
[L13V-TB] delta dist : total=0 +d=0 -d=0 max|d|=0
[L13V-TB] |d|=1 : 0 (0%)
[L13V-TB] |d|>1 (tol-exceed) : 0
[L13V-TB][Phase A] *** PASS (exact, 0 mismatches) ***
[L13V-TB] Phase A : PASS (mismatch=0, tol-exceed=0)
[L13V-TB] Phase B : PASS (mismatch=0, tol-exceed=0)
```

규칙: 태그 [LNNV-TB], TOLERANCE=1, 판정 3 분기 (exact PASS / tolerance PASS / FAIL), **ASCII** 만 (→ 는 ->, × 는 x). 단 코드 주석은 한글 유지 (CLAUDE.md 메모리).

19.5 Tolerance 와 delta 분포—디버깅의 단서

compare 작업은 단순 통과/실패가 아니라 오차의 분포를 셉니다.

```
if (got != exp) begin
    mismatch_cnt++;
    abs_diff = |got - exp|;
    // |d|=1, 2, 3, 4~8, >8 구간별로 카운트
```

```

    if (abs_diff > TOLERANCE) tol_fail_cnt++;    // 진짜 실패
end

```

이 분포가 버그의 종류를 알려줍니다:

패턴	의미
대부분 $\backslash d\backslash =1$	양자화/반올림 noise (정상, PASS)
$\backslash d\backslash $ 가 크거나 특정 패턴	진짜 RTL 버그

[핵심] 예: L14 의 got=00 exp=ff 는 ReLU 가 음수를 잘못 0 으로 만든 버그, $\backslash |d\backslash |=1$ 이 98% 면 toward-zero rounding 차이 (2 장 2.7). 분포를 보면 “어디를 고쳐야 하는 지”가 보입니다—20 장 20.3 의 L14 디버깅 사례에서 실제로 활용됩니다.

19.6 가짜 DRAM 모델

verify TB 는 각자 간단한 **AXI4 slave DRAM** 모델을 내장합니다. 실제 DDR2 대신 시뮬레이션에서 메모리 역할을 합니다.

```

reg [31:0] dram [0:DRAM_WORDS-1];    // 4M word = 16 MB 배열
// 읽기 FSM: M_ARVALID 받으면 dram[addr] 를 beat 로 출력
// 쓰기 FSM: M_WVALID 받으면 dram[addr] = M_WDATA

```

5 장의 AXI 핸드셰이크에 응답하는 단순 모델입니다. dram_zero 로 0 초기화해 4 장 4.7 의 X 문제를 피합니다.

더 정교한 sim_dram_model/(FIFO 기반 AXI SRAM) 은 전체 통합 TB(yolo_engine_tb) 에서 보드 DDR2 에 가까운 동작을 모델링합니다.

19.7 Software REPACK —골든 입력 준비

Phase A 는 golden 입력 (채널순서) 을 합성곱 입력 포맷 (NHWC entry) 으로 직접 변환해 DRAM 에 넣습니다. 이는 16 장 16.5 의 RTL REPACK 의 **소프트웨어 참조 구현**이기도 합니다.

```

// golden(채널순서) → DRAM(NHWC entry)
tb[col_l*4 + ch_l] = golden[(ci_g*4+ch_l)*64 + row*8 + (col_b*4+col_l)];

```

[핵심] Phase A(software REPACK) 와 Phase B(RTL REPACK) 가 둘 다 PASS 면, **RTL REPACK** 이 소프트웨어 참조와 일치함이 증명됩니다 (12 장 12.3).

19.8 이 장의 요약

- 검증 기준: “RTL == skeleton C 정수 골든”, $| \text{차이} | \leq 1$ 이면 PASS(양자화 noise).
- 3 계층 (블록 → 레이어 verify → 전체) 으로 작은 것부터 검증.

- verify TB 는 2-Phase: A(force/release 단독, 단위 테스트) + B(ap_start chain, 통합). 둘 다 PASS 여야 신뢰.
- 통일 로그 (canonical=l13): TOLERANCE=1, delta 분포, 3 분기 판정, ASCII 만.
- delta 분포가 버그 종류를 가리킴 ($|d|=1$ =noise, 큰 값/패턴 = 진짜 버그).
- 가짜 DRAM 모델 + software REPACK(RTL REPACK 의 참조) 로 검증.

다음 장에서는 L0~L18 각 TB 와 결과를 구체적으로 봅니다.

20 장. Layer 별 / 블록 Testbench

이 장을 읽기 위한 준비: 19 장 검증 전략.

19 장의 검증 인프라로, 실제 각 TB 가 무엇을 검증하고 어떤 결과를 냈는지 봅니다. 결과의 1 차 출처는 HISTORY.md 입니다.

20.1 Layer verify TB 목록

yolohw/testbench/의 verify TB 와 검증 대상입니다.

TB	레이어	연산	검증 포인트
l0	L0	Conv3×3	가중치 streaming, 첫 레이어
l1	L1	Pool/2	max_pool_unit
l2	L2	Conv3×3	REPACK(풀링 → 합성곱)
l5	L5	Pool/2	L0~L5 chain
l10	L10	Conv3×3	REPACK, Ci=256
l11	L11	Pool/1	max_pool_s1, same-padding
l12	L12	Conv1×1	1×1 모드, REPACK
l13 *	L13	Conv3×3	canonical TB
l14	L14	Conv1×1	검출 헤드 (linear)
l17	L17	Conv1×1	L15/L16 스킵
l18	L18	Upsample	upsample_unit

새 패턴이 등장하는 레이어마다 TB 를 추가했습니다. 같은 패턴 (L4·L6·L8 은 L2 와 동일) 은 대표 TB 로 검증합니다.

20.2 검증 결과—L11 이 마법을 부린다

HISTORY.md 의 결과입니다. 모든 단독 (Phase A) 이 **0 mismatch** → RTL 자체 버그 없음.

TB	Phase A	Phase B (chain)	해석
l0	(없음)	22	skeleton 산술 차이
l1	0	8	L0 오차 전파
l2	0	52	합성곱 확산
l5	0	22	L2 일부
l10	0	660	10 층 누적 ($\approx 2\%$)
l11	0	0	* 풀링이 오차 흡수!
l12~l18	0	0	흡수 효과 지속 [O]

[핵심] 왜 L11 을 지나면 chain mismatch 가 0 이 되나

L0~L10 chain 에는 ± 1 LSB 오차가 쌓입니다 (skeleton float 산술 vs RTL 정수, 2 장 2.10). 그런데 **L11(max pooling)** 을 지나면 **0** 이 됩니다.

L10 출력: 어떤 픽셀이 골든보다 1 작음 (예: 골든 200, RTL 199)
↓ max pooling (주변 4 개 중 최댓값)
주변에 더 큰 값 (예: 205) 이 있으면 → 205 가 선택됨
→ ± 1 오차가 있던 199 는 가려져 사라짐 → 결과 동일!

1 장 1.7 에서 예고한 대로, max 는 “주변 최댓값”만 남기므로 작은 오차가 더 큰 이웃에 묻힙니다. 이 흡수 효과가 L11 이후 (L12~L18) 내내 지속됩니다.

[비유] 그래서 L0~L10 의 chain mismatch(8, 52, 660 등) 는 걱정할 필요 없는 양자화 **noise** 입니다 (모두 tol-exceed=0, 즉 $|d| \leq 1$). 19 장 19.5 의 delta 분포로 이를 확인합니다.

20.3 주요 TB 깊이 보기

l11 —stride-1 풀링과 가짜 PASS 방지

max_pool_s1_unit(16 장 16.3) 의 same-padding 을 검증합니다. “혹시 우연히 PASS 한 건 아닌가”까지 의심해서, golden/DRAM 의 non-zero 비율 (golden 75%, DRAM 88%) 과 packed↔채널순서 수동 디코딩까지 대조했습니다 (HISTORY 5 차). chain 오차를 0 으로 흡수하는 지점이라 특히 꼼꼼히 검증했습니다.

l14 —검출 헤드 디버깅 (delta 분포 활용의 모범)

L14~L20 은 activation=linear(ReLU 없음) 라 INT8 raw 출력입니다 (6 장 6.5). 검출 헤드를 정확히 만들려면 세 가지가 모두 맞아야 했고, 디버깅 과정이 교육적입니다 (HISTORY 9 차, 메모리).

- (1) ReLU off (i_relu_en=0):
수정 전 → got=00 exp=ff (ReLU 가 음수를 0 으로 만든 버그)
→ post_process 에 i_relu_en 추가
- (2) toward-zero rounding:
(1) 후 → |d|=1 이 98% (C 나눗셈 vs Verilog 시프트 차이)
→ round_bias 보정 추가 ([2 장 2.7])
- (3) 올바른 descale shift:
L14=6 (다음 conv L17 존재), L20=9 (다음 conv 없음)
→ L20_SHIFT 를 6→9 로 정정 ([9 장 9.4])

세 가지 모두 적용 → 0 mismatch PASS

[핵심] 19 장 19.5 에서 본 **delta** 분포가 버그 종류를 정확히 가리킨 사례입니다:
got=00 exp=ff → ReLU 문제, |d|=1 98% → rounding 문제. 그리고 검출 헤드인
데도 L14(shift 6) 와 L20(shift 9) 이 다른 이유는 다음 합성곱 유무 때문입니다 (2 장
2.6).

I17 —레이어 스킵 검증

L15(yolo)·L16(route) 는 RTL 연산이 없어, FSM 이 L14 다음에 **L15·L16** 을 건너뛰고 **L17** 로
직행합니다 (13 장 13.10). Phase B 로그에 layer_idx 14 → 17 이 찍히는 것으로 스킵 정확
성을 확인합니다.

20.4 블록 TB —모듈 하나씩

단일 모듈을 격리 검증합니다.

TB	검증 모듈	핵심
mul_tb	mul	INT8 곱셈, 부호 처리
conv_top_tb	conv_top 등	합성곱 엔진 통합—Phase 2 에서 mismatch 31→0
pool_tb / pool_s1_tb	풀링	max-of-4 / stride-1
upsample_tb	upsample_unit	2× 복제
ifm_line_buf_tb	ifm_line_buf	윈도우 패킹
dpram_tb / spram_tb	메모리	read/write
axi_dma_rd/wr_tb	DMA	burst
conv1x1_micro_tb	1×1 경로	Ci=4 Co=1 회귀 테스트

conv1x1_micro_tb 는 15 장 15.1.1 의 1×1 채널 패킹 버그를 고친 뒤, 재발 방지를
위해 보존한 소규모 회귀 테스트입니다 (Ci=4, Co=1, 4×4).

20.5 빠른 검증—iverilog

Vivado 외에 리눅스 iverilog 로 문법/elaboration 을 빠르게 확인할 수 있습니다 (FPGA 매크로 off):

```
# yolo_hw 디렉토리에서  
iverilog -g2012 -I src -s <module> -o /tmp/<m>.vvp src/*.v testbench/<tb>.v
```

단, 정밀 chain 검증 (0 mismatch 판정) 은 Vivado 2025 에서 합니다 (4 장 4.7 의 X 처리 차이 때문).

20.6 이 장의 요약

- verify TB 는 새 패턴 레이어마다 추가 (L0,1,2,5,10,11~14,17,18). 모든 Phase A 가 0 mismatch → **RTL 버그 없음**.
- L0~L10 chain 의 ± 1 LSB 는 양자화 noise(tol-exceed=0). **L11 max pooling** 이 이를 **0 으로 흡수**, L12~L18 까지 지속.
- l14 검출 헤드는 (1) ReLU off (2) toward-zero (3) 올바른 shift(L14=6, L20=9) 세 가지가 모두 맞아야 PASS —delta 분포가 버그를 가리킨 모범 사례.
- 블록 TB(mul/conv_top/pool/upsample/line_buf/dpram/dma) 와 conv1x1_micro(회귀) 로 모듈 단위 검증.

다음 장에서는 이 RTL 을 합성·시뮬하는 **Vivado 프로젝트**를 봅니다.

21 장. Vivado 프로젝트

이 장을 읽기 위한 준비: 3 장 3.6 합성흐름, 4 장 4.7 X 초기화.

RTL 을 실제 FPGA 비트스트림으로 만들고 시뮬레이션하는 Vivado 환경을 봅니다. 프로젝트 생성·BMG IP·합성/시물 흐름, 그리고 검증 정확성을 좌우한 **Vivado** 버전 이슈를 다룹니다.

21.1 프로젝트 개요

항목	값
도구	Vivado 2025.x
FPGA	xc7a100tcsg324-1 (Nexys A7-100T)
Top 모듈	yolo_engine

yolohw/fpga/ 구성:

파일	역할
create_project_25.tcl	Vivado 2025 프로젝트 생성 (권장)
gen_bram_ips.tcl	BRAM IP 생성
NEXYS_A7_100T.xdc	핀·클럭 제약
vivado_yolohw/	생성된 프로젝트
vitis/	MicroBlaze 펌웨어 (Phase 3)

21.2 프로젝트 생성—create_project_25.tcl

create_project_25.tcl 이 깨끗한 새 프로젝트를 만듭니다.

```
# Windows Vivado 2025.x Tcl Console
set fpga_dir "Z:/2026 CAU/AIX2026/git/Yolo_Accelerator/yolohw/fpga"
source $fpga_dir/create_project_25.tcl
```

[코드] 핵심—합성 fileset 에만 매크로 주입

```
add_files -norecurse [glob $origin_dir/./src/*.v] # RTL 추가
set_property verilog_define {FPGA=1} [get_filesets sources_1] # 합성에만 FPGA 매크로
```

8 장 8.8 에서 본 매크로 문제를 우아하게 해결합니다: 파일의 `define FPGA 는 주석으로 두고 (시물 기본), 합성 **fileset(sources_1)** 에만 **FPGA=1** 을 주입합니다. 이러면: - 합성 → FPGA 매크로 ON (실제 IP) - 시물 → 매크로 OFF (behavioral)

이 분리가 파일을 수정하지 않고 자동으로 됩니다. 매크로를 깜빡 켜두거나 꺼두는 실수를 방지합니다 (8 장 8.8).

21.3 BRAM IP —gen_bram_ips.tcl

gen_bram_ips.tcl 이 3 장 3.4 의 BRAM(BMG IP) 을 생성합니다. 합성 시 15 장의 버퍼들이 이 IP 로 실체화됩니다.

IP	종류	크기	용도
dpram_4096x72	Simple DP	쓰기 72 / 읽기 288	가중치 (비대칭)
spram_2560x32	Single	32×2560	bias
dpram_2048x128_tdp	True DP	128×2048	라인 버퍼 (×4)
dpram_65536x32	Simple DP	32×65536	출력 버퍼 (256KB)

[핵심] 두 가지 중요 설정: - 모두 **“No Output Register”** → read latency **1** 클럭 (3 장 3.4, 4 장 4.4). 15 장 시물 모델의 N_DELAY=1 과 일치. output register 를 켜면 latency 2 가 되어 14 장 14.7 의 정렬이 깨집니다. - dpram_4096x72 는 쓰기 72/읽기 288 **비대칭** (12 장 12.4) → 한 번 읽으면 가중치 36 개.

21.4 제약—NEXYS_A7_100T.xdc

NEXYS_A7_100T.xdc 는 보드의 핀 매핑과 클럭 제약을 정의합니다. 클럭 입력, 리셋, LED(network_done_led) 를 FPGA 핀에 연결합니다. 실제 클럭 주파수는 구현 후 확정되며 18 장 18.7 의 fps 추정을 좌우합니다.

21.5 합성 / 시물 흐름

```
# 합성 (FPGA 매크로 자동 ON)
launch_runs synth_1 -jobs 4
launch_runs impl_1 -to_step write_bitstream -jobs 4

# 시물 (FPGA 매크로 OFF, behavioral)
set_property top l18_verify_tb [get_filesets sim_1]
launch_simulation
```

- 합성: DSP48·BRAM IP 를 명시적으로 사용 (3 장 3.3~3.4).
- 시물: log_all_signals OFF 유지 (디스크 폭증 방지, 8 장 8.7).

21.6 Vivado 2021 vs 2025 —검증 정확성의 함정

증상

L18 chain 검증에서 같은 RTL·데이터·TB 인데 환경에 따라 결과가 0 ↔ 3360 mismatch 로 달라졌습니다.

Vivado	L18 chain 결과
2025	0 mismatch [O]
2021	3360 mismatch

원인—uninitialized 메모리 (X) 의 버전별 처리 차이

4 장 4.7 에서 본 X(unknown) 가 범인이었습니다. 초기화 안 된 behavioral 메모리의 X 를 Vivado 버전마다 다르게 처리한 것입니다.

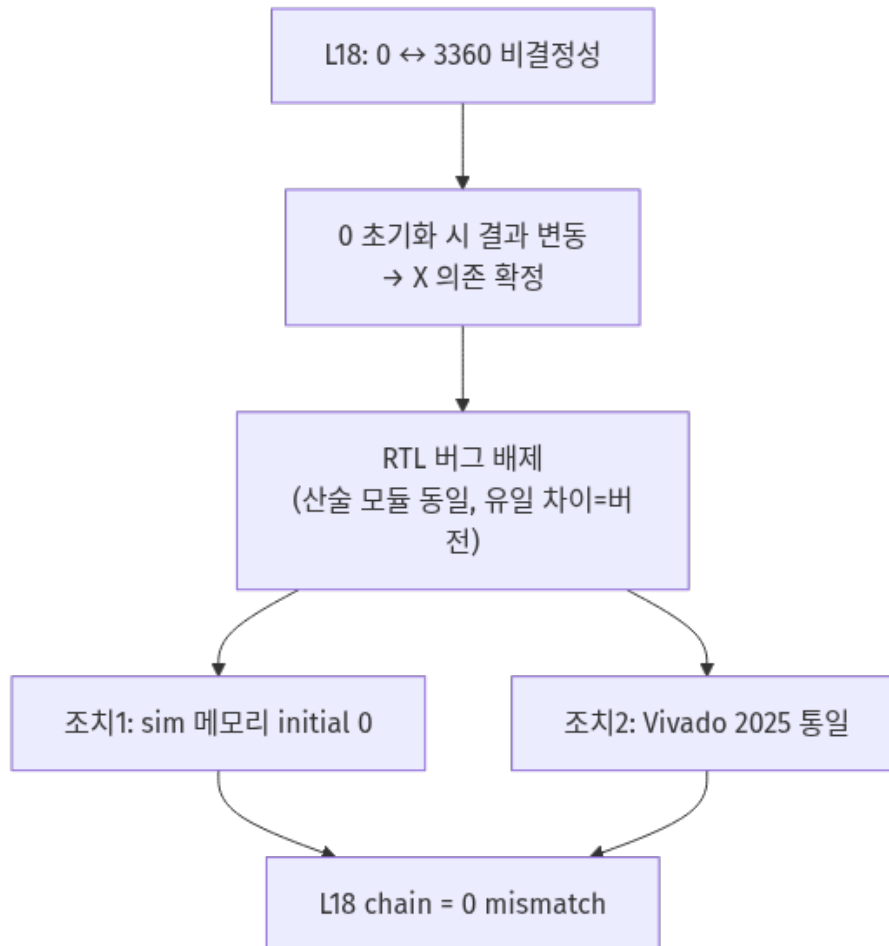
결정적 증거: 메모리를 0 으로 강제 초기화하니 2021 결과가 3360 → **3676** 으로 바뀜. 즉 chain 결과가 미초기화 메모리 값에 의존한다는 직접 증거.

RTL 버그가 아님을 확인한 과정

검증	결과
golden 자가 일관성	100% (인접 레이어 출력 == 입력)
다른 PC 와 산술 모듈 비교	mul/mac_kern/...완전히 동일 (byte-identical)
입력·골든·TB 비교	모두 동일
유일한 차이	Vivado 버전

해결

1. 시뮬 메모리를 **initial 0** 초기화 (15 장 15.3): dpram_wrapper, spram_wrapper, ifm_line_buf, gbuff_param. 합성에는 영향 없고 (BMG IP 경로), 실제 FPGA BRAM 의 0 초기화와 일치 → **fps·Energy·정확도 무관**.
2. 검증을 **Vivado 2025** 로 통일.



[핵심] **교훈**: mismatch 가 보인다고 항상 RTL 버그는 아닙니다. 시뮬레이션 환경 (초기화·버전) 도 원인일 수 있습니다. 산술 모듈이 동일하고 유일한 차이가 도구 버전이라면, 환경을 의심해야 합니다. 이 결론으로 RTL 은 정확하며 ± 1 LSB 는 점수 (mAP/fps/Energy) 에 무관함이 확인되었습니다 (HISTORY 2026-05-24).

21.7 이 장의 요약

- Vivado **2025** + xc7a100tcsg324-1, top=yolo_engine. create_project_25.tcl 이 새 프로젝트 생성.
- 합성 fileset 에만 verilog_define FPGA=1 주입 → 파일 수정 없이 합성/시뮬 분리.
- BMG IP 4 종 (가중치 비대칭, 출력 256KB, 라인버퍼 등), 모두 **No Output Register = read latency 1**.
- Vivado 2021 은 uninitialized X 처리가 달라 chain 비결정성 → sim 메모리 0 초기화 + 2025

통일로 해결.

- 교훈: mismatch $\neq$ 항상 RTL 버그. 환경 (초기화·버전) 도 의심하라.

다음 장 (부록) 에서는 아직 검증 중인 L19~L21, Phase 3·4, 용어집을 정리합니다.

22 장. 부록—향후 작업 / 용어집

이 장은 현재 해설서 범위 (**L0~L18 검증**) 밖의 내용과, 학습에 도움이 되는 용어집·참고 자료를 모읍니다. L19~L21 과 Phase 3·4 가 완료되면 해당 절을 본문으로 승격할 예정입니다.

22.1 L19~L21 —검증 진행 중

L19 Route concat

L20 입력은 업샘플된 L18($16 \times 16 \times 128$) 과 백본 L8($16 \times 16 \times 256$) 을 채널 축으로 이어 붙인 $16 \times 16 \times 384$ 입니다 (6 장 6.6 skip connection). RTL 은 16 장 16.5 의 2-source REPACK 으로 처리합니다.

- L18 출력을 출력 버퍼 앞쪽에, L8 출력을 뒤쪽에 적재 → NHWC entry 로 묶어 L20 입력 생성.
- 검증 TB: l20_verify_tb.v.
- L8 출력 골든은 별도 생성 필요 (CONV20_input.hex 의 채널 128~383).

L20 검출 헤드 2

L14 와 같은 검출 헤드 구조 (linear, ReLU off) 이며, $C_i=384$, $C_o=195$, 16×16 격자. **descale shift = 9**(L20 은 다음 합성곱이 없어 next=1, 2 장 2.6, 9 장 9.4). post_process 의 toward-zero 보정도 적용됩니다 (14 장 14.6).

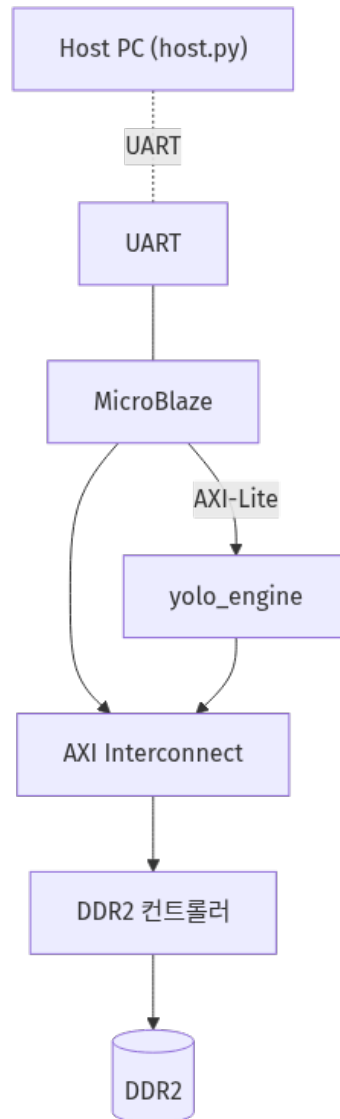
L21 YOLO output

RTL 동작 없음. L20 출력이 DRAM 에 있고 소프트웨어가 후처리합니다 (6 장 6.8).

승격 조건: L20 chain(Phase B) 이 0 mismatch 가 되면, 이 절을 10·14·20 장 본문으로 통합합니다.

22.2 Phase 3 —MicroBlaze + UART + DDR2

시스템 통합 (예정)



17 장 17.6 에서 본 통합입니다. Vivado block design 으로 MicroBlaze + UART + DDR2 + yolo_engine 을 연결하고, Vitis 펌웨어가 제어합니다.

host.py —Host PC 클라이언트

host.py 는 UART 로 가속기를 구동합니다 (5 장 5.1).

명령 (MODE):

MODE	값	역할
STORE_RAM	0x03	DRAM 에 데이터 적재
RUN_ENGINE	0x06	22-layer 추론 + 완료 대기

MODE	값	역할
CONCAT_L19	0x07	(레거시) L19 가 RTL REPACK 으로 처리되어 single-inference 에서는 미사용
YOLO_POST	0x08	Sigmoid/NMS → 검출 목록

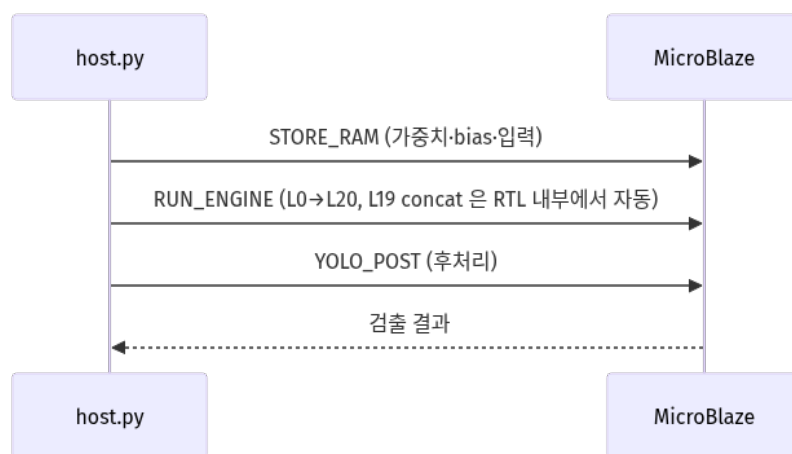
DRAM 메모리 맵:

```

0x0000_0000  weight
0x00A0_0000  bias
0x0100_0000  입력 이미지
0x0200_0000  출력 (레이어별)

```

추론 시퀀스 (한 번 추론—L19 는 RTL 이 처리):



[왜?] 왜 한 번이면 될까?: L19 concat 은 L8(추론 초반 생성) 과 L18(추론 후반 생성) 을 합치는데 (6 장 6.6), 이 합치기를 하드웨어 (**yolo_engine FSM 의 REPACK 단계**) 가 추론 도중 자동으로 수행합니다. 그래서 한 번의 추론 (L0→L20) 으로 L20 까지 끝납니다. 과거에는 software 가 L8 을 L18 뒤에 복사한 뒤 두 번째 추론을 했지만 (double-inference), 이제는 불필요합니다. 보드 통합 후 이 흐름의 타이밍·전력이 Phase 4 에서 측정됩니다.

22.3 Phase 4 —비트스트림 / 보드 / 측정

작업	산출물
합성 → 구현 → 비트스트림	.bit (21 장 21.5)
보드 프로그래밍	Nexys A7-100T

작업	산출물
100 장 테스트셋 추론	mAP, fps, 전력 측정
점수 산출	$10^4 / \text{Energy} \times \text{ReLU}(\text{mAP} - 0.2) \times \text{ReLU}(\text{fps} - 5)$ (7 장 7.2)

이 단계에서 18 장 18.7~18.8 의 fps·Energy 추정이 실측으로 확정됩니다. 결과는 새 장 (“23 장 보드 측정 결과”) 으로 추가할 예정입니다.

22.4 용어집

처음 보는 용어를 한곳에 모았습니다. 자세한 설명은 링크된 장에.

용어	쉬운 설명	장
추론 (inference)	학습된 신경망으로 새 입력의 답을 계산	1 장
특징맵 (feature map)	H×W×C 모양의 데이터 덩어리	1 장
합성곱 (convolution)	커널을 슬라이딩하며 곱하고 더하기	1 장
MAC	Multiply-Accumulate, “곱하고 더하기”한 번	1 장
양자화	실수를 정수 (INT8) 로 바꾸기	2 장
downscaling	부풀린 결과를 시프트로 되돌리기	2 장
LUT / FF / DSP48 / BRAM	FPGA 기본 부품 (논리/기억/곱셈/메모리)	3 장
FSM	상태 머신, 하드웨어의 “프로그램 흐름”	4 장
파이프라인	계산을 여러 클럭으로 쪼갬	4 장
latency / throughput	지연 / 단위 시간당 처리량	4 장
AXI	FPGA 표준 버스 프로토콜	5 장
DMA	CPU 없이 하드웨어가 메모리 옮기기	5 장
anchor / NMS	박스 후보 / 중복 박스 제거	6 장
NCHW / NHWC entry / 2×2 packed	세 가지 특징맵 저장 포맷	12 장
REPACK	포맷을 다음 합성곱 입력으로 재배치	12·16 장
streaming weight	필터마다 가중치를 새로 가져오기	15 장
cyclic row mapping	라인 버퍼가 줄을 순환 배치 (row mod 4)	15 장
look-ahead	BRAM 지연 보상 위해 주소 미리 보내기	4·14 장
Phase A / B	단독 / chain 검증	19 장

용어	쉬운 설명	장
tolerance	mismatch 허용 한도 (± 1 LSB)	19 장

22.5 약어

약어	풀이
IFM / OFM	Input / Output Feature Map (입력/출력 특징맵)
MAC	Multiply-Accumulate
BRAM / DPRAM / SPRAM	Block / Dual-Port / Single-Port RAM
BMG	Block Memory Generator (BRAM IP 생성기)
DSP48	Xilinx 곱셈 전용 블록
FSM	Finite State Machine
NMS	Non-Maximum Suppression
mAP	mean Average Precision (검출 정확도)
LSB	Least Significant Bit (최하위 비트)

22.6 참고 자료와 우선순위

자료	위치	용도
기술 레퍼런스	../technical_reference/	간결·정확한 빠른 참조 (AI·숙련 자용)
CLAUDE.md	../CLAUDE.md	작업 규칙 8 가지
HISTORY.md	../HISTORY.md	검증·버그 수정 이력
강의 PDF 12 종	../	DSP/MAC, BRAM, System Integration

충돌 시 우선순위: 실제 소스 코드 > 기술 레퍼런스 > 본 해설서 > 강의 PDF. 본 해설서는 “이해를 돕는” 것이 목적이라, 정밀한 사실은 소스·기술 레퍼런스로 교차 확인하세요.

강의 PDF 중 직접 관련된 것: - DSP/MAC → 3·14 장 - BRAM → 3·15 장 - System Integration → 13·17 장 - Quantization → 2·9 장

22.7 문서 유지보수 가이드

이 해설서를 최신으로 유지하려면:

1. **L19~L21 검증 완료 시:** 22.1 을 10·14·20 장 본문으로 통합.
 2. **Phase 3 보드 통합 시:** 22.2 를 새 장으로 승격.
 3. **Phase 4 측정 후:** 새 장에 fps/mAP/Energy/Score 실측 추가, 18 장 추정을 실측으로 교체.
 4. **RTL 변경 시:** 해당 모듈 장 (13~17 장) 의 코드 인용·설명을 갱신.
 5. **기술 레퍼런스와 동기화:** 같은 사실을 다루므로 한쪽을 고치면 다른 쪽도 점검.
-

22.8 맺음말—전체 한 장으로

이 해설서를 다 읽었다면, 이제 아래 한 문단이 모두 이해될 것입니다.

이 가속기는 **256×256** 이미지를 **INT8** 로 양자화 ([2 장]) 하여, **144** 개의 **MAC** ([14 장]) 으로 **22** 개 레이어의 합성곱·풀링·업샘플 ([1 장]) 을 순서대로 계산한다 ([13·18 장]). 가중치와 특징맵은 너무 커서 **DRAM** 에 두고 **AXI DMA** 로 필요한 부분만 **BRAM** 에 올려 재사용하며 ([5·15 장]), 출력은 두 개의 **YOLO** 검출 헤드 ([6 장]) 를 거쳐 객체의 위치와 종류를 찾는다. 모든 **RTL** 은 **skeleton C** 와 비트 단위로 일치하도록 검증되었고 ([9·19·20 장]), **INT8** 정수 연산과 버퍼 재사용으로 에너지를 최소화하여 점수를 극대화한다 ([7·18 장]).

각 괄호의 장으로 돌아가면 그 부분을 깊이 들여다볼 수 있습니다. L21 까지의 검증과 보드 동작이 완료되면, 이 해설서는 그 결과를 담아 완성됩니다.
